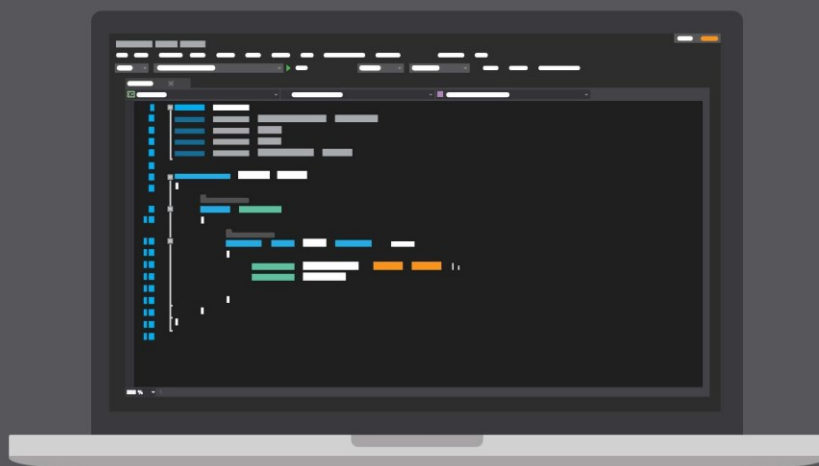




TRƯỜNG CAO ĐẲNG KỸ THUẬT CAO THẮNG
KHOA CÔNG NGHỆ THÔNG TIN
BỘ MÔN CÔNG NGHỆ PHẦN MỀM

GIÁO TRÌNH THỰC TẬP LẬP TRÌNH TRÊN MÔI TRƯỜNG WINDOWS



Biên soạn:
Nguyễn Đức Chuẩn
Trần Quang Khải
Lê Hữu Vinh

MỤC LỤC

MỤC LỤC

NỘI QUY PHÒNG MÁY KHOA CÔNG NGHỆ THÔNG TIN

BÀI TẬP THỰC HÀNH TUẦN 01 NGÔN NGỮ C# (CONSOLE)	1
1. Hướng dẫn sử dụng Visual Studio 2022 để lập trình C# cơ bản	1
a. Tạo project Visual C#	1
b. Biên dịch và thực thi chương trình	4
2. C# cơ bản	5
a. Phương thức xuất	5
b. Ghi chú (comment)	5
c. Biến	6
d. Kiểu dữ liệu	7
e. Phương thức nhập	8
f. String	8
g. Các cấu trúc điều khiển	10
BÀI TẬP THỰC HÀNH TUẦN 02 NGÔN NGỮ C# (CONSOLE)	21
1. Phương thức	21
a. Tạo phương thức	21
b. Gọi phương thức	21
2. C# tham số của phương thức	22
a. Tham số và đối số (Parameters and Arguments)	22
b. Giá trị tham số mặc định	22
c. Nhiều tham số	23
d. Giá trị trả về	23
e. Đối số được đặt tên	24
f. Nạp chồng phương thức	24
BÀI TẬP THỰC HÀNH TUẦN 03 LẬP TRÌNH WINDOWS FORM	29
1. Tạo ứng dụng Windows Form	29
2. Các control cơ bản	31
a. Label	31
b. Button	32
c. Textbox	33
d. Groupbox	35

BÀI TẬP THỰC HÀNH TUẦN 04 CONTROLS	
(CHECKBOX, RADIO BUTTON, LISTBOX, COMBOBOX)	41
1. Checkbox.....	41
2. Radio Button	41
3. ListBox	42
4. Checked ListBox	45
5. ComboBox.....	48
BÀI TẬP THỰC HÀNH TUẦN 05 CONTROLS	
(NUMERICUPDOWN, DATETIMEPICKER, LISTVIEW, PICTUREBOX, IMAGELIST).....	52
1. NumericUpDown.....	52
2. DateTimePicker.....	53
3. Listview	54
4. PictureBox.....	56
5. ImageList.....	57
BÀI TẬP THỰC HÀNH TUẦN 06 CONTROLS	
(MENU, MDI, TABCONTROL, TREEVIEW, PANEL)	60
1. Menu	60
2. MDI.....	62
3. TabControl	62
4. TreeView	64
5. Panel.....	66
KIỂM TRA LẦN 01.....	69
BÀI TẬP THỰC HÀNH TUẦN 08 ADO.NET	74
BÀI TẬP THỰC HÀNH TUẦN 09 ADO.NET (Command).....	82
BÀI TẬP THỰC HÀNH TUẦN 10 ADO.NET (Store Procedure)	86
BÀI TẬP THỰC HÀNH TUẦN 11 ADO.NET (Entity Framework).....	89
BÀI TẬP THỰC HÀNH TUẦN 12 MÔ HÌNH 3 LỚP (3 – LAYERS)	95
BÀI TẬP THỰC HÀNH TUẦN 13 REPORT (1)	98
BÀI TẬP THỰC HÀNH TUẦN 14 REPORT (2).....	106
KIỂM TRA LẦN 02	111

NỘI QUY PHÒNG MÁY KHOA CÔNG NGHỆ THÔNG TIN

HSSV khi thực hành tại phòng máy cần thực hiện nghiêm chỉnh các nội quy sau đây:

1. Học sinh sinh viên không được tự ý vào phòng máy khi chưa được sự đồng ý của giảng viên hoặc người hướng dẫn.
2. Đi học đúng giờ, đúng ca thực hành theo quy định. Phải mặc đồng phục, mang giày hoặc dép có quai hậu theo quy định của nhà trường.
3. Đầu ca thực hành, sinh viên có nhiệm vụ *“kiểm tra tất cả các thiết bị phòng máy, ghi vào sổ kiểm tra phòng máy và chịu trách nhiệm về tài sản trong phòng máy”*.
4. Để cặp sách trên bục giảng, ngồi đúng sơ đồ phòng máy.
5. Không được sử dụng máy giảng viên khi không có yêu cầu.
6. Không được hút thuốc, ăn uống xả rác gây mất vệ sinh trong phòng máy.
7. Không được sử dụng điện thoại di động, chơi game và làm việc riêng trong phòng máy. Không đùa giỡn gây mất trật tự trong phòng máy.
8. Không tự ý di chuyển thiết bị (chuột, bàn phím, máy tính). Không tự ý rút dây cáp nguồn và dây cáp mạng của máy tính trong phòng máy.
9. Không viết vẽ lên các thiết bị của máy tính, bàn ghế và tường. Không tự ý đổi mật khẩu, phát tán virus và cài đặt các chương trình khi chưa có sự cho phép của giảng viên.
10. Cuối ca thực hành phải sắp xếp lại ghế, chuột, bàn phím theo đúng vị trí. Đóng tất cả các cửa sổ và tắt điện trong phòng máy, *“kiểm tra lại thiết bị trong phòng máy và xác nhận vào sổ kiểm tra phòng máy”*.

**TÙY THEO MỨC ĐỘ VI PHẠM SẼ CÓ HÌNH THỨC KỶ LUẬT PHÙ HỢP
KHOA CÔNG NGHỆ THÔNG TIN**

BÀI TẬP THỰC HÀNH TUẦN 01

NGÔN NGỮ C# (CONSOLE)



Mục tiêu

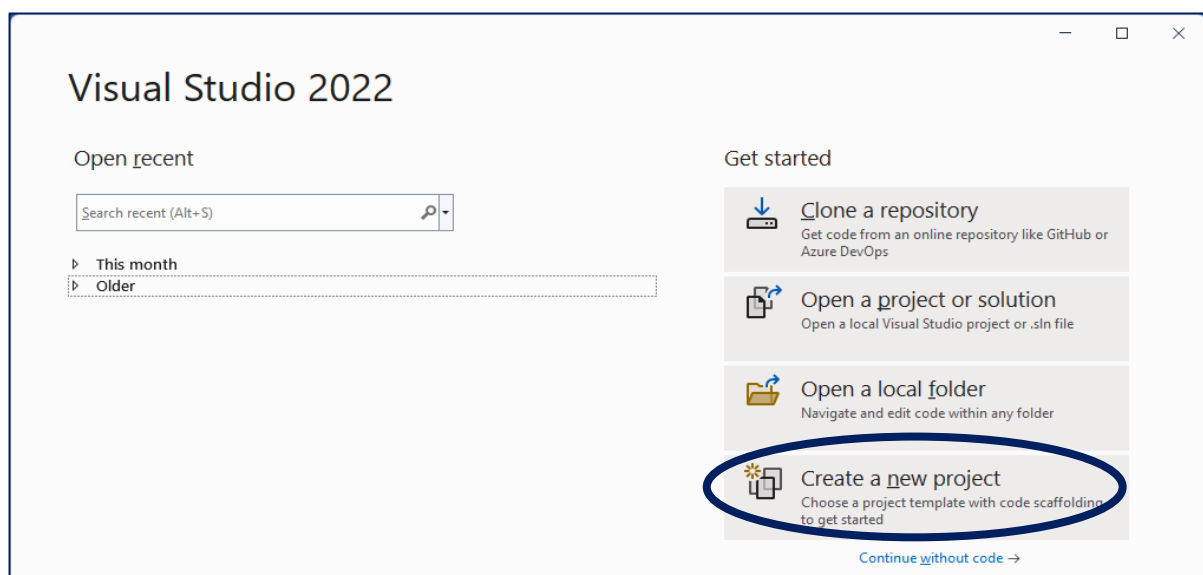
- Làm quen với môi trường console của ngôn ngữ lập trình C#
- Tìm hiểu cấu trúc chương trình được viết bằng ngôn ngữ C#
- Tìm hiểu cú pháp lệnh của ngôn ngữ C#
- Tìm hiểu các cấu trúc điều khiển của ngôn ngữ C#
- Viết được các chương trình chạy ở chế độ console bằng ngôn ngữ C#

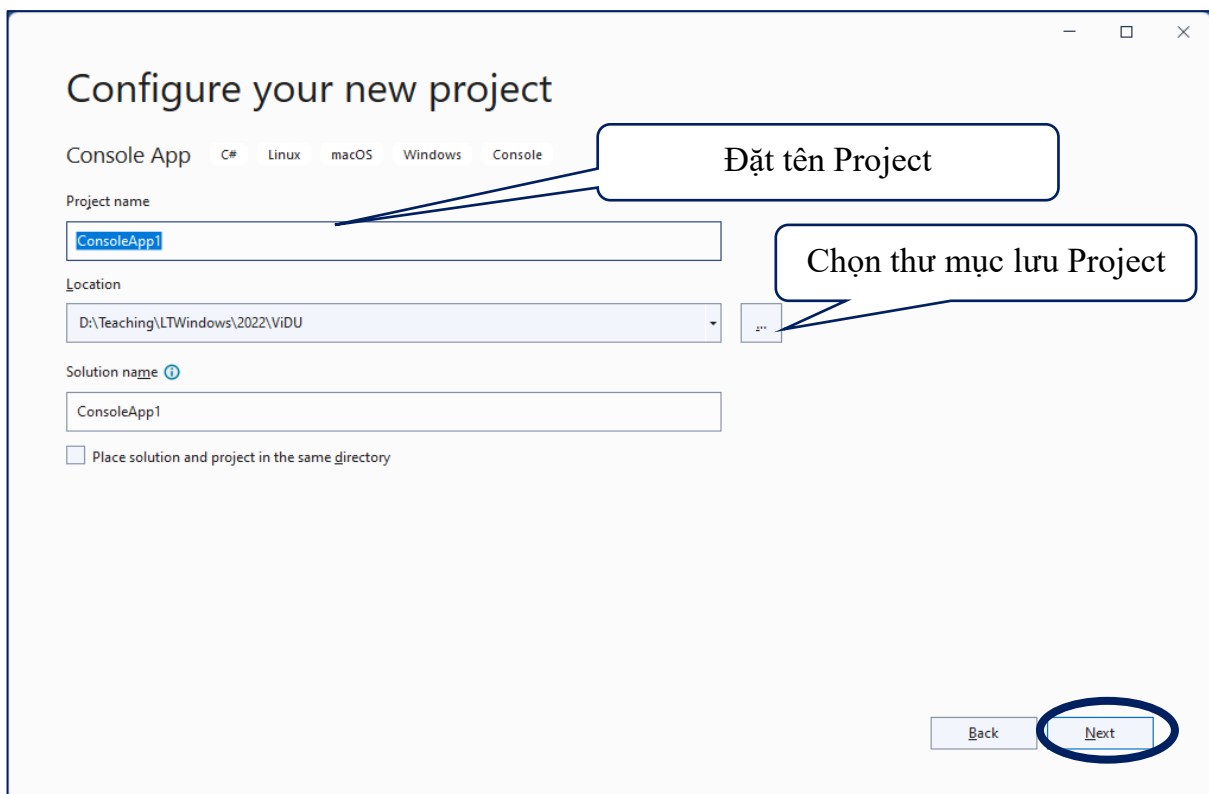
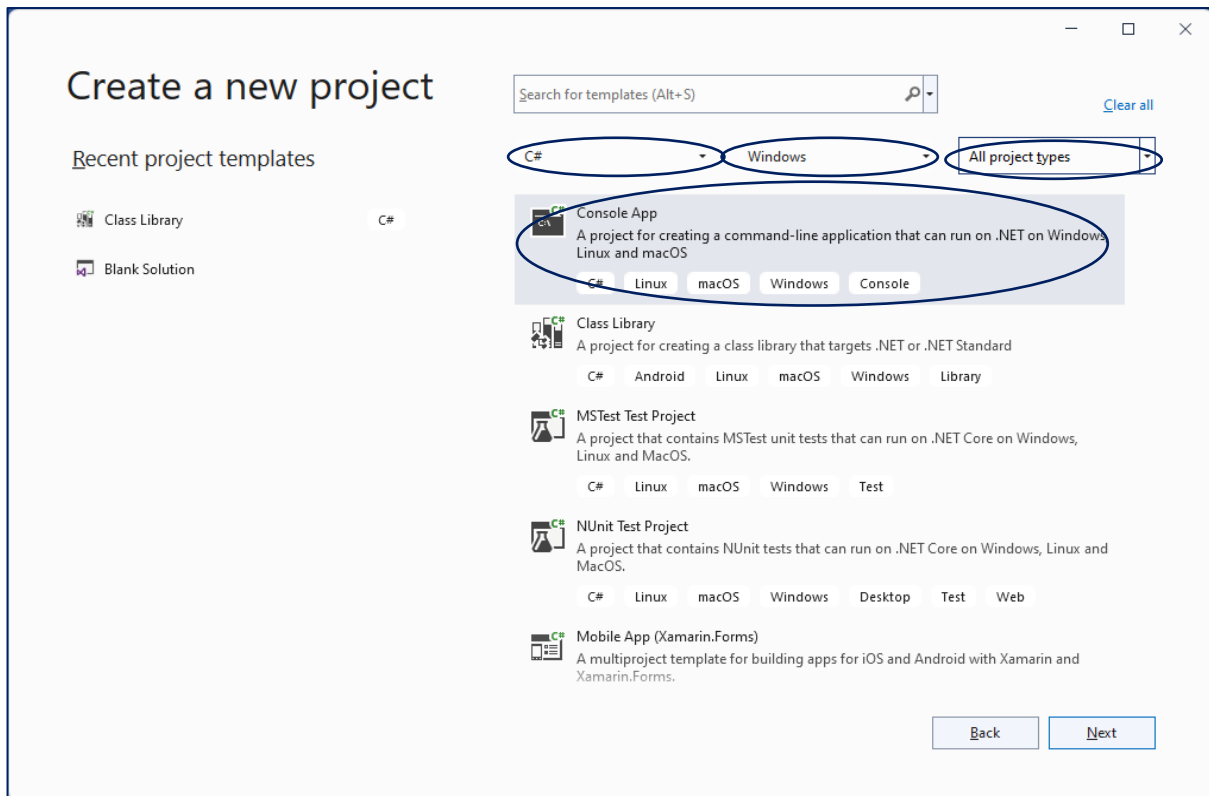
Kiến thức

1. Hướng dẫn sử dụng Visual Studio 2022 để lập trình C# cơ bản

a. Tạo project Visual C#

- Mở Visual Studio 2022





Additional information

Console App C# Linux macOS Windows

Framework ⓘ
.NET 6.0 (Long-term support)

☒ Do not use top-level statements ⓘ

```
using System;

namespace Application
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("Hello World!");
        }
    }
}
```

Back Create

Additional information

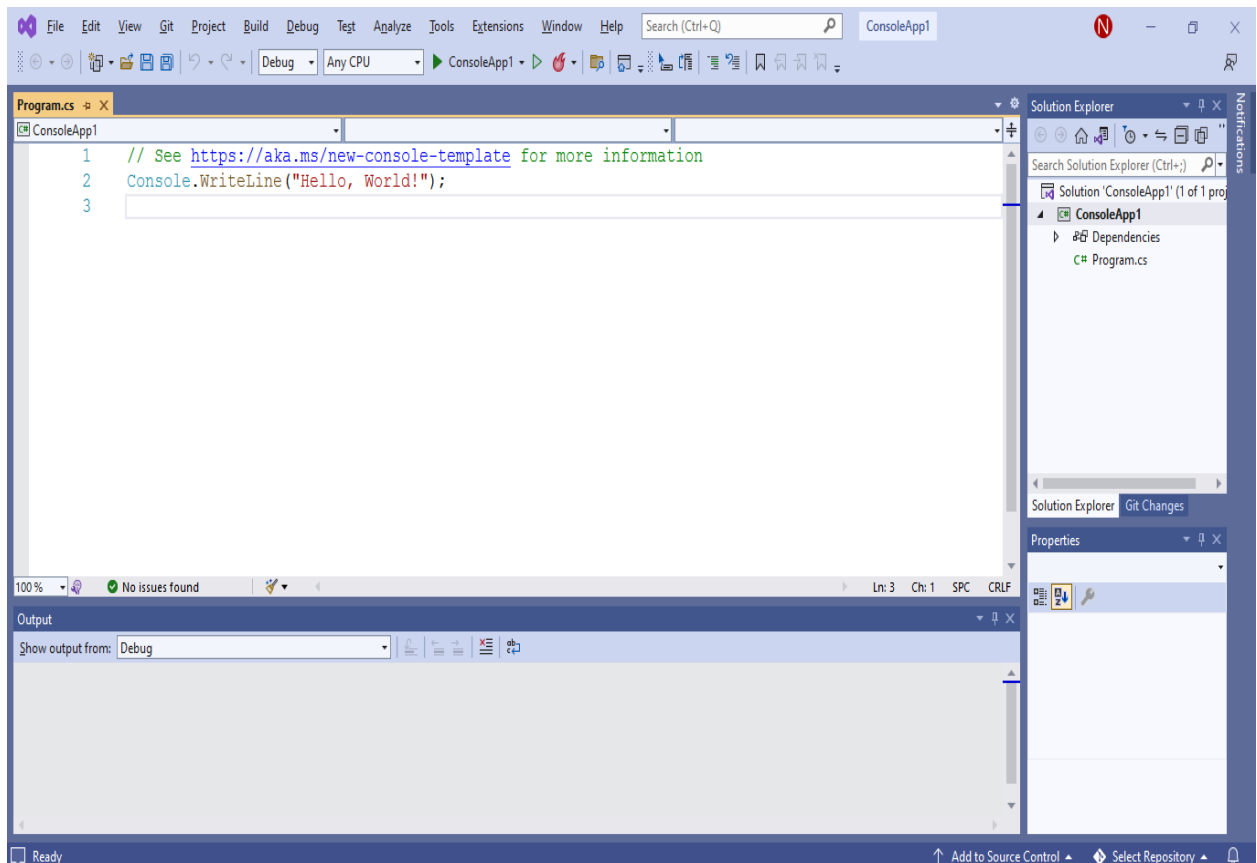
Console App C# Linux macOS Windows Console

Framework ⓘ
.NET 6.0 (Long-term support)

☐ Do not use top-level statements ⓘ

```
// See https://aka.ms/new-console-template for more information
Console.WriteLine("Hello, World!");
```

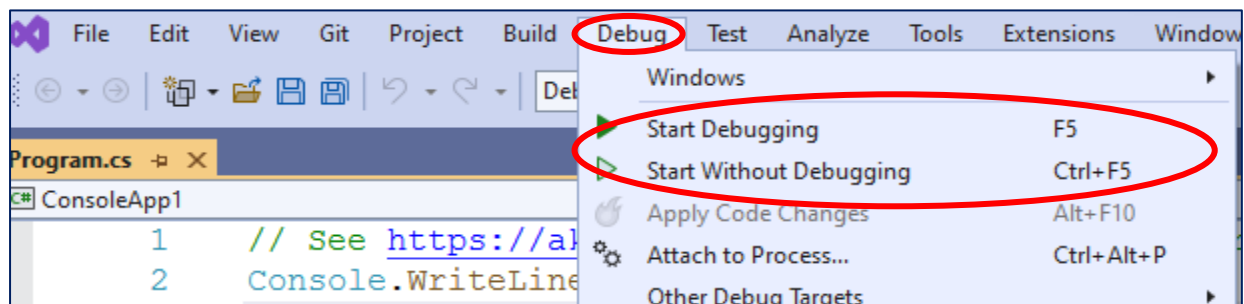
Back Create



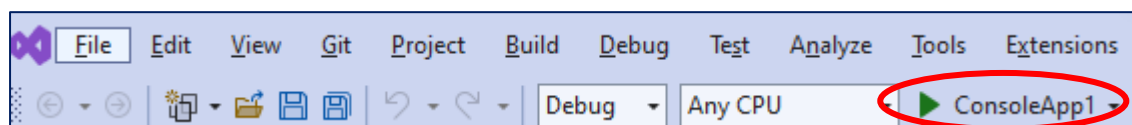
```
// See https://aka.ms/new-console-template for more information
Console.WriteLine("Hello, World!");
```

b. Biên dịch và thực thi chương trình

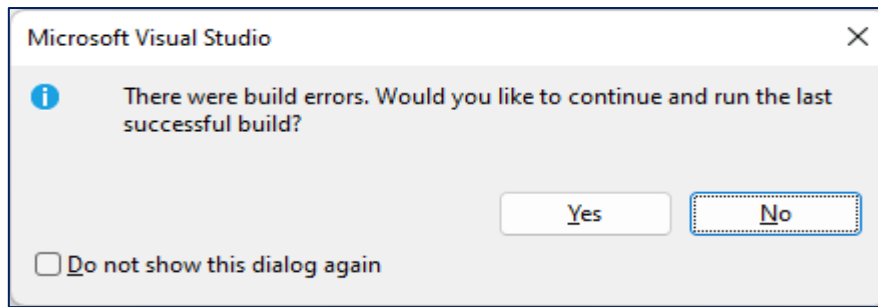
- Thực hiện **một** trong các thao tác sau:
 - + Nhấn phím F5 hoặc CTRL + F5
 - + Chọn menu Debug → Start Debugging



+ Nhấn vào nút  trên thanh công cụ

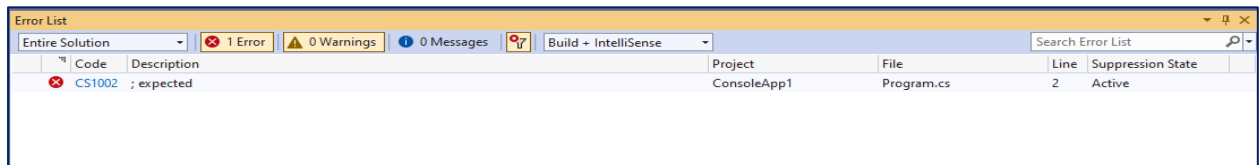


- Nếu chương trình có lỗi xảy ra thì hiện thông báo như hình bên dưới

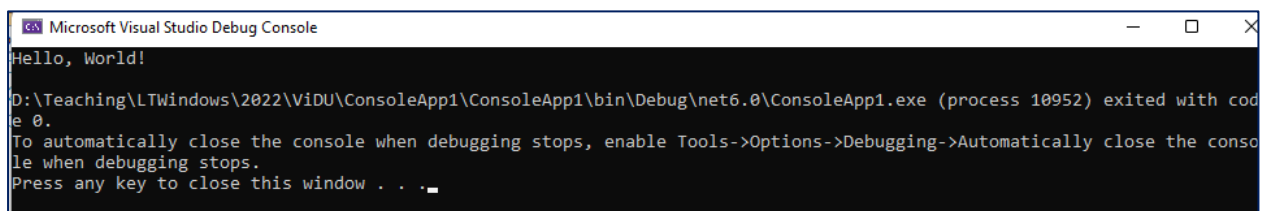


→ nên chọn **No** để tiến hành sửa lỗi chương trình rồi biên dịch lại.

Danh sách lỗi được hiển thị trong khung **Error list**



- Nếu chương trình không có lỗi sẽ thực thi như hình bên dưới



2. C# cơ bản

a. Phương thức xuất

Để xuất giá trị hoặc in văn bản trong C #, có thể sử dụng phương thức **WriteLine ()** hoặc **Write()**. Sự khác biệt giữa phương thức WriteLine và Write đó là phương thức WriteLine sẽ thêm dòng mới ở cuối nội dung xuất.

Ví dụ:

```

Console.WriteLine("Hello, World!");
Console.Write("Hello, World!");
  
```

b. Ghi chú (comment)

Ghi chú trên 1 dòng: ghi chú một dòng bắt đầu bằng hai dấu gạch chéo về phía trước (//)

Ví dụ:

```

//Day là ghi chú
Console.WriteLine("Hello World!"); // Day là ghi chú
  
```

Ghi chú nhiều dòng: ghi chú nhiều dòng bắt đầu bằng /* và kết thúc bằng */

Ví dụ:

```

/* Đoạn lệnh dưới đây sẽ xuất ra màn hình
   tu Hello World! */
Console.WriteLine("Hello World!");
  
```

c. Biến

Biến là vùng nhớ trong máy tính được sử dụng để chứa giá trị dữ liệu.

Trong C#, có các kiểu dữ liệu khác nhau của biến (được xác định bởi các từ khóa khác nhau):

Từ khóa	Kiểu dữ liệu
<code>int</code>	Lưu trữ số nguyên
<code>double</code>	Lưu trữ số thực
<code>char</code>	Lưu trữ các ký tự đơn
<code>string</code>	Lưu trữ văn bản
<code>bool</code>	Lưu trữ giá trị với 2 trạng thái: true hoặc false

Khai báo (tạo) biến

Để tạo một biến, phải chỉ định kiểu và gán cho nó một giá trị.

Cú pháp:

`<Kiểu dữ liệu> <tên biến> = <giá trị>;`

Ví dụ:

```
string name = "John";
Console.WriteLine(name);
```

Hằng số

Có thể thêm từ khóa **const** nếu không muốn người khác (hoặc chính mình) ghi đè các giá trị hiện có (điều này sẽ khai báo biến là "hằng số", có nghĩa là không thể thay đổi và chỉ đọc)

Ví dụ:

```
const int myNum = 15;
myNum = 20; // error
```

Từ khóa **const** rất hữu ích khi muốn một biến luôn lưu trữ cùng một giá trị để những người khác (hoặc chính người lập trình) không làm rối mã. Một ví dụ thường được gọi là hằng số, là PI (3,14159 ...)

Định danh

Tất cả các biến C# phải được xác định bằng các tên duy nhất. Những tên duy nhất này được gọi là **định danh**.

Các quy tắc chung để đặt tên biến là:

- Tên có thể chứa các chữ cái, chữ số và ký tự gạch dưới
- Tên phải bắt đầu bằng một chữ cái
- Tên phải bắt đầu bằng chữ thường và không được chứa khoảng trắng
- Tên có phân biệt chữ hoa chữ thường ("myVar" và "myvar" là các biến khác nhau)

- Các từ dành riêng (như từ khóa C #, chẳng hạn như **int** hoặc **double**) không thể được sử dụng làm tên

Ví dụ:

```
string firstName = "John ";
string lastName = "Doe";
string fullName = firstName + lastName;
Console.WriteLine(fullName);
```

d. Kiểu dữ liệu

Kiểu dữ liệu xác định kích thước và kiểu của các giá trị được lưu trong biến. Điều quan trọng là sử dụng đúng kiểu dữ liệu cho biến tương ứng; để tránh lỗi, tiết kiệm thời gian và bộ nhớ, đồng thời nó cũng sẽ làm cho mã của dễ bảo trì và dễ đọc hơn. Các kiểu dữ liệu phổ biến nhất là:

Kiểu dữ liệu	Kích thước	Diễn giải
int	4 bytes	Lưu trữ các số nguyên từ -2,147,483,648 đến 2,147,483,647
long	8 bytes	Lưu trữ các số nguyên từ -9,223,372,036,854,775,808 đến 9,223,372,036,854,775,807
float	4 bytes	Lưu trữ số phân số. Đủ để lưu trữ từ 6 đến 7 chữ số thập phân
double	8 bytes	Lưu trữ số phân số. Đủ để lưu trữ 15 chữ số thập phân
bool	1 bit	Lưu trữ giá trị true hoặc false
char	2 bytes	Lưu trữ một ký tự / chữ cái, được bao quanh bởi dấu nháy đơn
string	2 bytes	Lưu trữ một chuỗi ký tự, được bao quanh bởi dấu ngoặc kép

Chuyển đổi kiểu dữ liệu

Chuyển đổi kiểu dữ liệu là khi gán giá trị của một kiểu dữ liệu cho một kiểu khác.

Trong C #, có hai kiểu ép kiểu:

Chuyển đổi ngầm (tự động): chuyển đổi một kiểu nhỏ hơn sang kiểu có kích thước lớn hơn

char -> int -> long -> float -> double

Ví dụ:

```
int myInt = 9;
double myDouble = myInt; // Automatic casting: int to double
Console.WriteLine(myInt); // Outputs 9
Console.WriteLine(myDouble); // Outputs 9
```

Chuyển đổi rõ ràng (thủ công): chuyển đổi kiểu dữ liệu lớn hơn sang kiểu dữ liệu kích thước nhỏ hơn

double -> float -> long -> int -> char

Ví dụ:

```
double myDouble = 9.78;
int myInt = (int)myDouble; // Manual casting: double to int
Console.WriteLine(myDouble); // Outputs 9.78
```

```
Console.WriteLine(myInt); // Outputs 9
```

Các phương thức chuyển đổi kiểu dữ liệu

`Convert.ToBoolean`, `Convert.ToDouble`, `Convert.ToString`,
`Convert.ToInt32(int)` và `Convert.ToInt64(long)`

Ví dụ:

```
int myInt = 10;
double myDouble = 5.25;
bool myBool = true;

Console.WriteLine(Convert.ToString(myInt)); // convert int to
string
Console.WriteLine(Convert.ToDouble(myInt)); // convert int to
double
Console.WriteLine(Convert.ToInt32(myDouble)); // convert double to
int
Console.WriteLine(Convert.ToString(myBool)); // convert bool to
string
```

e. Phương thức nhập

Chúng ta sẽ sử dụng `Console.ReadLine ()` để lấy thông tin đầu vào của người dùng.

Ví dụ:

```
Console.WriteLine("Enter username:");
string userName = Console.ReadLine();
Console.WriteLine("Username is: " + userName);
```

Phương thức `Console.ReadLine ()` trả về một chuỗi (string). Do đó, không thể lấy thông tin từ một kiểu dữ liệu khác, chẳng hạn như `int`. Chương trình sau sẽ gây ra lỗi:

```
Console.WriteLine("Enter your age:");
int age = Console.ReadLine();
Console.WriteLine("Your age is: " + age);
```

Error List							
Entire Solution		1 Error	0 Warnings	0 Messages	Build + IntelliSense	Search Error List	
	Code	Description	Project	File	Line	Suppression State	
	CS0029	Cannot implicitly convert type 'string' to 'int'	ConsoleApp1	Program.cs	3	Active	

Có thể chuyển đổi bất kỳ kiểu nào một cách rõ ràng bằng cách sử dụng một trong các phương thức `Convert.To`

```
Console.WriteLine("Enter your age:");
int age = Convert.ToInt32(Console.ReadLine());
Console.WriteLine("Your age is: " + age);
```

f. String

String được sử dụng để lưu trữ văn bản. Một biến có kiểu string chứa một tập hợp các ký tự được bao quanh bởi dấu ngoặc kép.

Ví dụ:

```
string greeting = "Hello";
```

Chiều dài chuỗi: Một chuỗi trong C # thực sự là một đối tượng, chứa các thuộc tính và phương thức có thể thực hiện các thao tác nhất định trên chuỗi. Ví dụ: độ dài của một chuỗi có thể được tìm thấy với thuộc tính Length:

Ví dụ:

```
string txt = "ABCDEFGHIJKLMNOPQRSTUVWXYZ";
Console.WriteLine("The length of the txt string is: " + txt.Length);
```

Nối chuỗi: Toán tử + có thể được sử dụng giữa các chuỗi để kết hợp chúng. Đây được gọi là nối chuỗi.

Ví dụ:

```
string firstName = "John ";
string lastName = "Doe";
string name = firstName + lastName;
Console.WriteLine(name);
```

Cũng có thể sử dụng phương thức string.Concat () để nối hai chuỗi

```
string firstName = "John ";
string lastName = "Doe";
string name = string.Concat(firstName, lastName);
Console.WriteLine(name);
```

Nội suy chuỗi: Một tùy chọn khác của nối chuỗi, là nội suy chuỗi, thay thế các giá trị của các biến thành các trình giữ chỗ trong một chuỗi. Lưu ý không phải lo lắng về khoảng trắng.

Ví dụ:

```
string firstName = "John";
string lastName = "Doe";
string name = $"My full name is: {firstName} {lastName}";
Console.WriteLine(name)
```

Cũng lưu ý rằng phải sử dụng ký hiệu đô la (\$) khi sử dụng phương pháp nội suy chuỗi.

Truy xuất chuỗi

Có thể truy cập các ký tự trong một chuỗi bằng cách tham chiếu đến số chỉ mục (index) của nó bên trong dấu ngoặc vuông []

```
string myString = "Hello";
Console.WriteLine(myString[0]); // Outputs "H"
```

Cũng có thể tìm vị trí chỉ mục của một ký tự cụ thể trong một chuỗi, bằng cách sử dụng phương thức IndexOf ()

```
string myString = "Hello";
Console.WriteLine(myString.IndexOf("e")); // Outputs "1"
```

Một phương thức hữu ích khác là Substring (), nó trích xuất các ký tự từ một chuỗi, bắt đầu từ vị trí / chỉ mục của ký tự được chỉ định và trả về một chuỗi mới. Phương thức này thường được sử dụng cùng với IndexOf () để lấy vị trí ký tự cụ thể.

```
string name = "John Doe";
int charPos = name.IndexOf("D");
string lastName = name.Substring(charPos);
Console.WriteLine(lastName);
```

g. Các cấu trúc điều khiển

Câu lệnh if...else

Sử dụng câu lệnh if để chỉ định một khối mã C# sẽ được thực thi nếu một điều kiện là True.

Cú pháp:

```
if (condition)
{
    // khối lệnh thực hiện nếu condition là True
}
```

Ví dụ:

```
if (20 > 18)
{
    Console.WriteLine("20 is greater than 18");
}
```

Sử dụng câu lệnh else để chỉ định một khối mã sẽ được thực thi nếu điều kiện là False

```
if (condition)
{
    // khối lệnh được thực hiện nếu điều kiện là True
}
else
{
    // khối lệnh được thực hiện nếu điều kiện là False
}
```

Ví dụ:

```
int time = 20;
if (time < 18)
{
    Console.WriteLine("Good day.");
}
else
{
    Console.WriteLine("Good evening.");
}
```

Sử dụng câu lệnh else if để chỉ định một điều kiện mới nếu điều kiện đầu tiên là False

```
if (condition1)
{
    // khối lệnh được thực hiện nếu condition1 là True
}
else if (condition2)
{
    // khối lệnh được thực hiện nếu condition1 là false và
    condition2 là True
}
else
{
    // khối lệnh được thực hiện nếu condition1 là false và
    condition2 là False
}
```

Ví dụ

```
int time = 22;
if (time < 10)
{
    Console.WriteLine("Good morning.");
}
else if (time < 20)
{
    Console.WriteLine("Good day.");
}
else
{
    Console.WriteLine("Good evening.");
}
```

Dạng rút gọn if...else (toán tử 3 ngôi)

```
variable = (condition) ? expressionTrue : expressionFalse;
```

ví dụ

```
int time = 20;
string result = (time < 18) ? "Good day." : "Good evening.";
Console.WriteLine(result);
```

Câu lệnh switch

Sử dụng câu lệnh **switch** để chọn một trong nhiều khối lệnh sẽ được thực thi.

Cú pháp:

```
switch (expression)
{
    case x:
        // code block
        break;
    case y:
        // code block
        break;
    default:
        // code block
        break;
}
```

Ví dụ:

```
int day = 4;
switch (day)
{
    case 1:
        Console.WriteLine("Monday");
        break;
    case 2:
        Console.WriteLine("Tuesday");
        break;
    case 3:
        Console.WriteLine("Wednesday");
        break;
    case 4:
```

```
        Console.WriteLine("Thursday");  
        break;  
    case 5:  
        Console.WriteLine("Friday");  
        break;  
    case 6:  
        Console.WriteLine("Saturday");  
        break;  
    case 7:  
        Console.WriteLine("Sunday");  
        break;  
}
```

Từ khóa **break**

Khi C# gặp từ khóa **break**, nó sẽ thoát ra khỏi khối lệnh **switch**. Điều này sẽ dừng việc thực thi thêm các lệnh và kiểm tra trường hợp (**case**) bên trong khối lệnh. Khi giá trị của biểu thức trong **switch** trùng với giá trị của **case**, và các lệnh trong **case** đã thực hiện xong, đó là thời gian của lệnh **break**. Không cần kiểm tra thêm giá trị biểu thức trong **switch** với các **case** còn lại. **Break** có thể tiết kiệm rất nhiều thời gian thực thi vì nó "bỏ qua" việc thực thi tất cả phần còn lại của các lệnh trong khối **switch**.

Từ khóa **default**

Từ khóa **default** là tùy chọn và chỉ định một số lệnh để chạy nếu không có giá trị của **case** nào trùng khớp với giá trị biểu thức trong **switch**.

```
int day = 4;  
switch (day)  
{  
    case 6:  
        Console.WriteLine("Today is Saturday.");  
        break;  
    case 7:  
        Console.WriteLine("Today is Sunday.");  
        break;  
    default:  
        Console.WriteLine("Looking forward to the Weekend.");  
        break;  
}
```

Vòng lặp while

Các vòng lặp có thể thực thi một khối lệnh miễn là thỏa một điều kiện cụ thể.

Vòng lặp rất tiện dụng vì chúng tiết kiệm thời gian, giảm lỗi và giúp chương trình dễ đọc hơn.

Vòng lặp while lặp lại một khối lệnh miễn là một điều kiện được chỉ định là True

Cú pháp:

```
while (condition)  
{  
    // khối lệnh được thực thi  
}
```


Ví dụ: Trong ví dụ dưới đây, các lệnh trong vòng lặp (while) sẽ chạy đi chạy lại, miễn là một biến (i) nhỏ hơn 5

```
int i = 0;
while (i < 5)
{
    Console.WriteLine(i);
    i++;
}
```

Đừng quên tăng biến được sử dụng trong điều kiện, nếu không vòng lặp sẽ không bao giờ kết thúc!

Vòng lặp Do/While

Vòng lặp do/while là một biến thể của vòng lặp while. Vòng lặp này sẽ thực hiện khối lệnh một lần, trước khi kiểm tra xem điều kiện có đúng không, sau đó nó sẽ lặp lại vòng lặp miễn là điều kiện đúng.

Cú pháp:

```
do
{
    // khối lệnh được thực thi
}
while (condition);
```

Ví dụ: Ví dụ dưới đây sử dụng vòng lặp do/while. Vòng lặp sẽ luôn được thực hiện ít nhất một lần, ngay cả khi điều kiện sai, vì khối mã được thực thi trước khi điều kiện được kiểm tra

```
int i = 0;
do
{
    Console.WriteLine(i);
    i++;
}
while (i < 5);
```

Vòng lặp for

Khi biết chính xác số lần muốn lặp của một khối lệnh, hãy sử dụng vòng lặp for thay vì vòng lặp while.

Cú pháp:

```
for (statement 1; statement 2; statement 3)
{
    // khối lệnh được thực thi
}
```

statement 1 được thực hiện (một lần) trước khi khối lệnh được thực thi.

statement 2 xác định điều kiện để thực thi khối lệnh.

statement 3 được thực thi (mọi lúc) sau khi khối lệnh đã được thực thi

Ví dụ: Ví dụ dưới đây sẽ in các số từ 0 đến 4

```
for (int i = 0; i < 5; i++)
{
    Console.WriteLine(i);
}
```

Ví dụ: Ví dụ này sẽ chỉ in các giá trị chẵn từ 0 đến 10

```
for (int i = 0; i <= 10; i = i + 2)
{
    Console.WriteLine(i);
}
```

Vòng lặp foreach

Ngoài ra còn có một vòng lặp foreach, được sử dụng riêng để lặp qua các phần tử trong một mảng

Cú pháp:

```
foreach (<kiểu dữ liệu> <tên biến> in <tên mảng>)
{
    // khối lệnh được thực thi
}
```

Ví dụ: Ví dụ sau xuất ra tất cả các phần tử trong mảng cars, sử dụng vòng lặp foreach

```
string[] cars = { "Volvo", "BMW", "Ford", "Mazda" };
foreach (string i in cars)
{
    Console.WriteLine(i);
}
```

Break và continue

Break

Đã thấy câu lệnh break được sử dụng trong phần trước của hướng dẫn này. Nó được sử dụng để "nhảy ra" của một câu lệnh switch.

Câu lệnh break cũng có thể được sử dụng để nhảy ra khỏi vòng lặp.

Ví dụ: Ví dụ này nhảy ra khỏi vòng lặp khi i bằng 4

```
for (int i = 0; i < 10; i++)
{
    if (i == 4)
    {
        break;
    }
    Console.WriteLine(i);
}
```

Continue

Câu lệnh continue ngắt một lần lặp (trong vòng lặp), nếu một điều kiện cụ thể xảy ra và tiếp tục với lần lặp tiếp theo trong vòng lặp.

Ví dụ: Ví dụ này bỏ qua giá trị của 4

```
for (int i = 0; i < 10; i++)
{
    if (i == 4)
    {
        continue;
    }
    Console.WriteLine(i);
}
```

Break và continue trong vòng lặp while

Ví dụ break

```
int i = 0;
while (i < 10)
{
    Console.WriteLine(i);
    i++;
    if (i == 4)
    {
        break;
    }
}
```

Ví dụ continue

```
int i = 0;
while (i < 10)
{
    if (i == 4)
    {
        i++;
        continue;
    }
    Console.WriteLine(i);
    i++;
}
```

Bài tập vận dụng

Bài 1: Viết chương trình cho phép người dùng nhập vào 10 số nguyên. Tìm và xuất lên màn hình giá trị lớn nhất của 10 số vừa nhập. Chương trình chỉ sử dụng 3 biến sau:

counter: một bộ đếm từ 1 tới 10 (đếm số lượng giá trị đã nhập và xử lý)

number: giá trị được nhập hiện tại của chương trình

largest: số lớn nhất được tìm thấy

Bài 2: Cải tiến chương trình trên để tìm hai số lớn nhất của 10 số. [**Chú ý:** Chỉ được nhập mỗi giá trị một lần.]

Bài 3: Các tài xế thường quan tâm đến số quãng đường đi được với chiếc xe của họ. Một lái xe đã theo dõi một vài bình xăng bằng cách ghi lại số km đã đi và số xăng đã đổ vào bình xăng. Hãy viết chương trình nhập vào số km đi được và số lít xăng đã đổ vào bình. Chương trình sẽ tính toán và hiển thị số km trên một lít xăng với mỗi bình xăng. Sau khi thực hiện tất cả thông tin nhập, chương trình tính và xuất lên số km trên mỗi lít đạt được cho tất cả các bình xăng. Dưới đây là một ví dụ:

Hay nhap vao so lit xang da dung (-1 de ket thuc): 12.8

Hay nhap vao so km da chay: 287

So km/lit cho binh xang nay la: 22.421875

Hay nhập vào so lit xang da dung (-1 de ket thuc): 10.8

Hay nhập vào so km da chay: 200

So km/lit cho binh xang nay la: 19.417475

Hay nhập vào so lit xang da dung (-1 de ket thuc): 5

Hay nhập vào so km da chay: 120

So km/lit cho binh xang nay la: 24.000000

Hay nhập vào so lit xang da dung (-1 de ket thuc): -1

Trung binh la: 21.601423 km/lit

Bài 4: Một công ty hóa chất lớn trả tiền cho nhân viên bán hàng của mình trên cơ sở hoa hồng. Các nhân viên bán hàng nhận được \$200 mỗi tuần cộng với 9% doanh thu của họ trong tuần đó. Ví dụ: nhân viên bán hàng có doanh thu \$5000 hóa chất trong một tuần sẽ nhận \$200 cộng 9% của \$5000, hoặc tổng cộng là \$650. Viết chương trình nhập vào tổng doanh thu của mỗi nhân viên bán hàng cho tuần trước và tính và hiển thị lên thu nhập của nhân viên đó. Sau đây là một số ví dụ kết quả của chương trình:

Nhap doanh thu cua nhan vien (-1 de ket thuc): 5000.00

Luong cua nhan vien la: \$650

Nhap doanh thu cua nhan vien (-1 de ket thuc): 1234.56

Luong cua nhan vien la: \$311.11

Nhap doanh thu cua nhan vien (-1 de ket thuc): 1088.89

Luong cua nhan vien la: \$298.00

Nhap doanh thu cua nhan vien (-1 de ket thuc): -1

Bài 5: Viết chương trình cho phép người dùng nhập vào hai số nguyên, sau đó tính và xuất lên màn hình ước số chung lớn nhất của hai số vừa nhập.

Nhap hai so nguyen(-1 de ket thuc): 12 28

Uoc so chung lon nhat: 4

Nhap hai so nguyen(-1 de ket thuc): 6 9

Uoc so chung lon nhat: 3

Nhap hai so nguyen(-1 de ket thuc): -1

Bài 6: Viết chương trình yêu cầu người dùng nhập vào một phân số, sau đó rút gọn phân số và hiển thị kết quả lên màn hình.

Nhap mot phan so(-1 de ket thuc): 6/12

Phan so rut gon: ½

Bài 7: (Đếm số ký số) Viết chương trình cho phép người dùng nhập vào một số nguyên dương. Hãy cho biết số vừa nhập có bao nhiêu ký số?

Ví dụ:

Hay nhập so nguyen duong (-1 de ket thuc): 345

So 345 có 3 ký số!

Hãy nhập số nguyên dương (-1 để kết thúc): 12345

So 12345 có 5 ký số!

Hãy nhập số nguyên dương (-1 để kết thúc): -1

Bài 8: Viết chương trình cho phép người dùng nhập vào một số nguyên dương. Chương trình tính và xuất lên màn hình giá trị đảo ngược của số vừa nhập.

Ví dụ:

Hãy nhập số nguyên dương (-1 để kết thúc): 345

Giá trị đảo ngược là: 543

Hãy nhập số nguyên dương (-1 để kết thúc): 5378

Giá trị đảo ngược là: 8735

Hãy nhập số nguyên dương (-1 để kết thúc): 937492

Giá trị đảo ngược là: 294739

Hãy nhập số nguyên dương (-1 để kết thúc): 23

Giá trị đảo ngược là: 32

Hãy nhập số nguyên dương (-1 để kết thúc): 9

Giá trị đảo ngược là: 9

Hãy nhập số nguyên dương (-1 để kết thúc): -1

Bài 9: Một số được gọi là số đối xứng khi đọc xuôi cũng giống như đọc ngược. Ví dụ các số sau đây là số đối xứng: 34543, 455554,... Viết chương trình nhập vào một số nguyên dương, sau đó cho biết số vừa nhập có phải là số đối xứng hay không.

Hướng dẫn: Sử dụng phép chia và chia lấy phần dư để tính giá trị đảo ngược. Nếu giá trị đảo ngược bằng giá trị ban đầu thì đó là số đối xứng.

Ví dụ:

Hãy nhập số nguyên dương (-1 để kết thúc): 345

345 không phải số đối xứng!

Hãy nhập số nguyên dương (-1 để kết thúc): 343

343 là số đối xứng!

Hãy nhập số nguyên dương (-1 để kết thúc): 23432

23432 là số đối xứng!

Hãy nhập số nguyên dương (-1 để kết thúc): -1

Bài 10: Viết chương trình cho phép người dùng nhập vào một số nguyên dương. Sau đó, chương trình tìm ký số lớn nhất và nhỏ nhất xuất hiện trong số vừa nhập.

Ví dụ:

Hãy nhập số nguyên dương (-1 để kết thúc): 23432

Ký số lớn nhất: 4

Ky so nho nhat: 2

Hay nhap so nguyen duong (-1 de ket thuc): 549

Ky so lon nhat: 9

Ky so nho nhat: 4

Hay nhap so nguyen duong (-1 de ket thuc): -1

Bài 11: Viết chương trình xuất lên màn hình lịch của một tháng. Người dùng nhập vào số ngày của tháng và ngày bắt đầu của tháng đó.

CHƯƠNG TRÌNH XUẤT LÊN MÀN HÌNH LỊCH CỦA MỘT THÁNG						
Nhập số ngày của tháng: 31						
Nhập ngày bắt đầu của tháng (1=Chủ nhật, 7=Thứ bảy): 3						
		1	2	3	4	5
6	7	8	9	10	11	12
13	14	15	16	17	18	19
20	21	22	23	24	25	26
27	28	29	30	31		
—						

Hướng dẫn: Sử dụng vòng lặp để lặp lệnh in lên màn hình các giá trị i từ 1 tới N , với N là số ngày của tháng. Trong vòng lặp kiểm tra i có phải là ngày cuối cùng của tuần hay không, nếu đúng thì xuất lên màn hình ký tự xuống dòng.

Bài 12: Tìm và in lên màn hình tất cả các số nguyên trong phạm vi từ 10 đến 99 sao cho tích của 2 chữ số bằng 2 lần tổng của 2 chữ số đó.

Bài 13: Tìm bội số chung nhỏ nhất của 2 số nguyên dương a và b nhập từ bàn phím.

Bài 14: Nhập n . In n số đầu tiên trong dãy Fibonacci.

$$a_0 = a_1 = 1$$

$$a_n = a_{n-1} + a_{n-2}$$

Bài 15: Viết chương trình cho phép người dùng nhập vào một số nguyên dương. Chương trình tính và xuất lên màn hình giá trị của biểu thức sau:

$$S_1 = \frac{1+2+3+\dots+N}{N}$$

$$S_2 = \sqrt{1^2 + 2^2 + 3^2 + \dots + N^2}$$

$$S_3 = \sqrt{3 + \sqrt{3 + \sqrt{3 + \dots + \sqrt{3}}}} \text{ N dấu căn}$$

$$S_4 = \frac{1}{2 + \frac{1}{2 + \frac{1}{2 + \dots + \frac{1}{2}}}} \text{ N dấu chia}$$

Bài tập mở rộng

Bài 1: Viết chương trình nhập vào số nguyên dương N. Xác định chuỗi nhị phân biểu diễn cho N và xuất kết quả lên màn hình.

Ví dụ:

Hay nhập số nguyên dương: 5

Chuỗi nhị phân tương ứng: 0101

Bài 2: Viết chương trình nhập vào một số nguyên dương chỉ gồm các ký số 0 và 1 (nghĩa là số nhị phân) và xuất lên màn hình giá trị nguyên tương ứng.

Hay nhập số nhị phân: 1101

Giá trị nguyên tương ứng: 13

Bài 3: Viết chương trình cho phép người dùng nhập một số nguyên dương N. Sau đó xuất lên màn hình tất cả các giá trị căn chẵn từ 1 tới N.

Bài 4: Viết chương trình cho phép người dùng nhập vào một số nguyên dương. Sau đó, xác định xem có bao nhiêu ký số 7 trong số vừa nhập.

BÀI TẬP THỰC HÀNH TUẦN 02

NGÔN NGỮ C# (CONSOLE)



Mục tiêu

- Làm quen với cú pháp tạo phương thức trong ngôn ngữ lập trình C#
- Tìm hiểu các cách truyền tham số cho phương thức của ngôn ngữ C#
- Viết được các chương trình có các phương thức khác nhau bằng ngôn ngữ C#

Kiến thức

1. Phương thức

a. Tạo phương thức

Phương thức là một khối lệnh chỉ chạy khi nó được gọi. Có thể truyền dữ liệu, được gọi là tham số, vào một phương thức. Các phương thức được sử dụng để thực hiện các hành động nhất định và chúng còn được gọi là các hàm. Tại sao phải sử dụng các phương thức? Để sử dụng lại các khối lệnh (tính tái sử dụng): định nghĩa khối lệnh một lần và sử dụng nó nhiều lần.

Tạo phương thức

Một phương thức được định nghĩa với tên của phương thức, theo sau là dấu ngoặc đơn (). C# cung cấp một số phương thức được xây dựng trước mà đã quen thuộc, chẳng hạn như Main (), nhưng cũng có thể tạo phương thức của riêng mình để thực hiện một số hành động nhất định

Ví dụ: Tạo một phương thức bên trong lớp Program:

```
class Program
{
    static void MyMethod()
    {
        // khối lệnh được thực thi
    }
}
```

MyMethod () là tên của phương thức

static có nghĩa là phương thức thuộc về lớp Program và không phải là một đối tượng của lớp Program.

void có nghĩa là phương thức này không có giá trị trả về.

Trong C#, nên bắt đầu bằng một ký tự hoa khi đặt tên cho các phương thức, vì nó làm cho mã dễ đọc hơn.

b. Gọi phương thức

Để gọi (thực thi) một phương thức, hãy viết tên phương thức theo sau bởi hai dấu ngoặc đơn () và dấu chấm phẩy;

Trong ví dụ sau, MyMethod () được sử dụng để in một văn bản (hành động), khi được gọi.

```
static void MyMethod()  
{  
    Console.WriteLine("I just got executed!");  
}  
  
MyMethod();
```

Một phương thức có thể được gọi nhiều lần:

```
static void MyMethod()  
{  
    Console.WriteLine("I just got executed!");  
}  
  
MyMethod();  
MyMethod();  
MyMethod();
```

2. C# tham số của phương thức

a. Tham số và đối số (Parameters and Arguments)

Thông tin có thể được truyền cho các phương thức dưới dạng tham số. Các tham số hoạt động như các biến bên trong phương thức.

Chúng được chỉ định sau tên phương thức, bên trong dấu ngoặc đơn. Có thể thêm bao nhiêu tham số tùy thích, chỉ cần phân tách chúng bằng dấu phẩy.

Ví dụ sau có một phương thức lấy một chuỗi có tên là **fname** làm tham số. Khi phương thức được gọi, chúng ta truyền một tên đầu tiên, tên này được sử dụng bên trong phương thức để in ra tên đầy đủ:

Ví dụ:

```
static void MyMethod(string fname)  
{  
    Console.WriteLine(fname + " Refsnes");  
}  
MyMethod("Liam");  
MyMethod("Jenny");  
MyMethod("Anja");
```

Khi một tham số (**Parameters**) được truyền cho phương thức, nó được gọi là một đối số (**Arguments**).

Vì vậy, từ ví dụ trên: **fname** là một tham số, trong khi **Liam**, **Jenny** và **Anja** là các đối số.

b. Giá trị tham số mặc định

Cũng có thể sử dụng giá trị tham số mặc định bằng cách sử dụng dấu bằng (=). Nếu chúng ta gọi phương thức mà không có đối số, nó sẽ sử dụng giá trị mặc định ("Norway").

Ví dụ:

```
static void MyMethod(string country = "Norway")  
{  
    Console.WriteLine(country);  
}  
MyMethod("Sweden");  
MyMethod("India");  
MyMethod();
```

```
MyMethod("USA");
```

Một tham số có giá trị mặc định, thường được gọi là "tham số tùy chọn". Từ ví dụ trên, quốc gia là một tham số tùy chọn và "Norway" là giá trị mặc định.

c. Nhiều tham số

Một phương thức có bao nhiêu tham số tùy thích.

```
static void MyMethod(string fname, int age)
{
    Console.WriteLine(fname + " is " + age);
}

MyMethod("Liam", 5);
MyMethod("Jenny", 8);
MyMethod("Anja", 31);
```

d. Giá trị trả về

Từ khóa void, được sử dụng trong các ví dụ trên, chỉ ra rằng phương thức không được trả về một giá trị. Nếu muốn phương thức trả về một giá trị, có thể sử dụng kiểu dữ liệu nguyên thủy (chẳng hạn như int hoặc double) thay vì void và sử dụng từ khóa return bên trong phương thức.

Ví dụ:

```
static int MyMethod(int x)
{
    return 5 + x;
}

Console.WriteLine(MyMethod(3));
```

Ví dụ này trả về tổng của hai tham số của một phương thức:

```
static int MyMethod(int x, int y)
{
    return x + y;
}

Console.WriteLine(MyMethod(5, 3));
```

Cũng có thể lưu trữ kết quả trong một biến (được khuyến nghị, vì dễ đọc và dễ bảo trì hơn)

```
static int MyMethod(int x, int y)
{
    return x + y;
}
int z = MyMethod(5, 3);
Console.WriteLine(z);
```

e. Đối số được đặt tên

Cũng có thể gửi các đối số bằng cú pháp key: value. Bằng cách đó, thứ tự của các đối số không quan trọng.

Ví dụ:

```
static void MyMethod(string child1, string child2, string
child3)
{
    Console.WriteLine("The youngest child is: " + child3);
}
MyMethod(child3: "John", child1: "Liam", child2: "Liam");
```

f. Nạp chồng phương thức

Với tính năng nạp chồng phương thức, nhiều phương thức có thể có cùng tên với các tham số khác nhau.

Ví dụ:

```
int MyMethod(int x)
float MyMethod(float x)
double MyMethod(double x, double y)
```

Hãy xem xét ví dụ sau, có hai phương thức cộng các số thuộc kiểu dữ liệu khác nhau.

```
static int PlusMethodInt(int x, int y)
{
    return x + y;
}

static double PlusMethodDouble(double x, double y)
{
    return x + y;
}

int myNum1 = PlusMethodInt(8, 5);
double myNum2 = PlusMethodDouble(4.3, 6.26);
Console.WriteLine("Int: " + myNum1);
Console.WriteLine("Double: " + myNum2);
```

Thay vì xác định hai phương thức sẽ làm cùng một việc, tốt hơn là nên nạp chồng cho một phương thức.

Trong ví dụ dưới đây, chúng ta nạp chồng phương thức **PlusMethod** để hoạt động cho cả **int** và **double**.

```
static void Main(string[] args)
{
    int myNum1 = PlusMethod(8, 5);
    double myNum2 = PlusMethod(4.3, 6.26);
    Console.WriteLine("Int: " + myNum1);
    Console.WriteLine("Double: " + myNum2);
}
static int PlusMethod(int x, int y)
{
    return x + y;
}
static double PlusMethod(double x, double y)
{
    return x + y;
}
```

Nhiều phương thức có thể có cùng tên miễn là số lượng và/hoặc kiểu dữ liệu của tham số khác nhau.

Bài tập vận dụng

Bài 1: (Hoán vị hai số) Viết chương trình cho phép người dùng nhập vào hai số thực (số thứ nhất lưu vào biến *first* và số thứ hai lưu vào biến *second*) và sau đó gọi hàm *HoanVi* (swap) với các tham số thực là *first* và *second*. Hàm *HoanVi* (swap) có các tham số hình thức là *number1* và *number2* sẽ hoán vị giá trị của hai biến. Các giá trị sau khi hoán vị sẽ được xuất trong hàm *main*.

Ví dụ:

Hay nhập số thu nhất, sau đó enter: 80

Hay nhập số thu hai, sau đó enter: 70

Bạn nhập các số là 80 và 70.

Sau khi hoán vị, số thu nhất có giá trị 70.

Số thu hai có giá trị là 80.

Bài 2: (Tính tốc độ Km/h) Viết chương trình nhập vào số Km và số giờ đã đi du lịch. Chương trình xác định số tốc độ Km trên một giờ. Việc tính toán này được thực hiện trong một hàm khác hàm *main*, hàm *main* sẽ xuất lên kết quả tính toán. Hàm tính toán sẽ có ba tham số: Km, Hours và KmPerHour. Tham số nào được truyền tham trị và truyền tham chiếu? Kết quả xuất lên màn hình lấy chính xác đến hai số thập phân.

Ví dụ:

Hay nhập số km đã đi du lịch:

475

Hay nhập số giờ đã đi du lịch:

8

Tốc độ của bạn là 59.38 km trên giờ

Bài 3: (Chuyển điểm số sang điểm chữ) Viết chương trình cho phép người dùng nhập vào số lượng điểm số. Sau đó, chương trình cho người dùng nhập vào từng điểm số. Chương trình sẽ tính tổng của các điểm được nhập và truyền nó cùng với số lượng điểm tới một hàm có một tham số “truyền tham chiếu”. Hàm này sẽ tính điểm trung bình của các điểm được nhập. Hàm *main* sẽ xác định điểm chữ tương ứng với điểm trung bình trên theo quy đổi:

90 – 100	A
80 – 89	B
70 – 79	C
60 – 69	D
0 – 59	F

Ví dụ:

Hay nhập số lượng điểm số:

3

Hay nhập điểm số thu 1 (0 – 100):

90

Hay nhập điểm số thu 2 (0 – 100):

80

Hay nhập điểm số thu 3 (0 – 100):

50

Điểm chu tương ứng là: C

Bài 4: (Tính số giây đồng hồ) Viết một hàm tính số giây theo giờ (gồm ba thông tin giờ, phút và giây) tính từ thời điểm cuối cùng của đồng hồ theo cấu trúc 24h. Sử dụng hàm này để tính khoảng cách theo giây giữa hai giờ cho trước, cả hai giờ đều nằm trong chu kỳ 24h đồng hồ. Hàm được mô tả như sau:

```
//*****
// Ten ham: Seconds(int, int, int, int &)
// Tac vu: Ham nay tinh tong so giay
// Dau vao: hours, minutes, seconds
// Dau ra: TotalSeconds (thay doi tham so thuc tuong ung)
//*****
```

Bài 5: (Tính cạnh huyền) Định nghĩa một hàm có tên Hypotenuse để tính chiều dài cạnh huyền của một tam giác vuông khi biết chiều dài hai cạnh còn lại. Hàm này được mô tả như sau:

```
//*****
// Ten ham: Hypotenuse(double, double, double &)
// Tac vu: Ham nay tinh va xuat ra chieu dai canh huyen
// Dau vao: Side1 va Side2
// Dau ra: Hypotenuse (thay doi tham so thuc tuong ung)
//*****
```

Bài 6: (Liệt kê số nguyên tố) Nhập số nguyên dương n (n>0). Liệt kê tất cả các số nguyên tố nhỏ hơn n.

Ví dụ:

Hay nhập số nguyên dương: 11

Các số nguyên tố nhỏ hơn 11 là: 2 3 5 7

Bài 7: (Liệt kê số chính phương) Nhập số nguyên dương n (n>0). Liệt kê n số chính phương đầu tiên.

Ví dụ:

Hay nhập số nguyên dương: 5

5 số chính phương đầu tiên: 4 9 16 25 36

Bài tập mở rộng

Bài 1: (Đếm số lượng số hoàn hảo) Nhập số nguyên dương n ($n > 0$). Cho biết có bao nhiêu số hoàn thiện nhỏ hơn n .

Ví dụ:

Hay nhập số nguyên dương:

3

Số lượng số hoàn hảo nhỏ hơn 3: 0

Hay nhập số nguyên dương:

36

Số lượng số hoàn hảo nhỏ hơn 36: 2

Bài 2: (Tam giác) Nhập vào tọa độ 3 điểm A, B, C và kiểm tra xem chúng có phải là 3 đỉnh của một tam giác hay không? Nếu có hãy tính diện tích, chiều dài mỗi đường cao của tam giác và in kết quả ra màn hình.

- Công thức tính diện tích $s = \sqrt{p(p-a)(p-b)(p-c)}$
- Công thức tính các đường cao: $h_a = \frac{2s}{a}, h_b = \frac{2s}{b}, h_c = \frac{2s}{c}$

(Với $p = \frac{a+b+c}{2}$: là nửa chu vi của tam giác, a, b, c là chiều dài 3 cạnh)

Bài 3: (Tìm ước chung và bội chung) Viết chương trình nhập vào 2 số nguyên dương a và b . Tìm ước số chung lớn nhất và bội số chung nhỏ nhất của 2 số vừa nhập.

BÀI TẬP THỰC HÀNH TUẦN 03

LẬP TRÌNH WINDOWS FORM



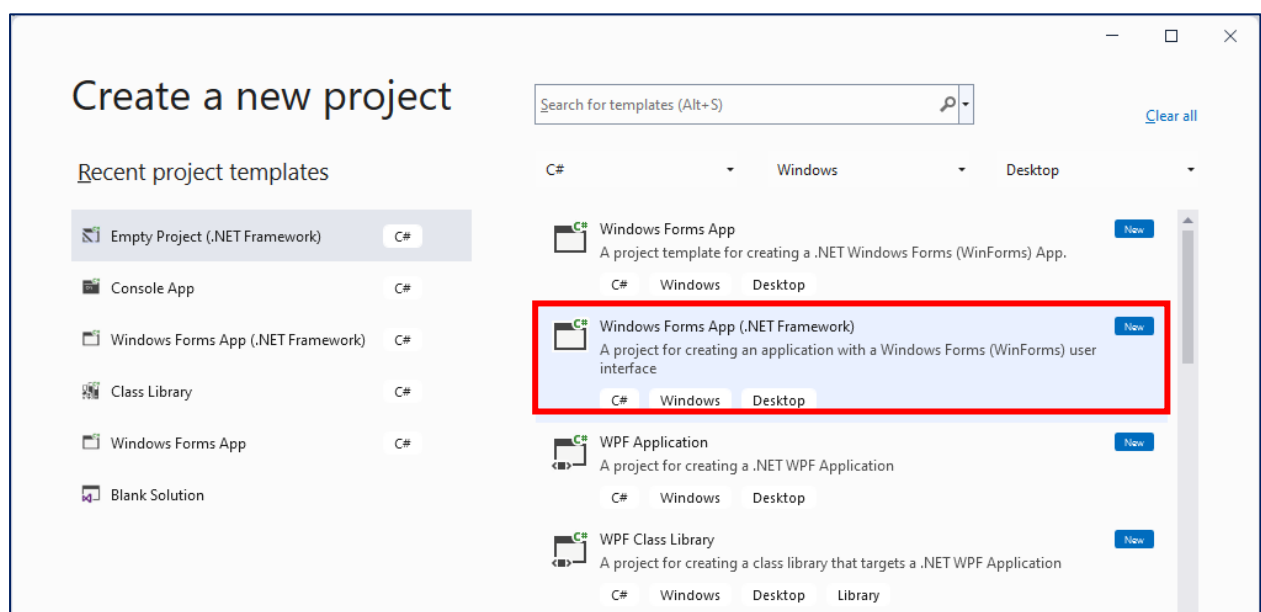
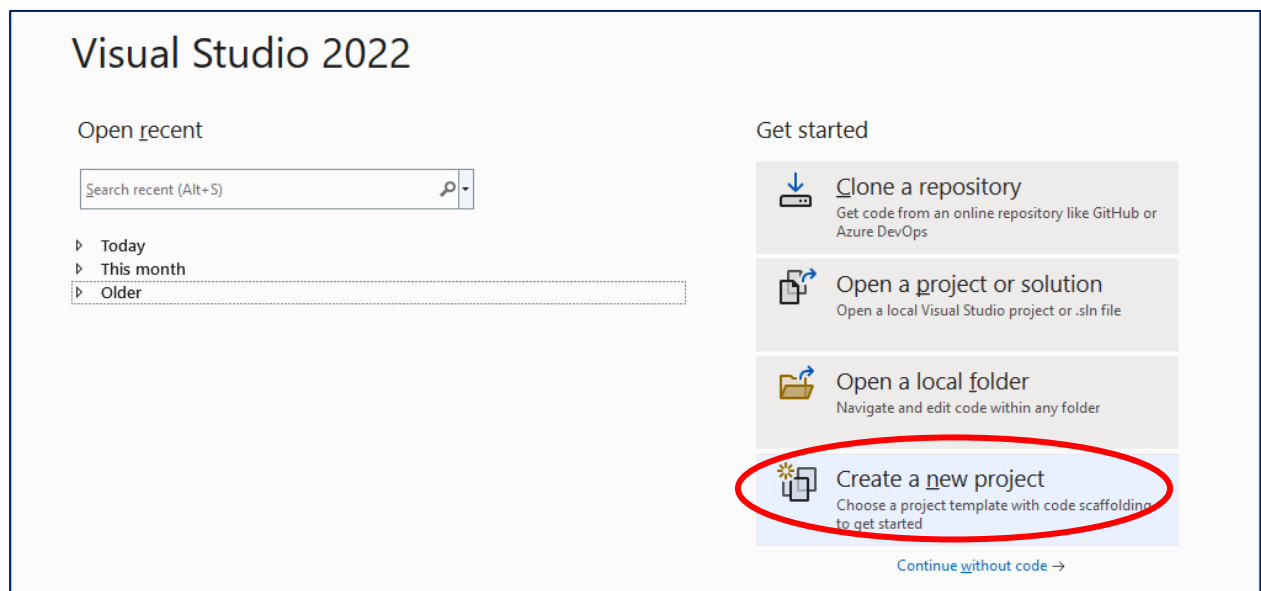
Mục tiêu

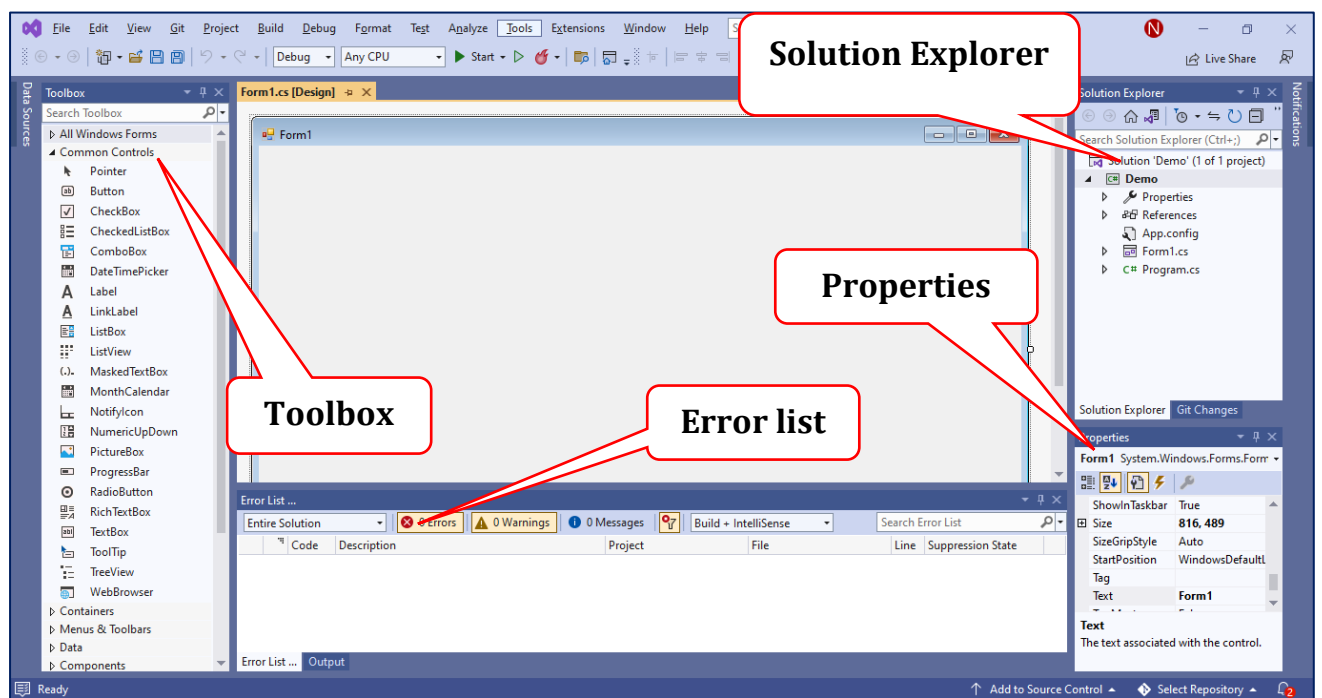
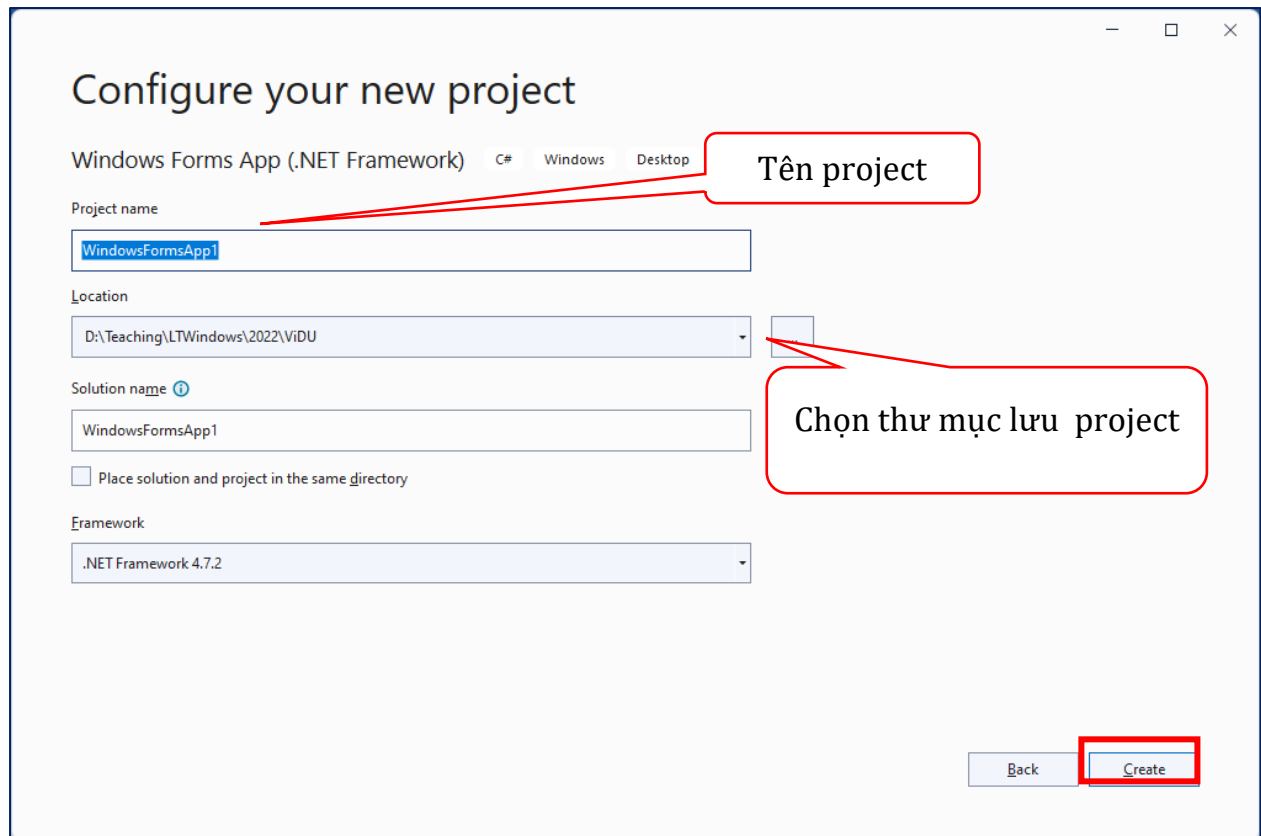
- Làm quen với lập trình ứng dụng Windows Form bằng ngôn ngữ lập trình C#
- Tìm hiểu các control cơ bản
- Viết được các ứng dụng khác nhau bằng các control được giới thiệu

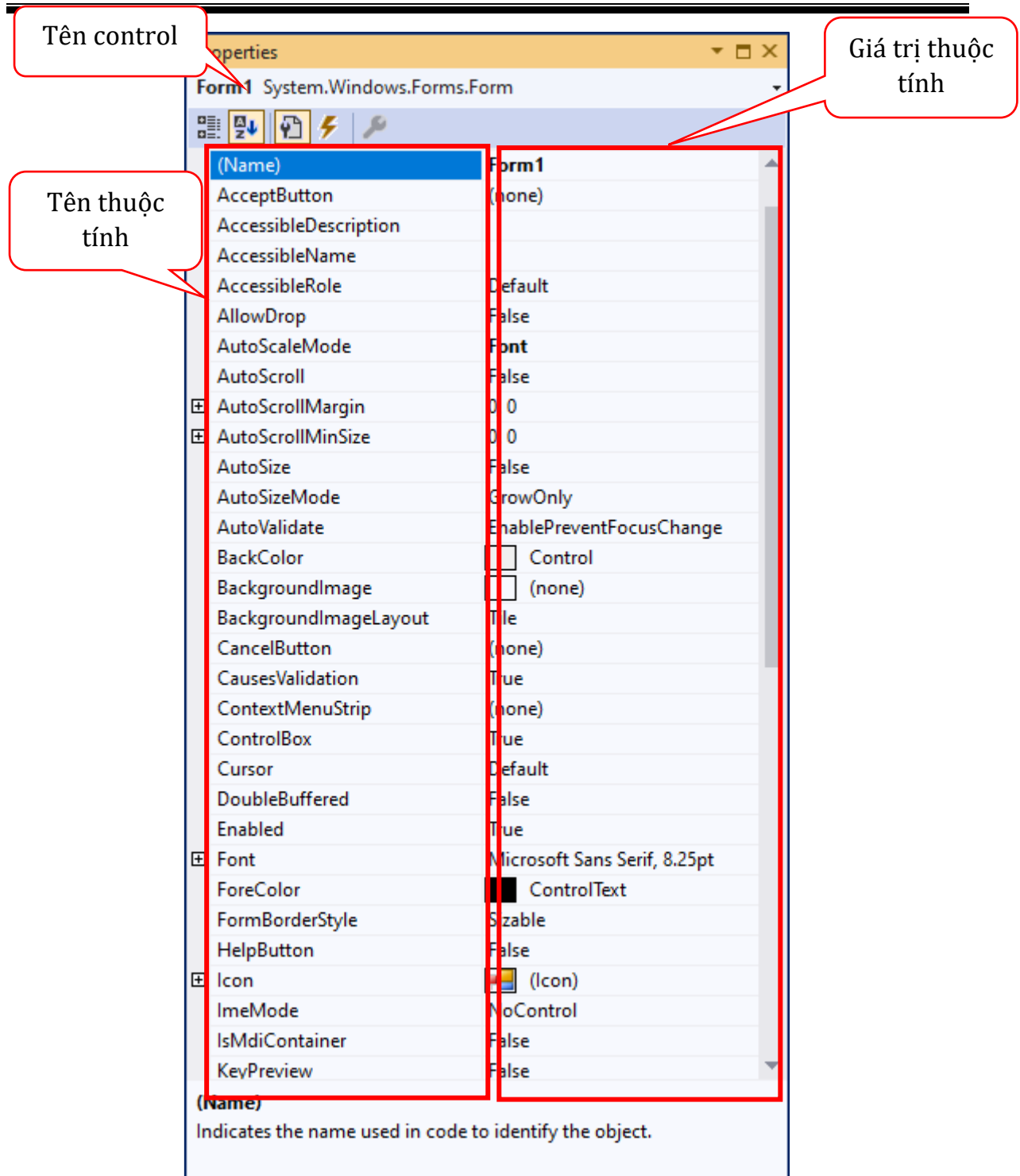
Kiến thức

1. Tạo ứng dụng Windows Form

Khởi động Visual Studio 2022.







2. Các control cơ bản

a. Label

Chúng ta có thể sử dụng điều khiển Label để hiển thị văn bản ở một vị trí đã định trên trang. Điều khiển Label cũng có thể được sử dụng để thêm văn bản mô tả vào Form nhằm cung cấp cho người dùng thông tin hữu ích. Lớp Label được định nghĩa trong không gian tên (namespace) System.Windows.Forms.

Nếu muốn thay đổi văn bản hiển thị của Label, phải đặt văn bản mới vào thuộc tính Text của Label.

```
label1.Text = "This is my first Label";
```

Ngoài việc hiển thị văn bản, điều khiển Label cũng có thể hiển thị hình ảnh bằng cách sử dụng thuộc tính Image hoặc kết hợp các thuộc tính ImageIndex và ImageList:

Ví dụ:

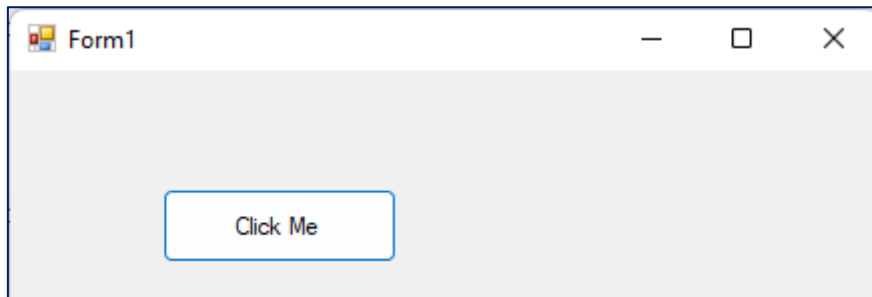
```
label1.Image = Image.FromFile("C:\\testimage.jpg");
```

Đoạn lệnh C # sau đây cho thấy thiết lập một số thuộc tính Label thông qua dòng lệnh.

```
private void Form1_Load(object sender, EventArgs e)
{
    label1.Text = "This is my first Lable";
    label1.BorderStyle = BorderStyle.FixedSingle;
    label1.TextAlign = ContentAlignment.MiddleCenter;
}
```

b. Button

Button là một điều khiển, là một thành phần tương tác cho phép người dùng giao tiếp với một ứng dụng. Lớp Button kế thừa trực tiếp từ lớp ButtonBase. Một nút có thể được nhấp bằng cách sử dụng chuột, phím ENTER hoặc phím SPACEBAR nếu nút có focus.



Khi muốn thay đổi văn bản hiển thị của Button, có thể thay đổi thuộc tính Text của Button.

```
button1.Text = "Click Here";
```

Tương tự, nếu muốn tải một Hình ảnh vào một điều khiển Button, có thể viết lệnh như sau.

```
button1.Image = Image.FromFile("C:\\testimage.jpg");
```

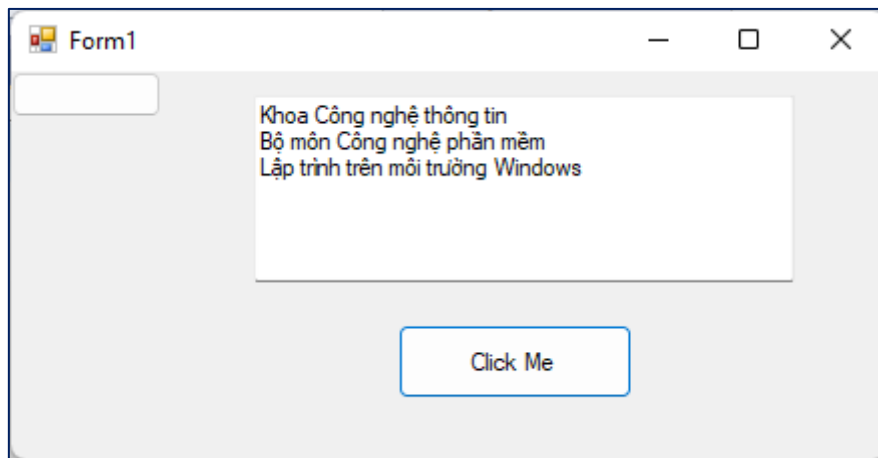
Cách gọi sự kiện nhấp chuột của một Button trong lập trình sự kiện

Sự kiện Click chuột xảy ra khi điều khiển Button được nhấp. Sự kiện này thường được sử dụng khi không có tên phương thức nào được liên kết với điều khiển Button.

```
private void Form1_Load(object sender, EventArgs e)
{
    Button b = new Button();
    b.Click += new EventHandler(ShowMessage);
    Controls.Add(b);
}
private void ShowMessage(object sender, EventArgs e)
{
    MessageBox.Show("Button Click");
}
```

c. Textbox

Điều khiển TextBox được sử dụng để hiển thị hoặc chấp nhận làm đầu vào, một dòng văn bản. Điều khiển này có chức năng bổ sung không có trong điều khiển hộp văn bản tiêu chuẩn của Windows, bao gồm chỉnh sửa nhiều dòng và che ký tự mật khẩu.



Đối tượng TextBox được sử dụng để hiển thị văn bản trên Form hoặc để nhận thông tin đầu vào của người dùng trong khi chương trình C# đang chạy. Trong TextBox, người dùng có thể nhập hoặc dán dữ liệu vào điều khiển từ clipboard.

Để hiển thị văn bản trong điều khiển TextBox, có thể viết mã như thế này

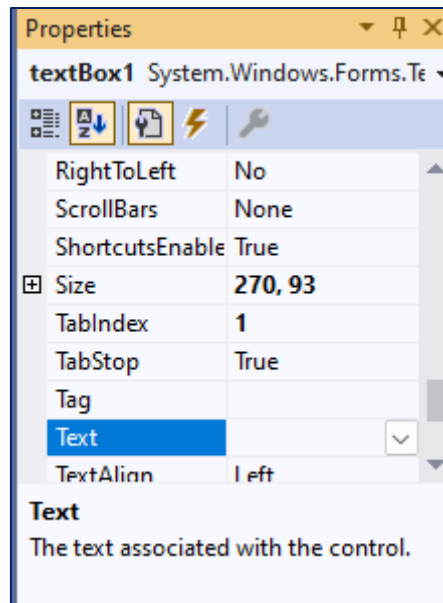
```
textBox1.Text = "Khoa Công nghệ thông tin\r\nBộ môn Công nghệ phần mềm\r\nLập trình\r\ntrên môi trường Windows";
```

Cũng có thể thu thập giá trị đầu vào từ điều khiển TextBox vào một biến như cách này

```
string var;  
var = textBox1.Text;
```

Các thuộc tính của TextBox C#

Có thể thiết lập giá trị thuộc tính của TextBox thông qua cửa sổ Properties hoặc thông qua lệnh. Có thể mở cửa sổ Properties bằng cách nhấn F4 hoặc nhấp chuột phải vào một điều khiển và chọn mục menu Properties.



Các lệnh bên dưới đặt chiều rộng Textbox là 250 và chiều cao là 50 thông qua lệnh.

```
textBox1.Width = 250;
textBox1.Height = 50;
```

Background Color and Foreground Color

Có thể đặt màu nền và màu nền trước thông qua cửa sổ thuộc tính và theo lệnh.

```
textBox1.BackColor = Color.Blue;
textBox1.ForeColor = Color.White;
```

Textbox BorderStyle

Có thể đặt 3 kiểu kiểu đường viền khác nhau cho hộp văn bản, chúng là None, FixedSingle and fixed3d.

```
textBox1.BorderStyle = BorderStyle.Fixed3D;
```

TextBox Events

Keydown event

Có thể bắt được phím nào được nhấn bởi người dùng bằng sự kiện KeyDown

```
private void textBox1_KeyDown(object sender, KeyEventArgs e)
{
    if (e.KeyCode == Keys.Enter)
    {
        MessageBox.Show("You press Enter Key");
    }
    if (e.KeyCode == Keys.CapsLock)
    {
        MessageBox.Show("You press Caps Lock Key");
    }
}
```

TextChanged Event

Khi người dùng nhập hoặc thiết lập thuộc tính Textbox thành một giá trị mới sẽ kích hoạt sự kiện TextChanged

```
private void textBox1_TextChanged(object sender, EventArgs e)
{
    label1.Text = textBox1.Text;
}
```

Textbox Maximum Length

Đặt số lượng ký tự hoặc từ tối đa mà người dùng có thể nhập vào điều khiển Textbox

```
textBox1.MaxLength = 40;
```

Textbox ReadOnly

Khi chương trình muốn ngăn người dùng thay đổi văn bản xuất hiện trong Textbox, chương trình có thể thiết lập thuộc tính ReadOnly thành True.

```
textBox1.ReadOnly = true;
```

Multiline TextBox

Có thể sử dụng thuộc tính Multiline và ScrollBars để cho phép hiển thị hoặc nhập nhiều dòng văn bản

```
textBox1.Multiline = true;
```

Textbox password character

Điều khiển TextBox cũng có thể được sử dụng để chấp nhận mật khẩu và các thông tin nhạy cảm khác. Có thể sử dụng thuộc tính PasswordChar để che dấu các ký tự được nhập.

```
textBox1.PasswordChar = '*';
```

Cách tạo dòng mới trong TextBox

Có thể thêm dòng mới trong hộp văn bản bằng nhiều cách.

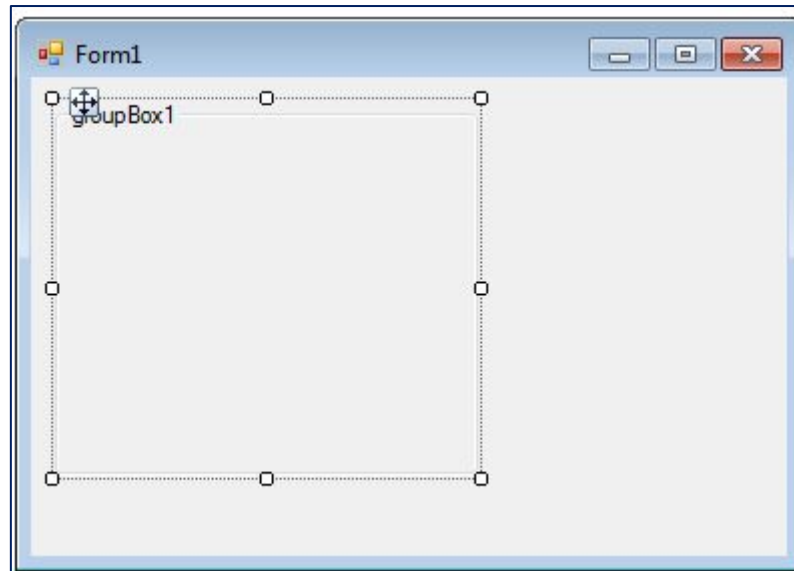
```
textBox1.Text += "your text" + "\r\n";
```

hoặc

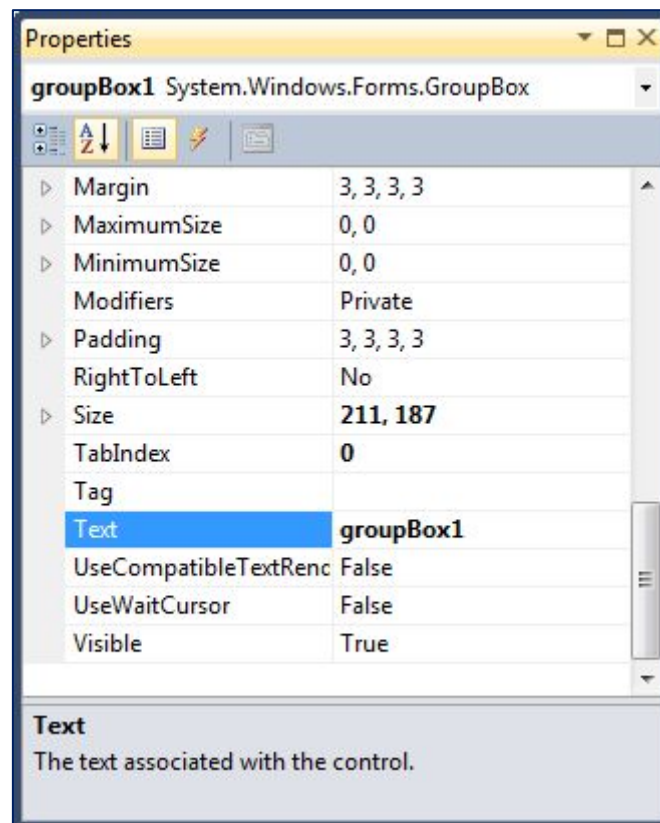
```
textBox1.Text += "your text" + Environment.NewLine;
```

d. Groupbox

Điều khiển GroupBox là một điều khiển vùng chứa được sử dụng để đặt các điều khiển con của Windows Forms vào một nhóm. Mục đích của GroupBox là xác định giao diện người dùng nơi chúng ta có thể phân loại các điều khiển liên quan trong một nhóm.



Thiết lập thuộc tính GroupBox



Location, Height, Width, and Size

thuộc tính **Location** lấy một Điểm xác định vị trí bắt đầu của GroupBox trên một Form. Thuộc tính **Size** chỉ định kích thước của điều khiển. Chúng ta cũng có thể sử dụng thuộc tính **Width** và **Height** thay vì thuộc tính **Size**.

```
groupBox1.Location = newPoint(10, 10);  
groupBox1.Width = 250;  
groupBox1.Height = 200;
```


Background and Foreground

```
groupBox1.BackColor = Color.LightBlue;  
groupBox1.ForeColor = Color.Maroon;
```

Name

Thuộc tính name đại diện cho một tên duy nhất của điều khiển GroupBox. Nó được sử dụng để truy cập điều khiển trong viết lệnh.

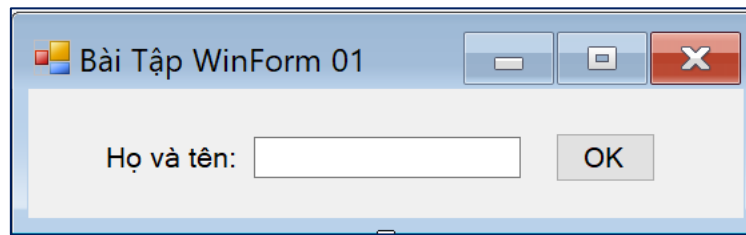
```
groupBox1.Name = "groupBox1";
```

Text

Thuộc tính Text của GroupBox đại diện cho văn bản tiêu đề của điều khiển GroupBox.

Bài tập vận dụng

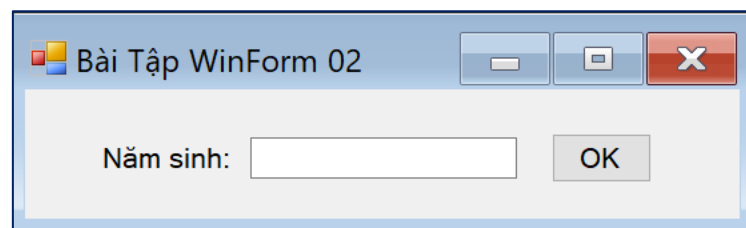
Bài 1: Thiết kế giao diện chương trình như hình bên dưới.



Nhập họ và tên vào Textbox, sau đó nhấn vào Button OK thì hiển thị thông báo “Xin chào ...” lên MessageBox. Với ... là họ tên được nhập từ người dùng.

Lưu ý kiểm tra và thông báo lên MessageBox nếu chưa nhập dữ liệu.

Bài 2: Thiết kế giao diện chương trình như hình bên dưới.



Nhập năm sinh vào Textbox, sau đó nhấn vào Button OK thì hiển thị thông báo “Số tuổi của là: ...” lên MessageBox.

Lưu ý kiểm tra và thông báo lên MessageBox nếu chưa nhập dữ liệu.

Bài 3: Thiết kế giao diện chương trình như hình bên dưới.

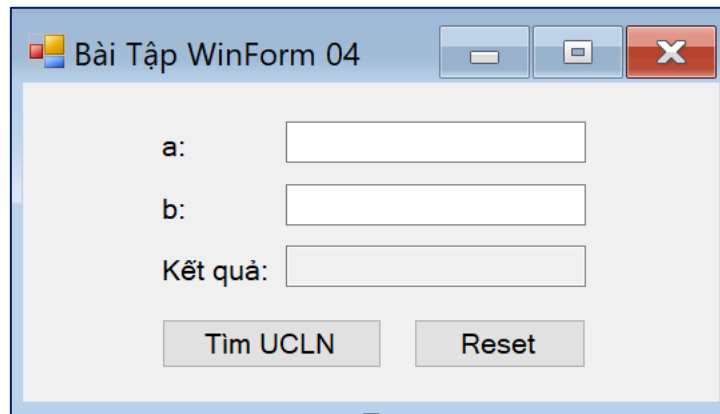


Nhập 2 số nguyên a, b vào 2 Textbox, sau đó thực hiện các phép tính khi nhấn vào các Button tương ứng Cộng ($a + b$), Trừ ($a - b$), Nhân ($a * b$), Chia (a / b) và hiển thị kết quả lên Textbox Kết quả.

Nhấn vào nút Reset thì xoá tất cả nội dung trong các Textbox.

Lưu ý kiểm tra và thông báo lên MessageBox nếu chưa nhập dữ liệu

Bài 4: Thiết kế giao diện chương trình như hình bên dưới.



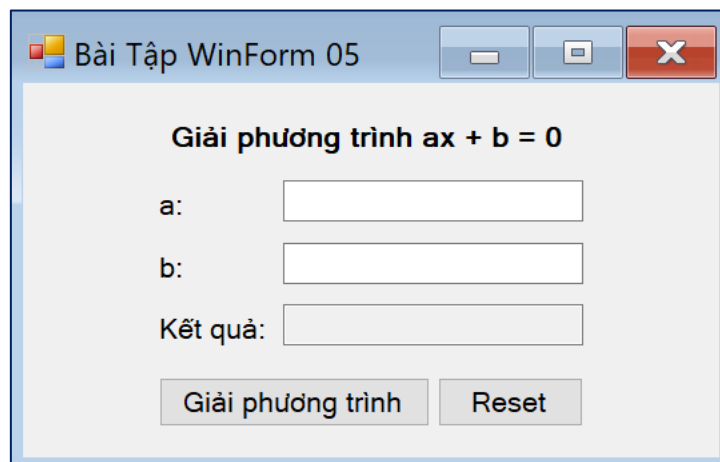
The screenshot shows a Windows application window titled "Bài Tập WinForm 04". Inside the window, there are three text input fields. The first is labeled "a:", the second is labeled "b:", and the third is labeled "Kết quả:". Below these text boxes are two buttons: "Tìm UCLN" (Find GCD) and "Reset". The window has standard Windows window controls (minimize, maximize, close) in the title bar.

Nhập 2 số nguyên a, b vào 2 Textbox, sau đó nhấn vào Button Tìm UCLN thì sẽ hiển thị ước chung lớn nhất của 2 số nguyên a, b vào Textbox kết quả .

Nhấn vào nút Reset thì xoá tất cả nội dung trong các Textbox.

Lưu ý kiểm tra và thông báo lên MessageBox nếu chưa nhập dữ liệu.

Bài 5: Thiết kế giao diện chương trình như hình bên dưới



The screenshot shows a Windows application window titled "Bài Tập WinForm 05". At the top of the form area, the text "Giải phương trình $ax + b = 0$ " is displayed. Below this text are three text input fields labeled "a:", "b:", and "Kết quả:". At the bottom of the form are two buttons: "Giải phương trình" (Solve equation) and "Reset". The window has standard Windows window controls in the title bar.

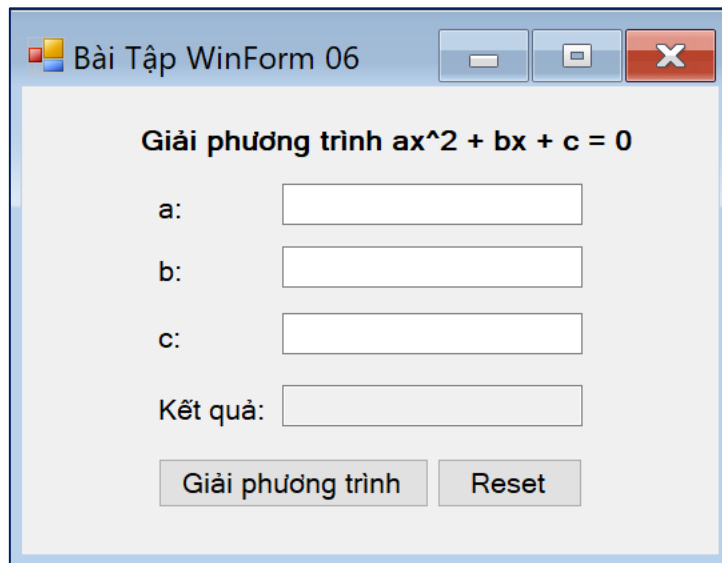
Nhập 2 số nguyên a, b vào 2 Textbox, sau đó nhấn vào Button Giải phương trình thì sẽ hiển thị kết quả của việc giải phương trình $ax + b = 0$ vào Textbox kết quả.

Nhấn vào nút Reset thì xoá tất cả nội dung trong các Textbox.

Lưu ý kiểm tra và thông báo lên MessageBox nếu chưa nhập dữ liệu.

Bài tập mở rộng

Bài 1: Thiết kế giao diện chương trình như hình bên dưới.



The screenshot shows a Windows application window titled "Bài Tập WinForm 06". Inside the window, the text "Giải phương trình $ax^2 + bx + c = 0$ " is displayed. Below this, there are three labels: "a:", "b:", and "c:", each followed by a text input box. Below these is a label "Kết quả:" followed by another text input box. At the bottom of the form, there are two buttons: "Giải phương trình" and "Reset".

Nhập 3 số nguyên a, b vào 3 Textbox, sau đó nhấn vào Button Giải phương trình thì sẽ hiển thị kết quả của việc giải phương trình $ax^2 + bx + c = 0$ vào Textbox kết quả. Nhấn vào nút Reset thì xoá tất cả nội dung trong các Textbox.

Lưu ý kiểm tra và thông báo lên MessageBox nếu chưa nhập dữ liệu.

BÀI TẬP THỰC HÀNH TUẦN 04

CONTROLS

(CHECKBOX, RADIO BUTTON, LISTBOX, COMBOBOX)



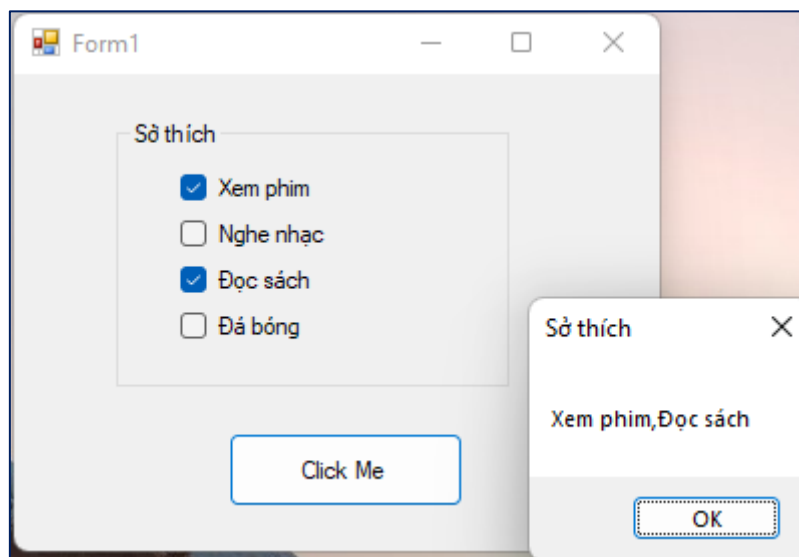
Mục tiêu

- Tìm hiểu các control cơ bản: checkbox, radio button, listbox, combobox
- Viết được các ứng dụng khác nhau bằng các control được giới thiệu

Kiến thức

1. Checkbox

CheckBox cho phép người dùng thực hiện nhiều lựa chọn từ một số tùy chọn. Có thể nhấp vào hộp kiểm để chọn nó và nhấp lại vào nó để bỏ chọn.



Điều khiển CheckBox có thể hiển thị hình ảnh hoặc văn bản hoặc cả hai. Thông thường CheckBox đi kèm với chú thích mà có thể đặt trong thuộc tính Text.

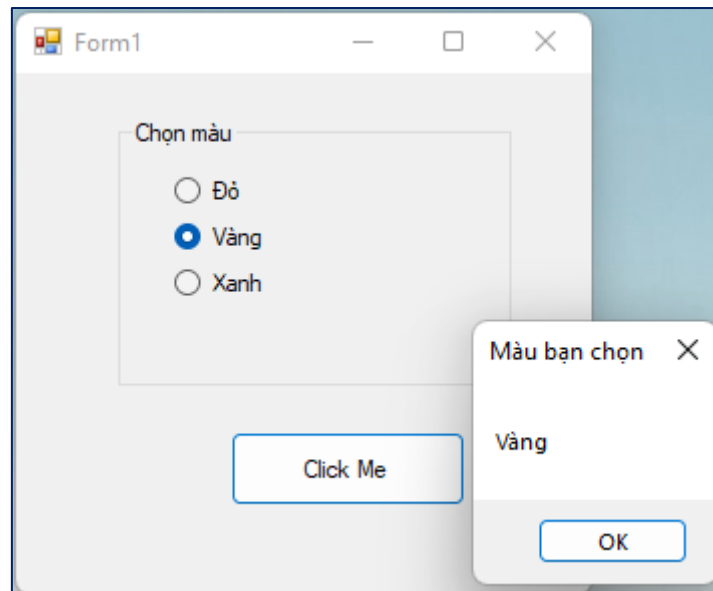
```
chkXemPhim.Text = "Xem phim";
```

Có thể sử dụng thuộc tính CheckBox control ThreeState để chỉ đạo điều khiển trả về các giá trị Đã kiểm tra, Bỏ kiểm tra và Không xác định. Cần đặt hộp kiểm thuộc tính ThreeState thành True để cho biết rằng muốn nó hỗ trợ ba trạng thái:

```
chkXemPhim.ThreeState = true;
```

2. Radio Button

Radio button hoặc nút tùy chọn cho phép người dùng chọn một tùy chọn từ một nhóm các lựa chọn khi được ghép nối với các điều khiển RadioButton khác. Khi người dùng nhấp vào một nút radio, nút đó sẽ được chọn và tất cả các nút radio khác có cùng nhóm sẽ không được chọn.

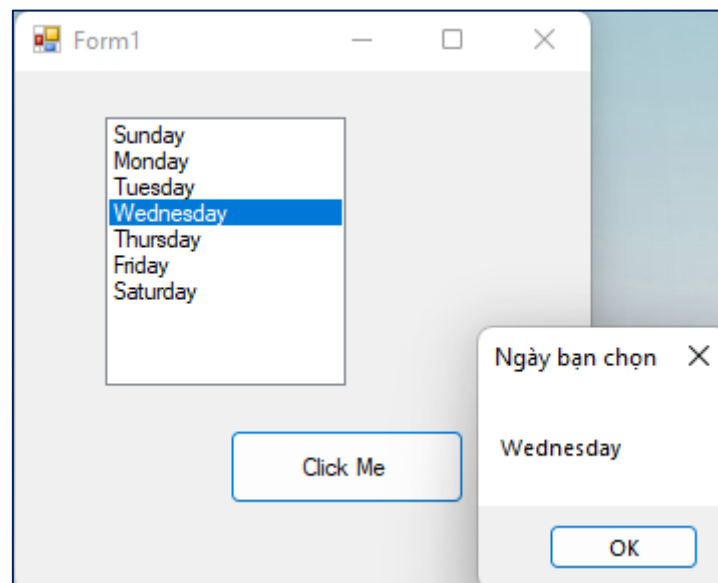


Điều khiển RadioButton có thể hiển thị văn bản, Hình ảnh hoặc cả hai. Sử dụng thuộc tính Checked để lấy hoặc thiết lập trạng thái của RadioButton..

```
radioButton1.Checked = true;
```

3. ListBox

Điều khiển ListBox cho phép hiển thị danh sách các mục cho người dùng mà người dùng có thể chọn bằng cách nhấp vào.



Một số thuộc tính thường dùng:

Items – danh sách các item có trong list

Sorted = true: tự động sắp xếp theo từ điển

SelectedIndex, SelectedIndices, SelectedItem, SelectedItems và Text – cung cấp các cách khác nhau để truy cập các mục đã chọn

Thêm các phần tử vào Listbox

Cú pháp:

```
public int Add(object item);
```

Ví dụ:

```
listBox1.Items.Add("Sunday");
```

Chèn (Insert) các phần tử vào Listbox

Cú pháp:

```
public void Insert(int index, object item);
```

Ví dụ: Có thể chèn một phần tử vào Listbox tại chỉ mục được chỉ định.

```
listBox1.Items.Insert(0, "First");
```

```
listBox1.Items.Insert(1, "Second");
```

```
listBox1.Items.Insert(2, "Third");
```

```
listBox1.Items.Insert(3, "Forth");
```

Phần tử đã chọn trong Listbox

```
string item = listBox1.GetItemText(listBox1.SelectedItem);
```

Hoặc

```
string item = listBox1.SelectedItem.ToString();
```

Hoặc

```
string item = listBox1.Text;
```

Chọn nhiều mục từ Listbox

Thuộc tính **SelectionMode** xác định số lượng mục trong danh sách có thể được chọn tại một thời điểm. Nếu thay đổi thuộc tính chế độ lựa chọn thành nhiều lựa chọn, thì sẽ truy xuất một tập các mục từ thuộc tính **ListBox1.SelectedItems**.

```
listBox1.SelectionMode = SelectionMode.MultiSimple;
```

Lớp **Listbox** có hai **SelectionMode**. **Multiple** hoặc **Extended**. Ở chế độ **Multiple**, có thể chọn hoặc bỏ chọn bất kỳ mục nào trong **Listbox** bằng cách nhấp vào nó. Ở chế độ **Extended**, cần giữ phím **Ctrl** để chọn các mục bổ sung hoặc phím **Shift** để chọn một loạt các mục.

```
private void Form1_Load(object sender, EventArgs e)
{
    listBox1.Items.Add("Sunday");
    listBox1.Items.Add("Monday");
    listBox1.Items.Add("Tuesday");
    listBox1.Items.Add("Wednesday");
    listBox1.Items.Add("Thursday");
    listBox1.Items.Add("Friday");
    listBox1.Items.Add("Saturday");
    listBox1.SelectionMode = SelectionMode.MultiSimple;
}
1 reference
private void button1_Click(object sender, EventArgs e)
{
    foreach (Object obj in listBox1.SelectedItems)
    {
        MessageBox.Show(obj.ToString());
    }
}
```

Nhận tất cả các mục từ Listbox

```
private void button1_Click_1(object sender, EventArgs e)
{
    string items = "";
    foreach (var item in listBox1.Items)
    {
        items += item.ToString() + ", ";
    }
    MessageBox.Show(items);
}
```

Làm thế nào để liên kết một ListBox với một List?

Có thể liên kết một List với một điều khiển ListBox bằng cách tạo một Đối tượng List mới và thêm các mục vào List.

Tạo một List

```
List<string> nList = new List<string>();
nList.Add("January");
nList.Add("February");
nList.Add("March");
nList.Add("April");
```

Liên kết vào List

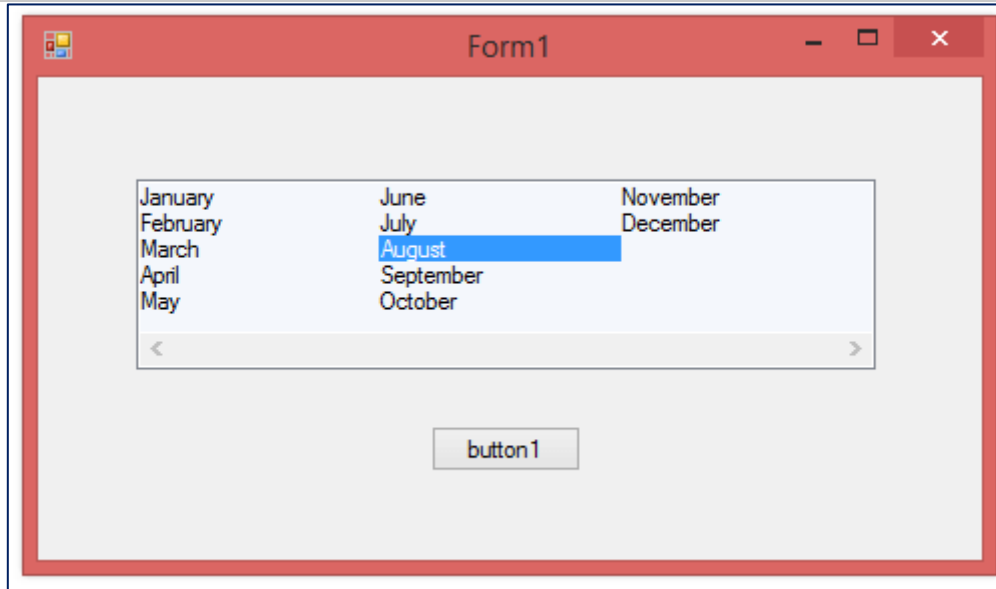
Bước tiếp theo là liên kết List này với ListBox. Để làm được điều đó, nên đặt datasource của ListBox.


```
listBox1.DataSource = nList;
```

Cột trong Listbox

Một **ListBox** nhiều cột đặt các mục vào nhiều cột nếu cần để làm cho việc cuộn dọc không cần thiết. Người dùng có thể sử dụng bàn phím để điều hướng đến các cột hiện không hiển thị. Trước hết, có **Gets** hoặc **Sets** một giá trị cho biết **ListBox** có hỗ trợ **multiple columns**.

```
public bool MultiColumn { get; set; }
```



Thanh cuộn dọc của ListBox

ListBox chỉ hiển thị một thanh cuộn dọc khi có nhiều mục hơn khu vực hiển thị. Có thể sử dụng thuộc tính **ListBox.ScrollAlwaysVisible** nếu muốn thanh luôn hiển thị.

Làm cách nào để xóa tất cả dữ liệu trong listBox?

Phương thức **ListBox.Items.Clear ()** xóa tất cả các mục khỏi ListBox.

```
listBox1.Items.Clear();
```

Phương thức **ListBox.ClearSelected** Bỏ chọn tất cả các mục trong ListBox

```
listBox1.ClearSelected();
```

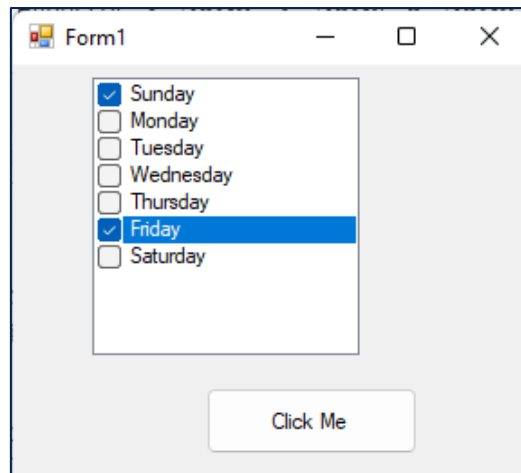
Xóa mục khỏi Hộp danh sách

```
public void RemoveAt(int index);
```

Phương thức **RemoveAt** loại bỏ mục tại chỉ mục được chỉ định trong bộ sưu tập. Khi xóa một mục khỏi danh sách, các chỉ mục sẽ thay đổi cho các mục tiếp theo trong danh sách. Tất cả thông tin về mục đã loại bỏ sẽ bị xóa.

4. Checked ListBox

Điều khiển **CheckedListBox** cung cấp cho tất cả khả năng của hộp danh sách và cũng cho phép hiển thị dấu kiểm bên cạnh các mục trong hộp danh sách.



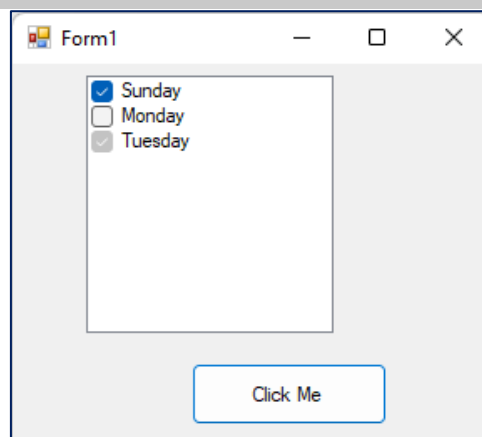
Người dùng có thể đặt dấu kiểm cho một hoặc nhiều mục và các mục đã đánh dấu có thể được điều hướng bằng **CheckedListBox.CheckedItemCollection** và **CheckedListBox.CheckedIndexCollection**.

Thêm các mục vào CheckedListbox

```
public int Add(object item, bool isChecked);
```

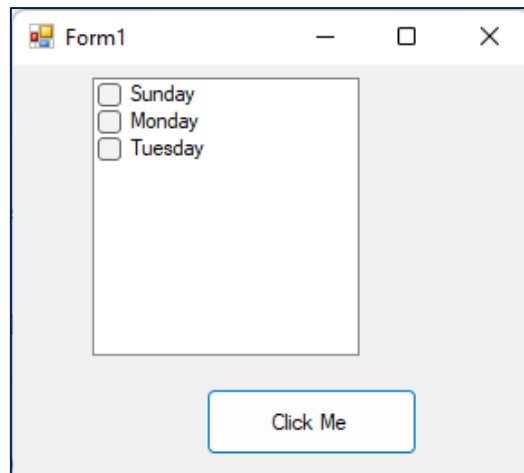
Có thể thêm các mục riêng lẻ vào danh sách bằng phương thức **Add**. Đối tượng **CheckedListBox** hỗ trợ ba trạng thái thông qua kiểu liệt kê **CheckState**: Đã **Checked**, **Indeterminate**, and **Unchecked**.

```
checkedListBox1.Items.Add("Sunday", CheckState.Checked);
checkedListBox1.Items.Add("Monday", CheckState.Unchecked);
checkedListBox1.Items.Add("Tuesday", CheckState.Indeterminate);
```



Nếu muốn thêm các đối tượng vào danh sách tại thời điểm chạy, hãy gán một mảng tham chiếu đối tượng bằng phương thức **AddRange**. Sau đó, danh sách sẽ hiển thị giá trị chuỗi mặc định cho mỗi đối tượng. Theo mặc định, các mục trong CheckedListbox là **unchecked**.

```
string[] days = new[] { "Sunday", "Monday", "Tuesday" };
checkedListBox1.Items.AddRange(days);
```



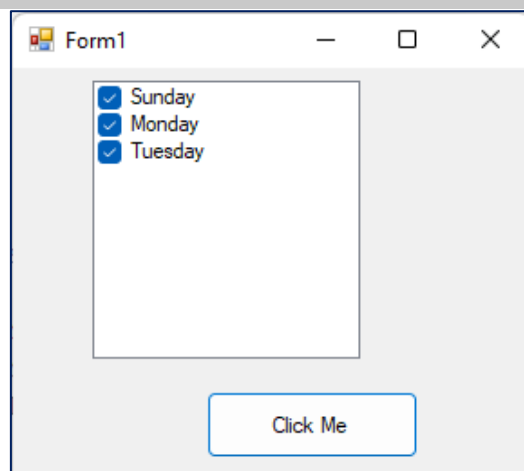
Đánh dấu chọn tất cả các mục trong CheckedListBox

Nếu muốn đánh dấu chọn một mục trong Checkedlistbox, cần gọi **SetItemChecked** với mục liên quan.

```
public void SetItemChecked(int index, bool value);
```

Nếu muốn thiết lập tất cả các mục trong CheckedListBox thành được chọn, hãy thay đổi giá trị của phương thức **SetItemChecked** thành **true**.

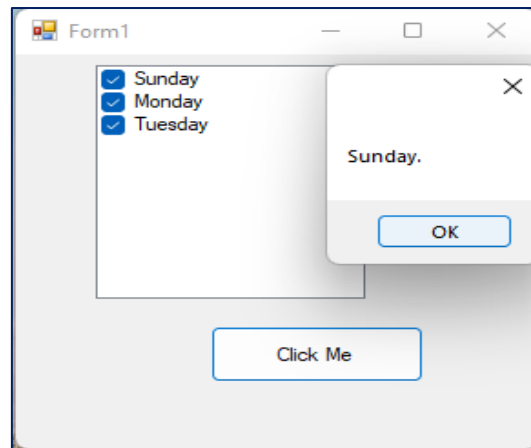
```
string[] days = new[] { "Sunday", "Monday", "Tuesday" };
checkedListBox1.Items.AddRange(days);
for (int i = 0; i < checkedListBox1.Items.Count; i++) {
    checkedListBox1.SetItemChecked(i, true);
}
```



Làm thế nào để nhận giá trị của mục đã chọn từ CheckedListBox?

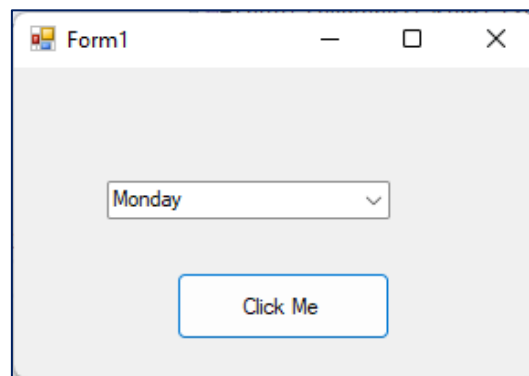
CheckedItems và **CheckedIndices** – truy cập mục đã chọn

```
foreach (string ItemChecked in checkedListBox1.CheckedItems)
{
    MessageBox.Show(ItemChecked + ".");
}
```

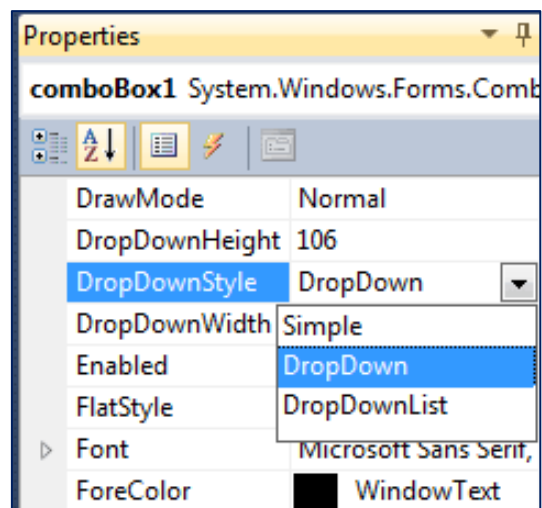


5. ComboBox

ComboBox hiển thị một hộp văn bản được kết hợp với ListBox, cho phép người dùng chọn các mục từ danh sách hoặc nhập một giá trị mới.



DropDownStyle



Thêm các mục vào ComboBox

```
comboBox1.Items.Add("Sunday");
comboBox1.Items.Add("Monday");
comboBox1.Items.Add("Tuesday");
```

Lấy mục được chọn trong ComboBox

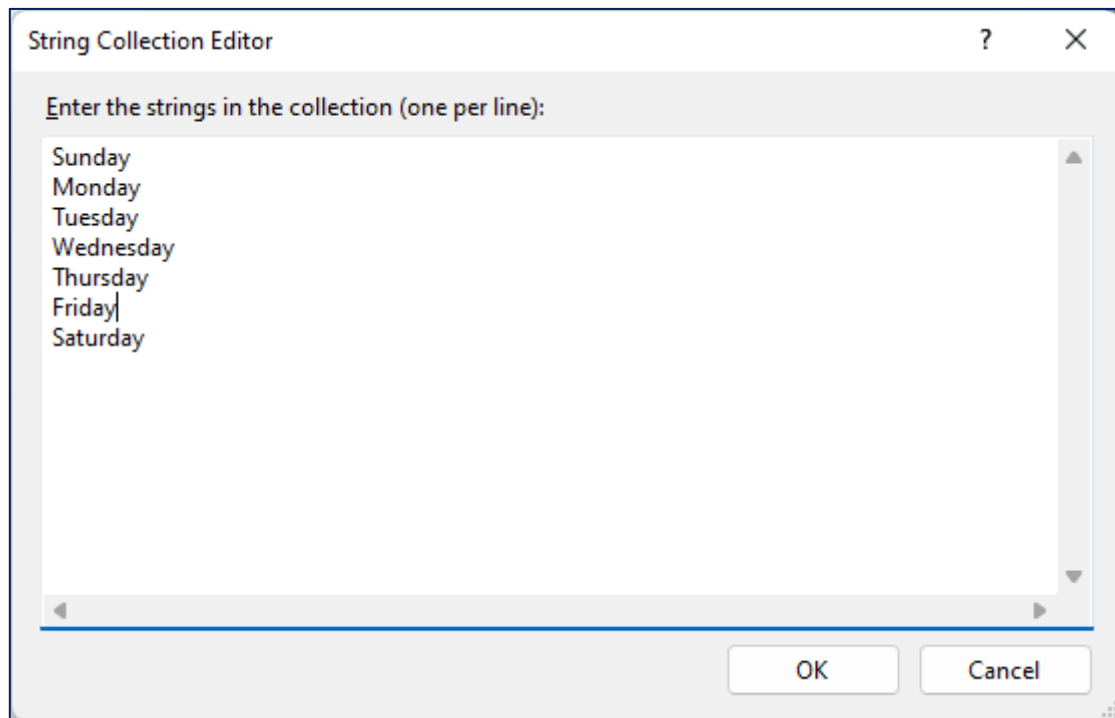
```
string var;  
var = comboBox1.Text;
```

Hoặc

```
var item = this.comboBox1.GetItemText(this.comboBox1.SelectedItem);  
MessageBox.Show(item);
```

Xóa một mục trong ComboBox

Có thể xóa mục tại một chỉ mục được chỉ định hoặc tên của mục muốn xóa.



```
comboBox1.Items.RemoveAt(5);
```

Hoặc

```
comboBox1.Items.Remove("Friday");
```

Hiển thị giá trị được chọn lên ComboBox

```
comboBox1.SelectedItem = "Friday";
```

Hoặc

```
comboBox1.SelectedIndex = comboBox1.FindStringExact("Friday")
```

Bài tập vận dụng

Bài 1: Thiết kế giao diện chương trình như hình bên dưới.

Danh sách items trong Combobox Title: Dr, Mr, Mrs, Miss

Chọn Title, nhập First name và Last name vào Textbox, sau đó nhấn vào Button Add Name thì sẽ thêm 1 phần tử có dạng <Title>.<First name> <Last name> vào cuối danh sách trong Listbox.

Lưu ý kiểm tra và thông báo lên MessageBox nếu chưa nhập dữ liệu

Bài 2: Thiết kế giao diện chương trình như hình bên dưới.

Nhập Tên món vào Textbox, sau đó nhấn vào Button Thêm thì

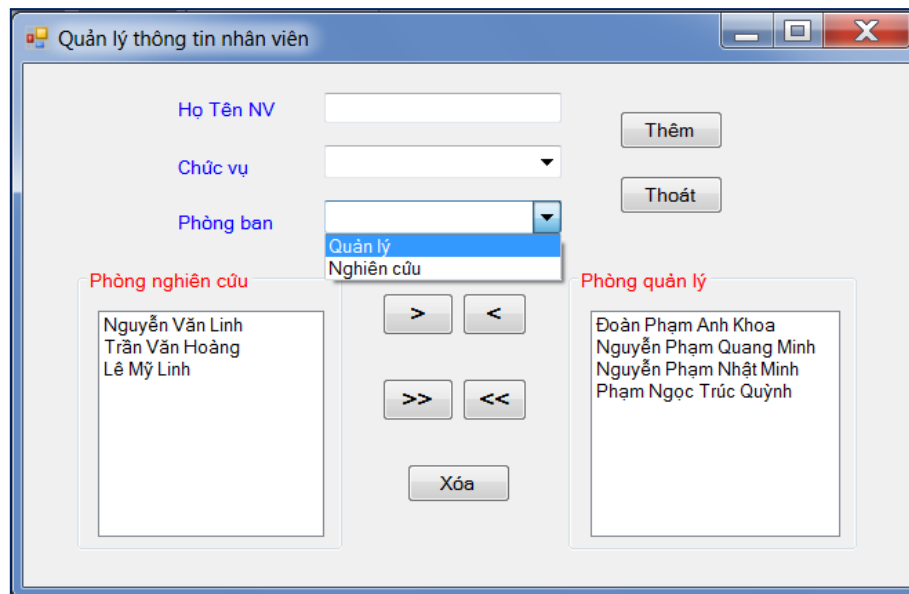
- Nếu chưa có tên món đó trong danh sách (Cả 2 Listbox) sẽ thêm tên món đó vào Listbox bên trái.
- Ngược lại thì hiển thị thông báo “Món ... đã có rồi”.

Các nút >, <: di chuyển 1 món ăn từ Listbox này sang Listbox kia và ngược lại.

Các nút <<, >>: di chuyển tất cả món ăn từ Listbox này sang Listbox kia và ngược lại.

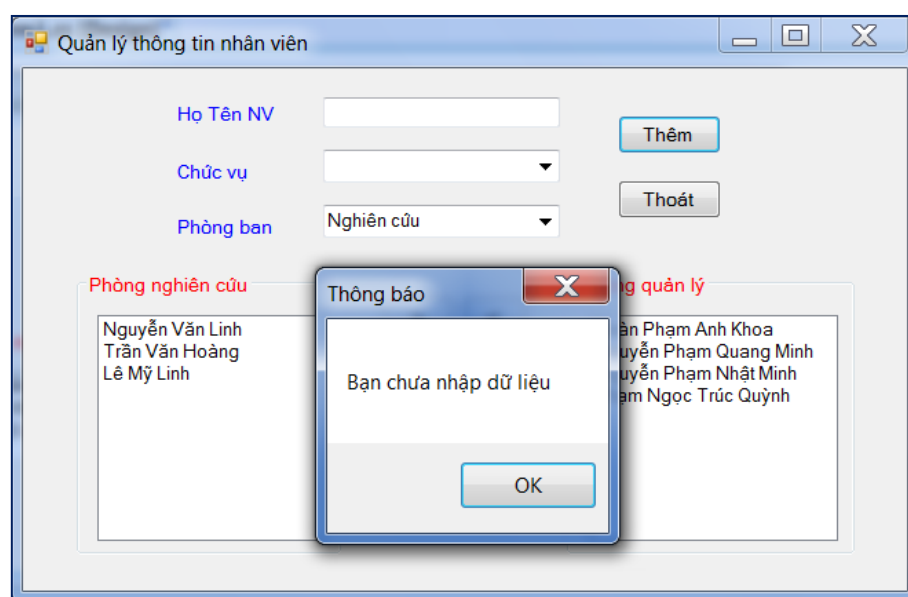
Lưu ý kiểm tra và thông báo lên MessageBox nếu chưa nhập dữ liệu.

Bài tập mở rộng

Bài 1: Thiết kế giao diện chương trình như hình bên dưới.

Yêu cầu chức năng:

- Mặc định phòng ban là phòng nghiên cứu.
- Khi nhấn nút thêm, giá trị nhập tại ô họ tên nhân viên sẽ được cập nhật vào danh sách phòng ban tương ứng được chọn.
- Khi nhấn vào các nút > (ToRight), < (ToLeft), >> (ToRightAll), << (ToLeftAll) danh sách các đối tượng được chọn sẽ di chuyển tương ứng và mất khỏi danh sách ban đầu. (Có hiển thị chú thích: áp dụng ToolTip)
- Khi nhấn nút xóa thì đối tượng tương ứng được chọn sẽ xóa khỏi danh sách.
- Khi nhấn nút thoát thì đóng form, thoát khỏi chương trình.
- Lưu ý kiểm tra và thông báo lên MessageBox nếu chưa nhập dữ liệu



BÀI TẬP THỰC HÀNH TUẦN 05

CONTROLS

(NUMERICUPDOWN, DATETIMEPICKER, LISTVIEW, PICTUREBOX, IMAGELIST)



Mục tiêu

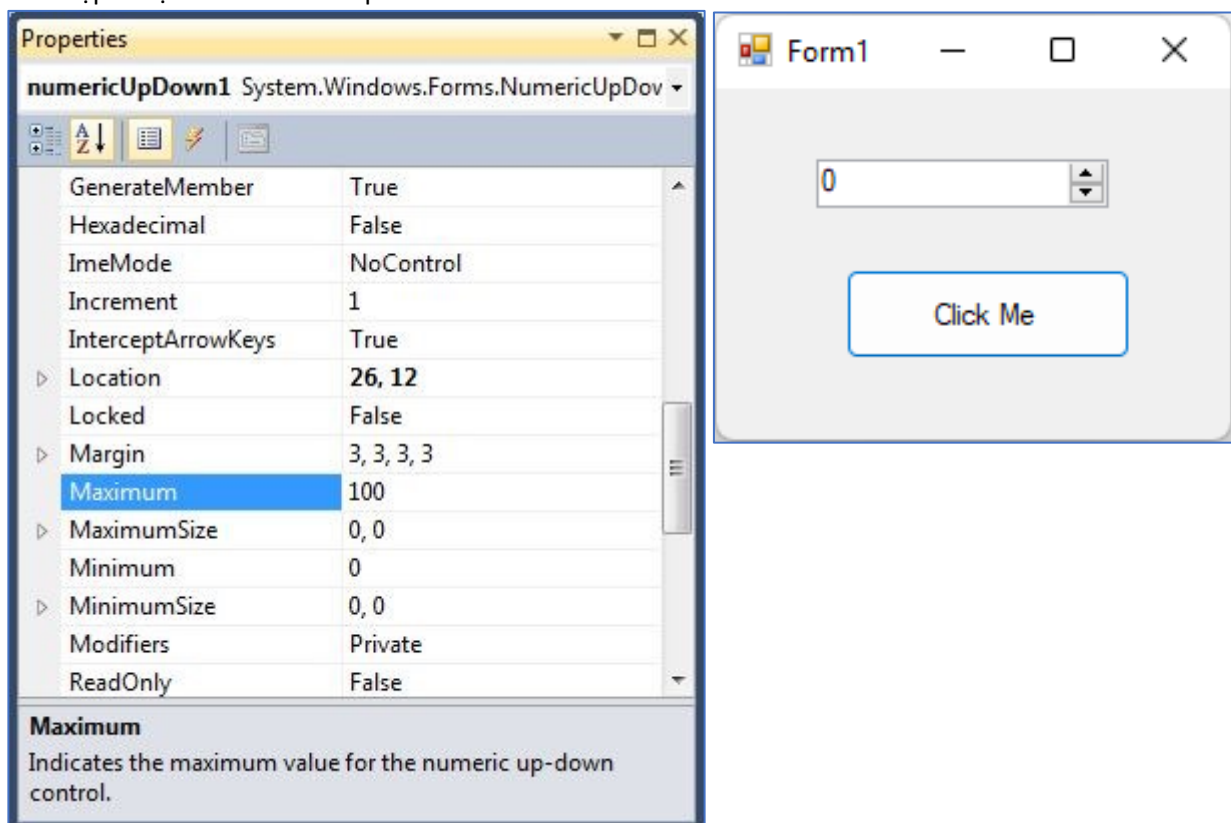
- Tìm hiểu các control cơ bản: NumericUpDown, DateTimePicker, ListView, PictureBox, ImageList.
- Viết được các ứng dụng khác nhau bằng các control được giới thiệu.

Kiến thức

1. NumericUpDown

Điều khiển NumericUpDown cho phép người dùng chọn tăng giảm giá trị bằng cách sử dụng mũi tên lên và xuống.

Thiết lập thuộc tính NumericUpDown.



Giá trị hiện tại.

Thuộc tính Value đại diện cho giá trị hiện được chọn trong một điều khiển.

```
numericUpDown1.Value = 10;
MessageBox.Show(numericUpDown1.Value.ToString());
```


Phạm vi tối thiểu và tối đa (Minimum and Maximum Range)

Thuộc tính **Minimum** và **Maximum** được sử dụng để đặt giá trị tối thiểu và tối đa của điều khiển **NumericUpDown**.

```
numericUpDown1.Minimum = -10;  
numericUpDown1.Maximum = 10;
```

Gia tăng (Increment)

Thuộc tính **Increment** được sử dụng để đặt giá trị tăng hoặc giảm khi nhấp vào các nút lên và xuống.

```
numericUpDown1.Increment = 5;
```

Vị trí thập phân (DecimalPlaces)

Chúng ta có thể định dạng số của một điều khiển **NumericDropDown** bằng cách sử dụng thuộc tính **DecimalPlaces**.

```
numericUpDown1.DecimalPlaces = 4;
```

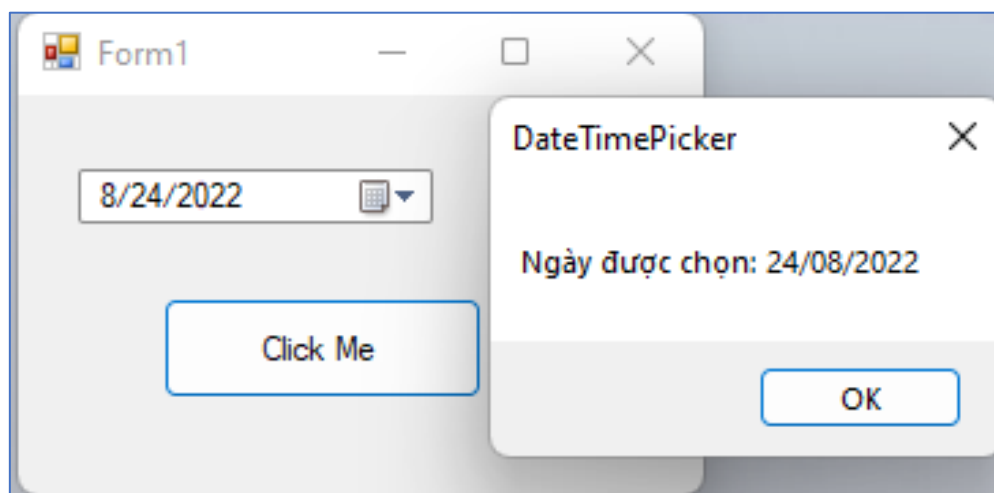
Hexadecimal

Chúng ta có thể sử dụng điều khiển **NumericDropDown** để hiển thị giá trị số ở định dạng thập lục phân

```
numericUpDown1.Hexadecimal = true;
```

2. DateTimePicker

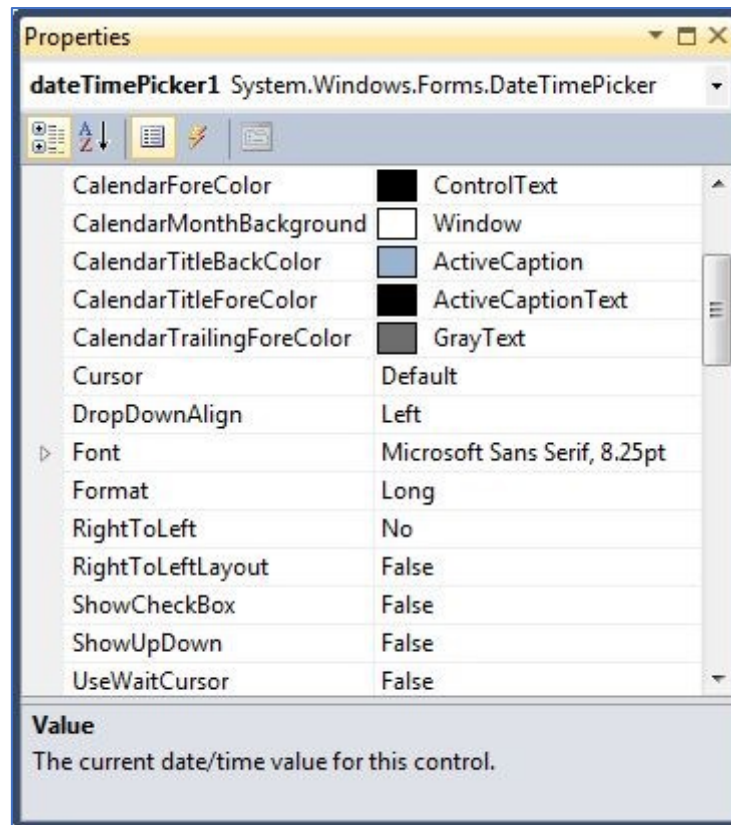
Điều khiển **DateTimePicker** cho phép hiển thị và lấy ngày giờ từ người dùng với định dạng được chỉ định.



Điều khiển **DateTimePicker** có hai phần, nhãn hiển thị ngày đã chọn và lịch bật lên cho phép người dùng chọn ngày mới. Thuộc tính quan trọng nhất của **DateTimePicker** là thuộc tính **Value**, chứa ngày và giờ đã chọn.

```
dateTimePicker1.Value = DateTime.Today;
```

Thiết lập các giá trị thuộc tính



Text và Value

Thuộc tính **Text** đại diện cho văn bản của điều khiển DateTimePicker. Thuộc tính **Value** đại diện cho giá trị ngày giờ được đặt hiện tại trong một điều khiển.

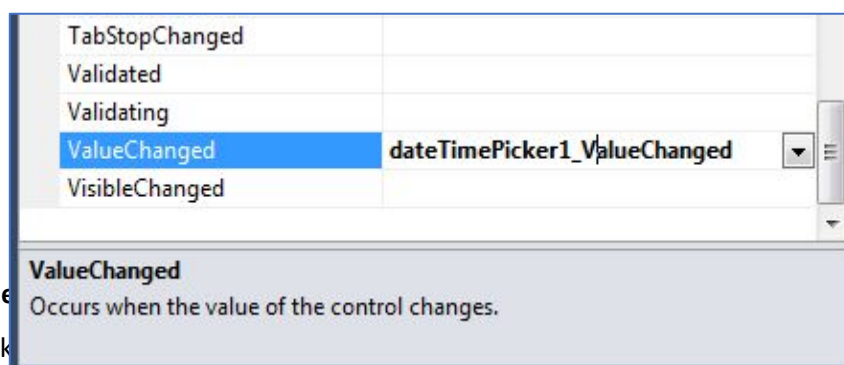
```
MessageBox.Show("Ngày được chọn: " + dateTimePicker1.Text, "DateTimePicker");
MessageBox.Show("Ngày          được          chọn:          "          +
dateTimePicker1.Value.Date.ToString("dd/MM/yyyy"), "DateTimePicker");
```

Định dạng lịch

Thuộc tính **MinDate** và **MaxDate** được sử dụng để đặt giá trị tối thiểu và tối đa của ngày và giờ có thể được chọn bởi một điều khiển.

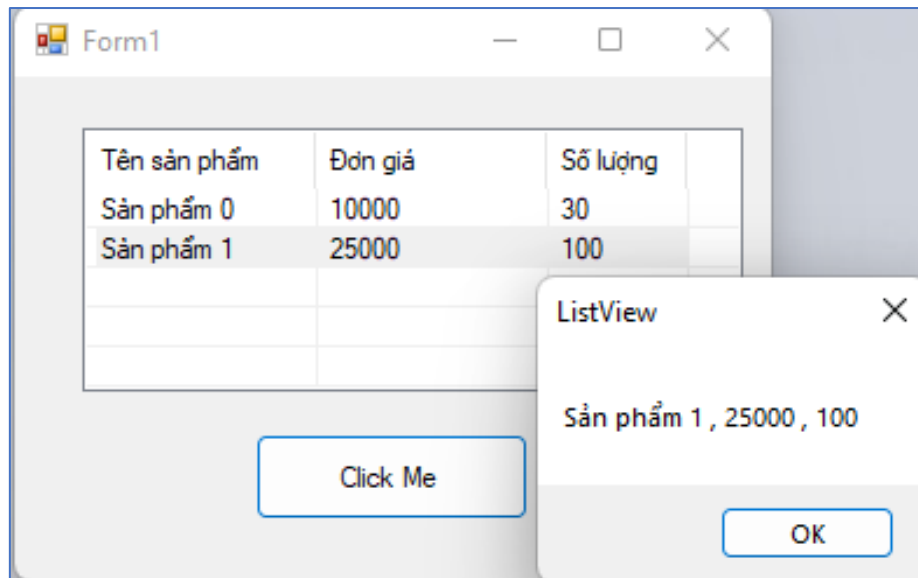
```
dateTimePicker1.MinDate = new DateTime(2020, 1, 1);
dateTimePicker1.MaxDate = DateTime.Today;
```

Xử lý sự kiện DateTimePicker



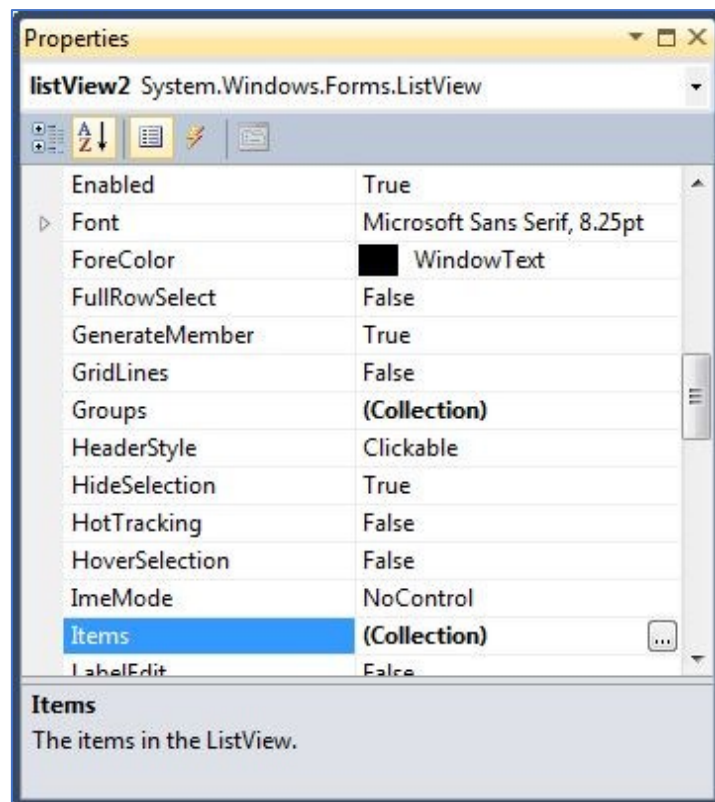
3. ListView

Điều khiển



Set và Get các phần tử của ListView

Thuộc tính `Items` được sử dụng để thêm và làm việc với các phần tử trong `ListView`. Chúng ta có thể thêm các phần tử vào `ListView` tại thời điểm thiết kế từ Cửa sổ `Properties` bằng cách nhấp vào `Items Collection`.



Có thể thêm các phần tử vào `ListView` bằng dòng lệnh theo cú pháp:

Cú pháp:

```
public int Add(object item);
```

Ví dụ:

```
listView1.Items.Add("Sản phẩm 0");
```

```
listView1.Items.Add("Sản phẩm 1");
```

Lấy tất cả các phần tử trong ListView

```
System.Text.StringBuilder sb = new System.Text.StringBuilder();
foreach (ListViewItem item in listView1.Items) {
    sb.Append(item.Text);
    sb.Append(" ");
}
MessageBox.Show(sb.ToString(), "ListView");
```

Lấy phần tử được chọn trong ListView

```
foreach (ListViewItem item in listView1.SelectedItems) {
    MessageBox.Show(item.Text + " , " + item.SubItems[1].Text + " , " + item.SubItems[2].Text,
        "ListView");
}
```

Sắp xếp các phần tử trong ListView

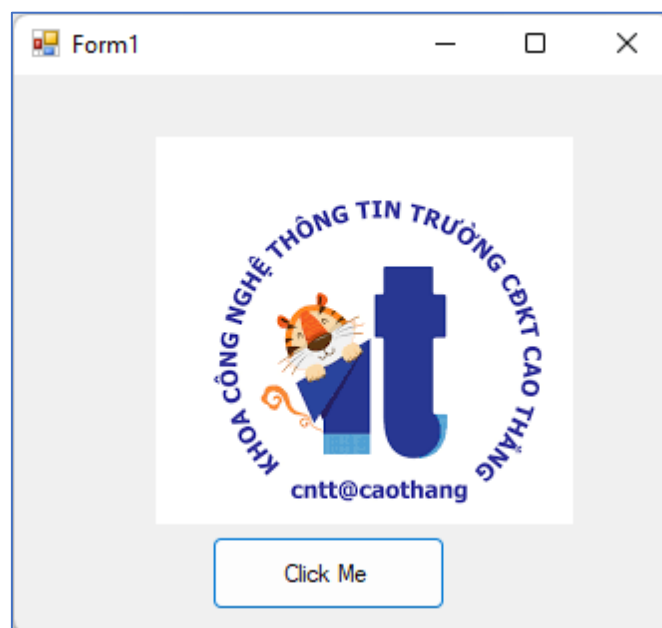
```
listView1.Sorting = SortOrder.Ascending;
```

Thêm Checkbox vào ListView

```
listView1.CheckBoxes = true;
```

4. PictureBox

Điều khiển PictureBox của Windows Forms được sử dụng để hiển thị hình ảnh ở định dạng bitmap, GIF, icon hoặc JPEG.



Có thể lập trình thay đổi hình ảnh được hiển thị PictureBox.

```
pictureBox1.Image = Image.FromFile("C:\\download.png");
```

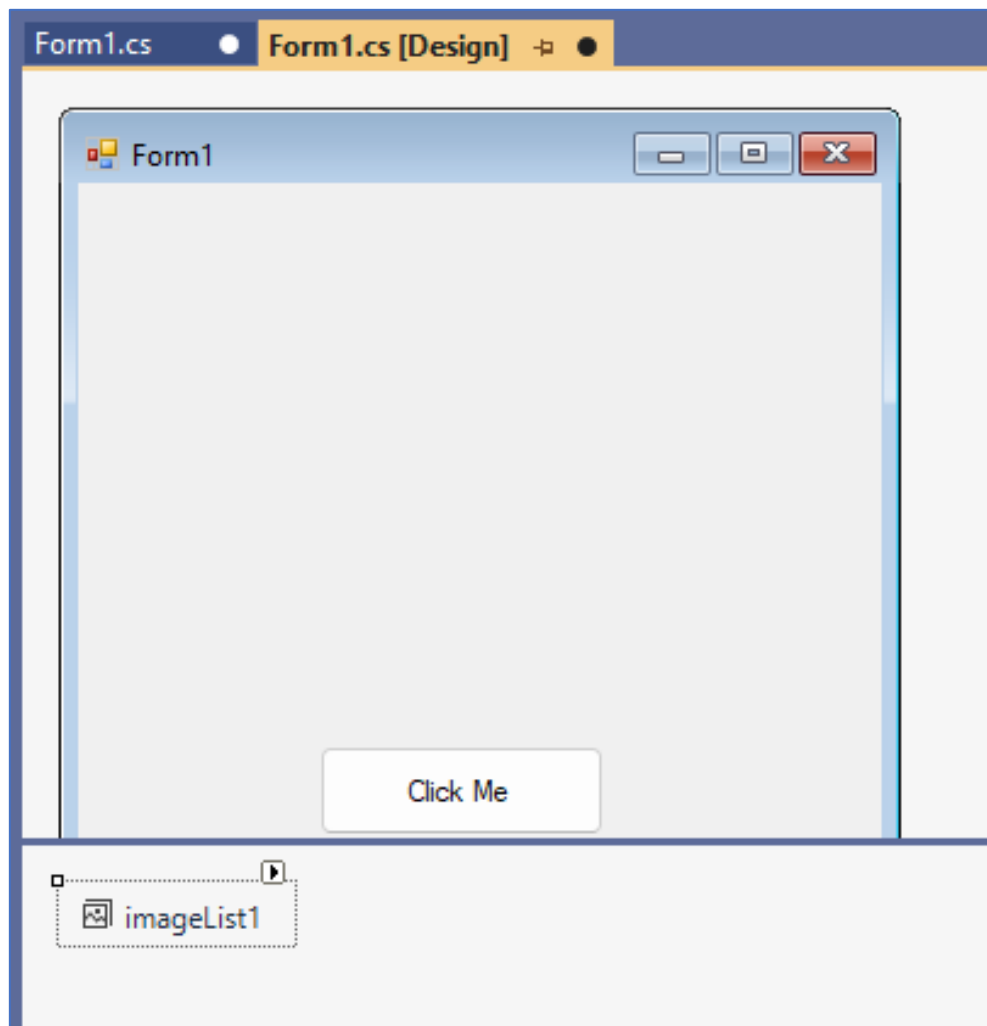
Thuộc tính `SizeMode`, được đặt thành các giá trị trong kiểu liệt kê `PictureBoxSizeMode`, kiểm soát việc cắt và định vị hình ảnh trong vùng hiển thị.

- **AutoSize** - Định kích thước hộp hình ảnh cho hình ảnh.
- **CenterImage** - Căn giữa hình ảnh trong hộp hình ảnh.
- **Normal** - Đặt góc trên bên trái của hình ở phía trên bên trái trong hộp hình ảnh
- **StretchImage** - Cho phép kéo dài hình ảnh trong mã (code)

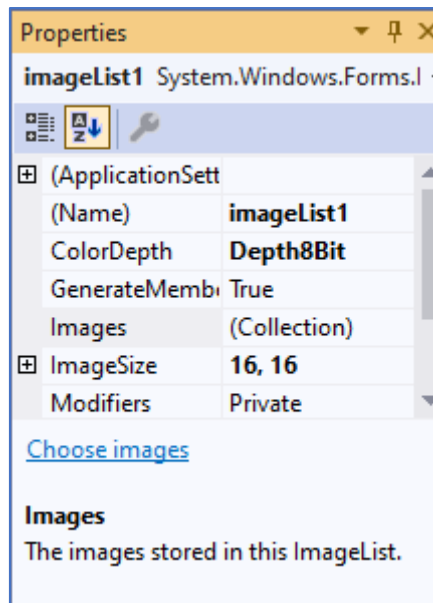
```
pictureBox1.SizeMode = PictureBoxSizeMode.StretchImage;
```

5. ImageList

`ImageList` là một điều khiển hỗ trợ thường được sử dụng bởi các điều khiển khác, chẳng hạn như `ListView` nhưng được hiển thị như một thành phần cho các nhà phát triển. Chúng ta có thể sử dụng thành phần này trong các ứng dụng của mình khi chúng ta xây dựng các điều khiển của riêng mình, chẳng hạn như thư viện ảnh hoặc điều khiển xoay vòng hình ảnh.



Thiết lập giá trị thuộc tính `ImageList`



Thêm ảnh vào imagelist

```
imageList1.Images.Add(Image.FromFile(@"C:\Images\Garden.jpg"));
imageList1.Images.Add(Image.FromFile(@"C:\Images\Tree.jpg"));
imageList1.Images.Add(Image.FromFile(@"C:\Images\Waterfall.jpg"));
```

Bài tập vận dụng

Bài 1: Thiết kế giao diện chương trình như hình bên dưới:

Nhập thông tin vào phần “thông tin nhân viên”, khi nhấn các nút lệnh sẽ thực hiện như sau:

1. Nút Thêm: Thêm các thông tin vào listview. Chú ý : Kiểm tra thông tin họ tên nhân viên không được rỗng, nếu rỗng thì không cho thêm.
2. Nút Xóa: Xóa 1 dòng trên listview (chú ý nếu người dùng chưa chọn 1 dòng trên listview thì yêu cầu người dùng phải chọn rồi mới xóa).
3. Nút Sửa: Cập nhật thông tin đã điều chỉnh 1 dòng trên listview.
4. Thoát: thoát khỏi chương trình.
5. Khi chọn 1 dòng trên listview thì nội dung của dòng được chọn sẽ được thể hiện trên các ô “Thông tin nhân viên”.

Bài tập mở rộng

BÀI TẬP THỰC HÀNH TUẦN 06

CONTROLS

(MENU, MDI, TABCONTROL, TREEVIEW, PANEL)



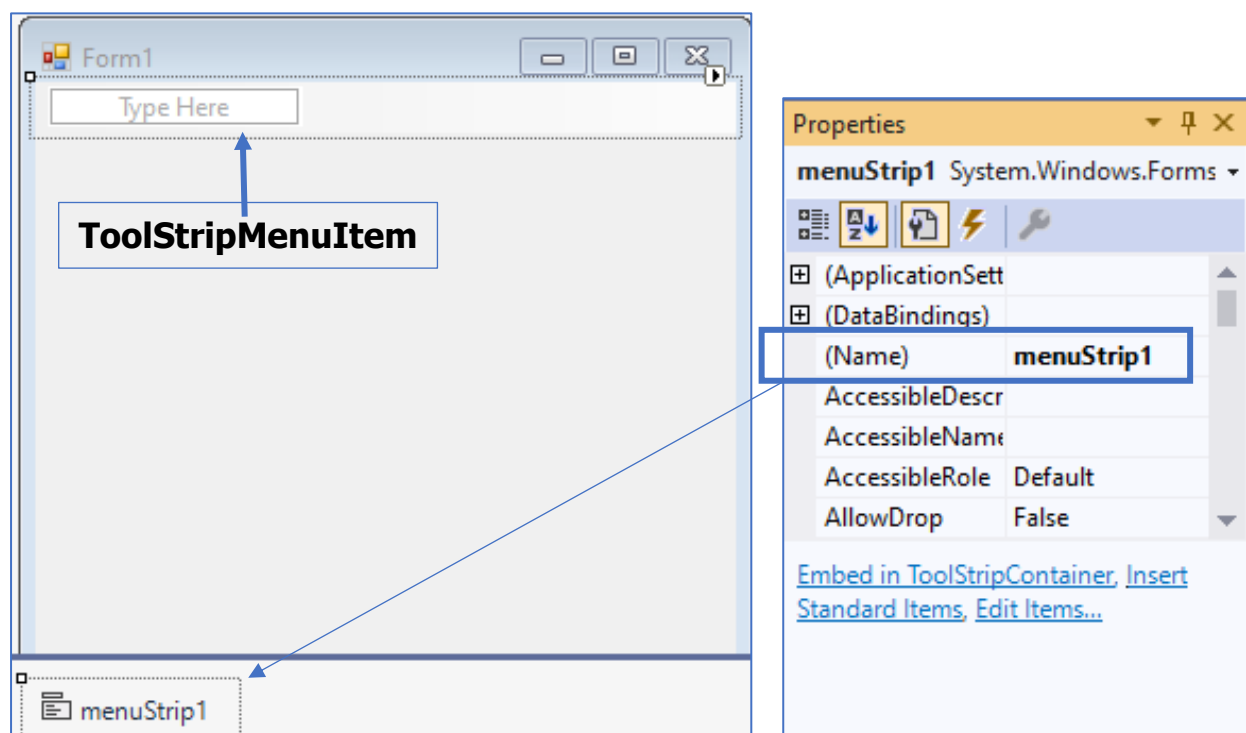
Mục tiêu

- Tìm hiểu các control cơ bản: Menu, MDI, TabControl, TreeView, Panel.
- Viết được các ứng dụng khác nhau bằng các control được giới thiệu.

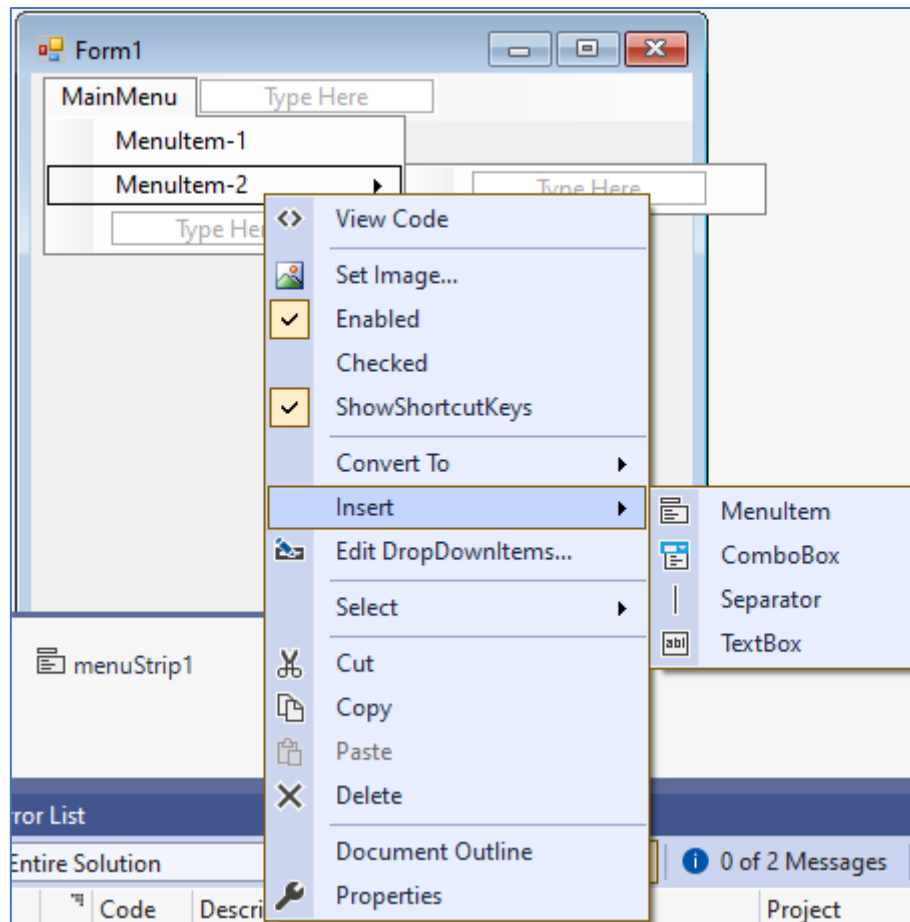
Kiến thức

1. Menu

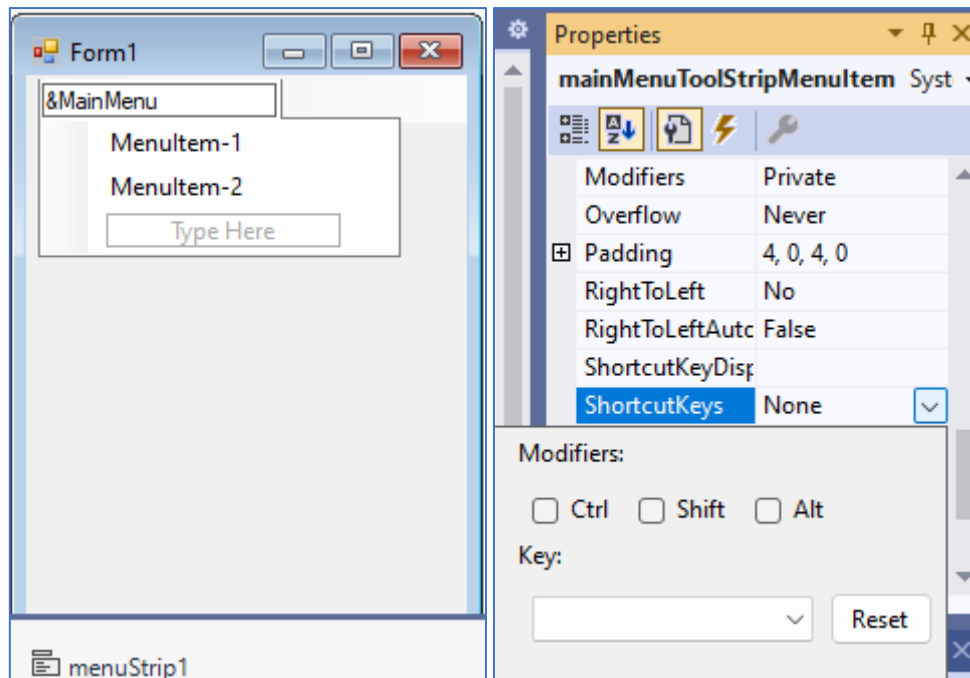
Menu trên Windows Form được tạo bằng đối tượng menuStrip, là một tập hợp các đối tượng MenuItem. menuStrip là vùng chứa cho cấu trúc Menu của biểu mẫu và các menu được làm bằng các đối tượng MenuItem đại diện cho các phần riêng lẻ của menu



Nếu cần một thanh phân tách, nhấp chuột phải vào menu, sau đó tới **Insert-> Separator**.



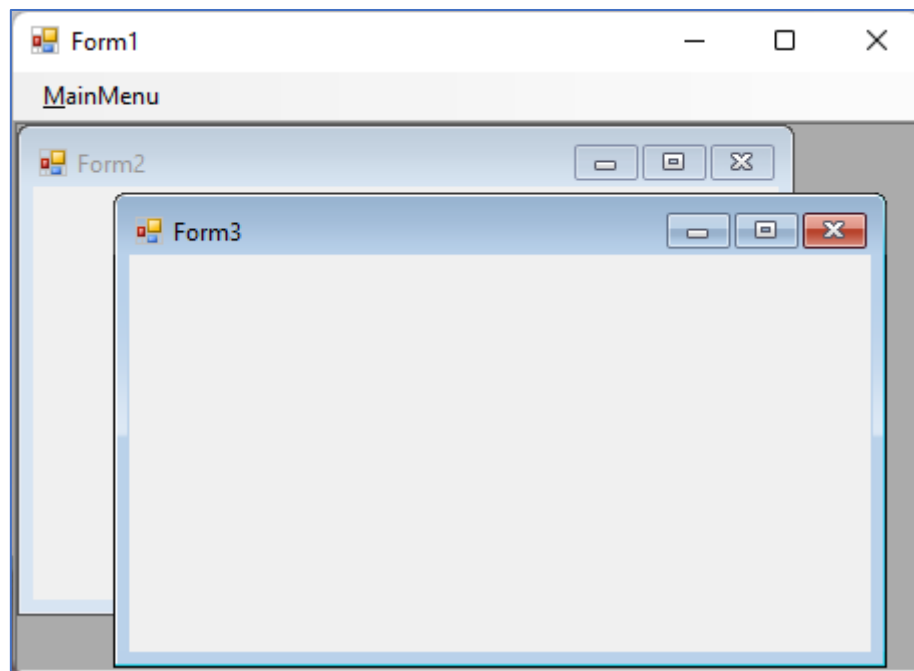
Thiết lập Short cut key cho menu item.



Sau khi tạo Menu trên Form, phải nhấp đúp chuột vào từng mục menu và viết các lệnh vào đó tùy theo yêu cầu ứng dụng. Môi trường Visual Studio là một ví dụ về Giao diện Nhiều Tài liệu (MDI) và notepad là một ví dụ về ứng dụng MDI.

2. MDI

Các chương trình Giao diện Nhiều Tài liệu (Multiple Document Interface - MDI) có thể hiển thị nhiều cửa sổ con bên trong chúng. Điều này trái ngược với các ứng dụng giao diện tài liệu đơn lẻ (SDI), chỉ có thể thao tác một tài liệu tại một thời điểm.



Bất kỳ cửa sổ nào cũng có thể trở thành MDI cha, nếu đặt thuộc tính **IsMdiContainer** thành **True**.

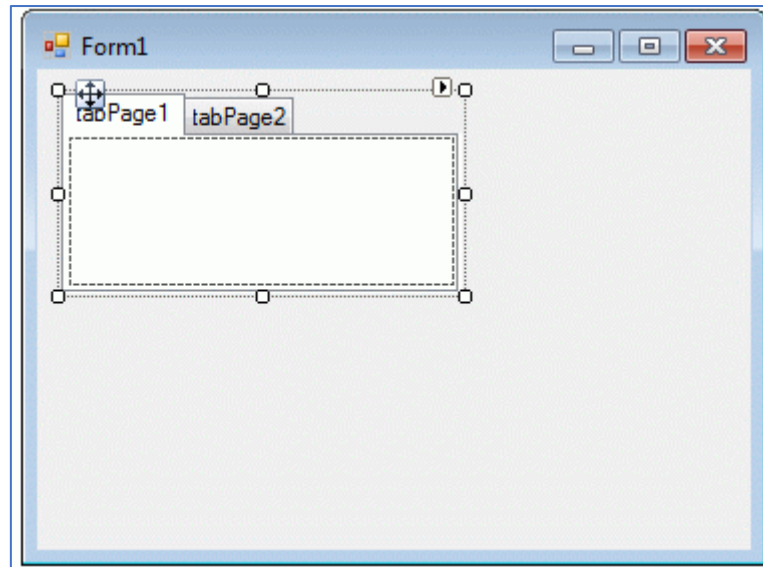
```
IsMdiContainer = true;

private void menuItem1ToolStripMenuItem_Click(object sender, EventArgs e)
{
    Form2 frm2 = new Form2();
    frm2.Show();
    frm2.MdiParent = this;
}

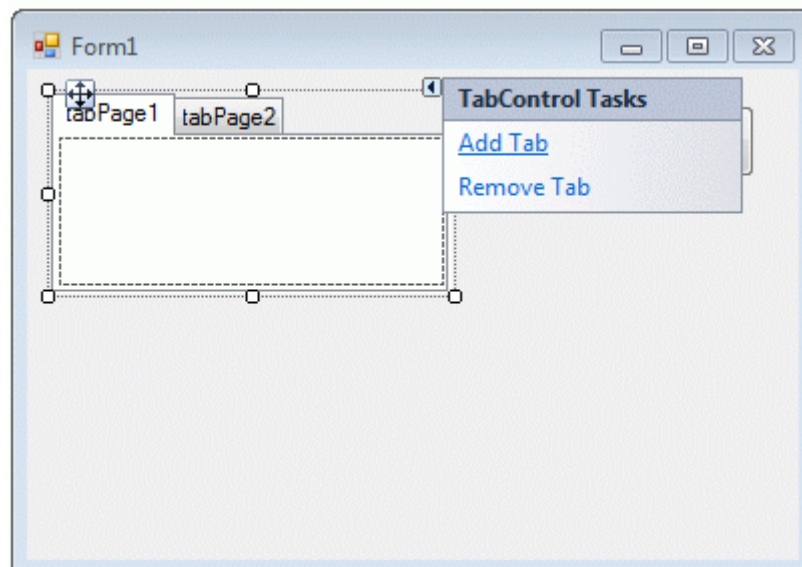
1 reference
private void menuItem2ToolStripMenuItem_Click(object sender, EventArgs e)
{
    Form3 frm3 = new Form3();
    frm3.Show();
    frm3.MdiParent = this;
}
```

3. TabControl

TabControl quản lý các trang tab trong đó mỗi trang có thể lưu trữ các điều khiển con khác nhau.

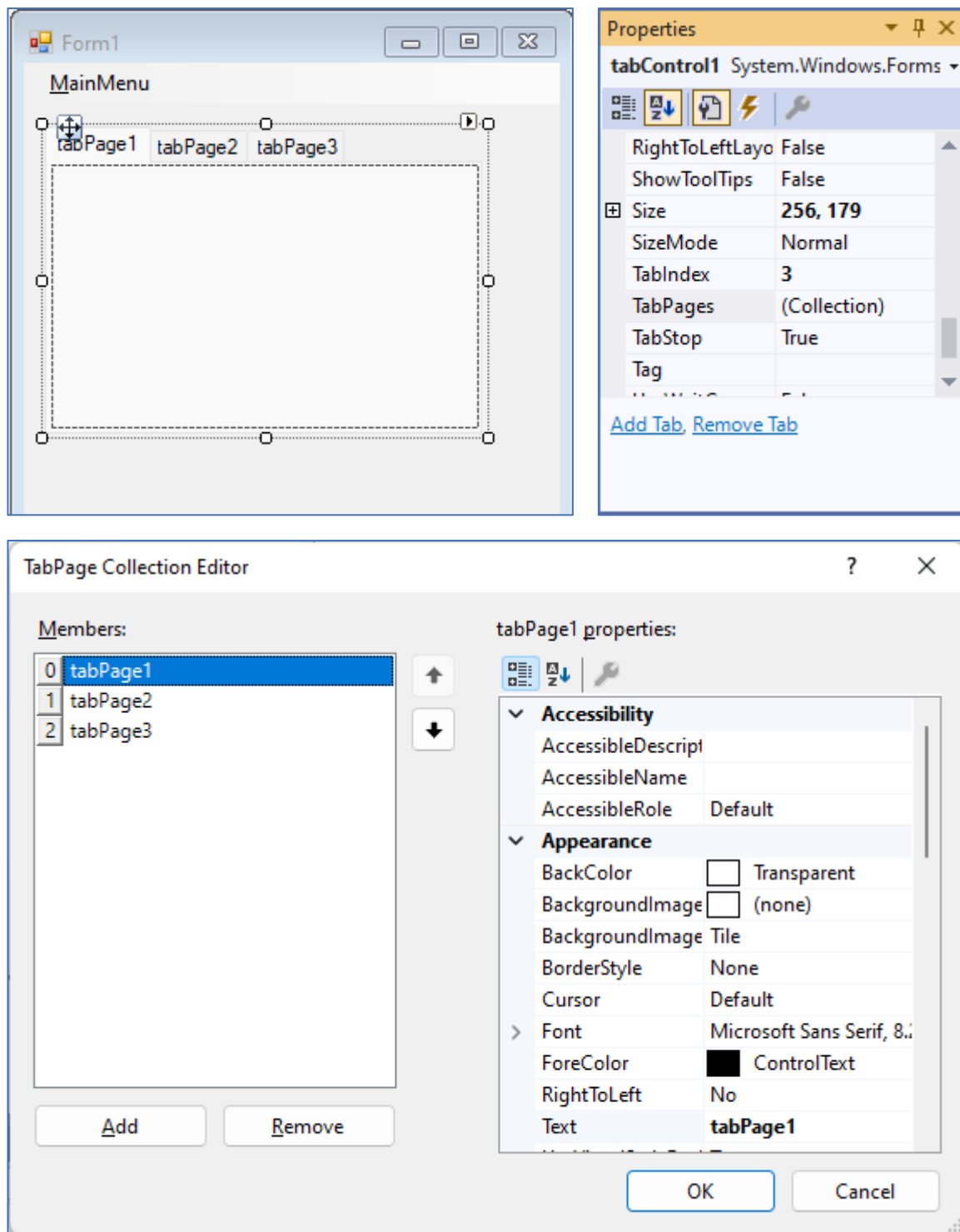


TabControl chỉ là một vùng chứa và không có giá trị nếu không có các trang tab. Chúng ta có thể thêm và xóa các trang tab bằng cách nhấp vào nút điều khiển Tasks và chọn các liên kết Add và Remove



Chỉnh sửa các Tabpage:

- Chọn thuộc tính Tabpages của TabControl
- Sử dụng màn hình Tabpage Collection Editor để chỉnh sửa

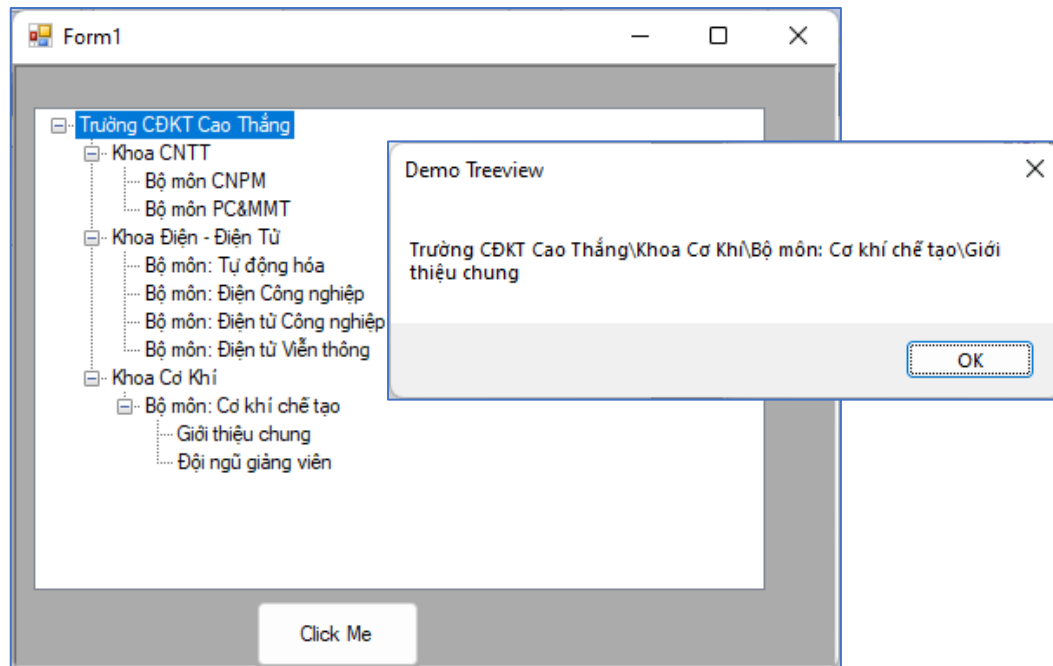


Bổ sung Control vào TabControl

- Chọn Tabpage cần thêm control
- Kéo control từ Toolbox thả vào Tabpage đã chọn

4. TreeView

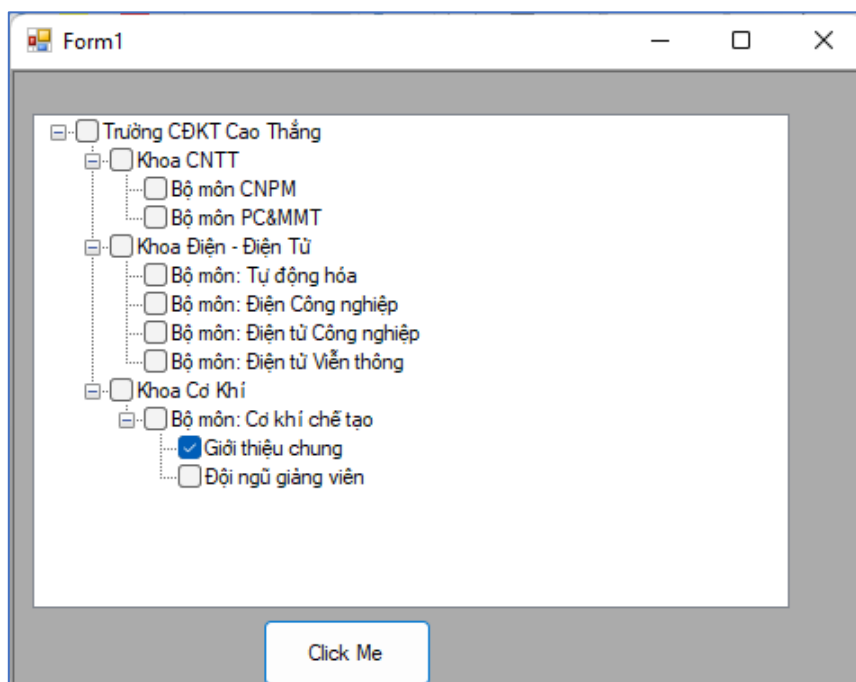
Điều khiển TreeView chứa một hệ thống phân cấp các điều khiển TreeViewItem. Nó cung cấp một cách để hiển thị thông tin trong cấu trúc phân cấp bằng cách sử dụng các nút có thể thu gọn.



Người dùng có thể mở rộng `TreeNode` bằng cách nhấp vào nút dấu cộng (+), nếu một nút được hiển thị bên cạnh `TreeNode` hoặc có thể mở rộng `TreeNode` bằng cách gọi phương thức `TreeNode.Expand`. Cũng có thể điều hướng qua các chế độ xem dạng cây với các thuộc tính khác nhau: `FirstNode`, `LastNode`, `NextNode`, `PrevNode`, `NextVosystemNode`, `PrevVibleNode`. Phương thức `fullpath` của điều khiển `treeview` cung cấp đường dẫn từ nút gốc đến nút đã chọn

```
treeView1.SelectedNode.FullPath.ToString();
```

Các nút trên `treeview` có thể hiển thị thành các checkbox. Để hiển thị các checkbox, hãy đặt thuộc tính **CheckBoxes** của **TreeView** thành **true**.



```
treeView1.CheckBoxes = true;
```

```

private void Form1_Load(object sender, EventArgs e)
{
    TreeNode tNode;
    tNode = treeView1.Nodes.Add("Trường CĐKT Cao Thắng");

    treeView1.Nodes[0].Nodes.Add("Khoa CNTT");
    treeView1.Nodes[0].Nodes[0].Nodes.Add("Bộ môn CNPM");
    treeView1.Nodes[0].Nodes[0].Nodes.Add("Bộ môn PC&MMT");

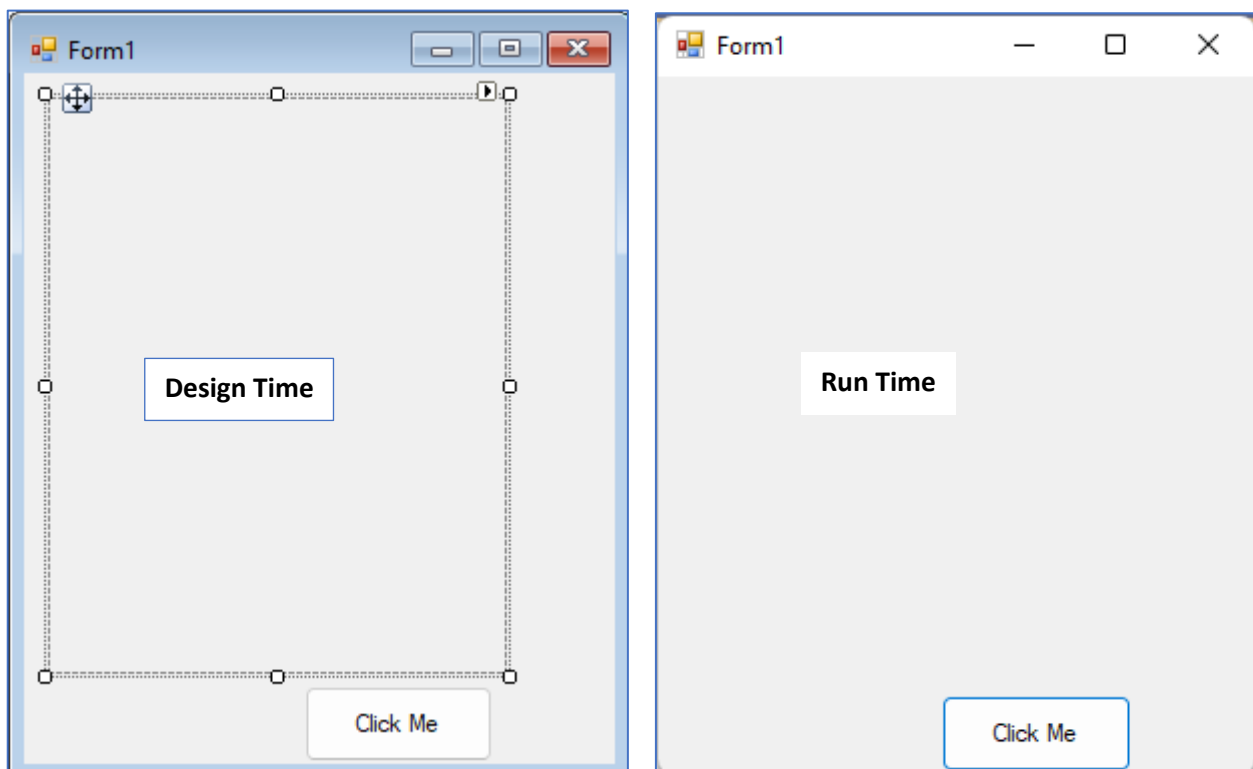
    treeView1.Nodes[0].Nodes.Add("Khoa Điện - Điện Tử");
    treeView1.Nodes[0].Nodes[1].Nodes.Add("Bộ môn: Tự động hóa");
    treeView1.Nodes[0].Nodes[1].Nodes.Add("Bộ môn: Điện Công nghiệp");
    treeView1.Nodes[0].Nodes[1].Nodes.Add("Bộ môn: Điện tử Công nghiệp");
    treeView1.Nodes[0].Nodes[1].Nodes.Add("Bộ môn: Điện tử Viễn thông");

    treeView1.Nodes[0].Nodes.Add("Khoa Cơ Khí");
    treeView1.Nodes[0].Nodes[2].Nodes.Add("Bộ môn: Cơ khí chế tạo");
    treeView1.Nodes[0].Nodes[2].Nodes[0].Nodes.Add("Giới thiệu chung");
    treeView1.Nodes[0].Nodes[2].Nodes[0].Nodes.Add("Đội ngũ giảng viên");
}
1 reference
private void button1_Click_1(object sender, EventArgs e)
{
    MessageBox.Show(treeView1.SelectedNode.FullPath.ToString(), "Demo Treeview");
}

```

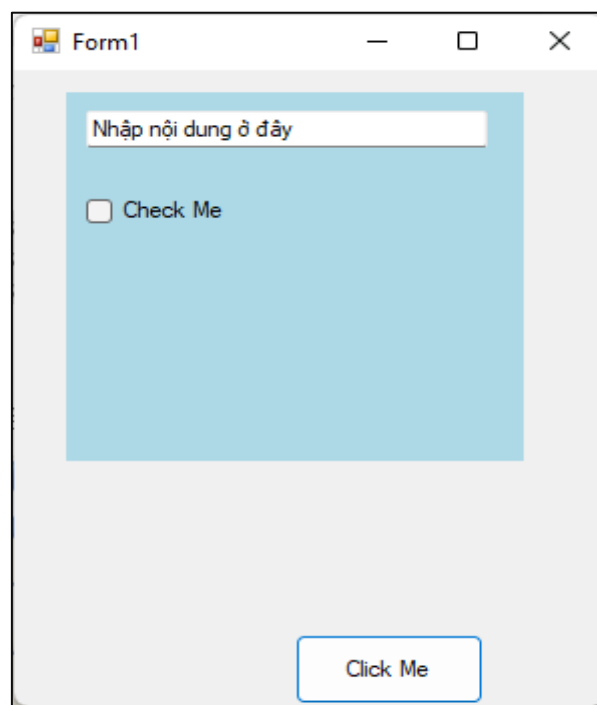
5. Panel

Panel Control là một điều khiển vùng chứa để lưu trữ một nhóm các điều khiển con tương tự. Một trong những cách sử dụng chính của Panel Control là khi cần hiển thị và ẩn một nhóm điều khiển. Thay vì hiển thị và ẩn các điều khiển riêng lẻ, có thể chỉ cần ẩn và hiển thị một panel duy nhất và tất cả các điều khiển con sẽ hiển thị và ẩn theo.



Có thể thêm điều khiển vào Panel bằng cách kéo và thả điều khiển vào Panel. Chúng ta có thể thêm các điều khiển vào Panel bằng cách sử dụng phương pháp **Add**.

```
private void Form1_Load(object sender, EventArgs e)
{
    Panel dynamicPanel = new Panel();
    dynamicPanel.Location = new System.Drawing.Point(26, 12);
    dynamicPanel.Name = "Panel1";
    dynamicPanel.Size = new System.Drawing.Size(228, 200);
    dynamicPanel.BackColor = Color.LightBlue;
    TextBox textBox1 = new TextBox();
    textBox1.Location = new Point(10, 10);
    textBox1.Text = "Nhập nội dung ở đây";
    textBox1.Size = new Size(200, 30);
    CheckBox checkBox1 = new CheckBox();
    checkBox1.Location = new Point(10, 50);
    checkBox1.Text = "Check Me";
    checkBox1.Size = new Size(200, 30);
    dynamicPanel.Controls.Add(textBox1);
    dynamicPanel.Controls.Add(checkBox1);
    Controls.Add(dynamicPanel);
}
```



Hiển thị và ẩn Panel

Thay vì hiển thị và ẩn các điều khiển riêng lẻ, chúng ta có thể nhóm các điều khiển mà chúng ta muốn hiển thị và ẩn và đặt chúng trên hai Panel khác nhau và hiển thị và ẩn các Panel. Để hiển thị và ẩn Panel, chúng ta sử dụng thuộc tính **Visible**.

```
dynamicPanel.Visible = false;
```

```
dynamicPanel.Visible = false;
```

Bài tập vận dụng

Bài 1: Thiết kế giao diện chương trình như hình bên dưới.

E-Orders Application

Tất cả

- Món ăn
 - Món khai vị
 - Món nướng
 - Món lẩu
 - Món tráng miệng
- Thức uống
 - Bia
 - Nước ngọt
 - Khác

Tên sản phẩm	Giá (VNĐ)	Mô tả sản phẩm
Cà phê sữa	15,000	Hương vị thơm ngon của chúng tôi
Nước trái cây	25,000	Nước trái cây ép nguyên chất đến từ trái ...
Món khai vị	50,000	Ngoài ra, cửa hàng còn phục vụ các món...
Heineken	25,000	Bia Heineken xuất xứ từ Hà Lan
Lẩu cá kèo	10,000	Lẩu cá kèo của cửa hàng với nguyên liệu...
Bạch tuộc nướng	45,000	Món bạch tuộc nướng ướp gia vị cay của ...
Coca cola	15,000	Nước giải khát coca cola
Thạch trái cây	25,000	Món thạch trái cây thơm ngon bổ dưỡng

Thông tin chi tiết sản phẩm

Loại sản phẩm: Món ăn

Tên món: Món khai vị

Giá: 50,000

Chi tiết: Ngoài ra, cửa hàng còn phục vụ các món ăn Việt Nam.

View: ☒ Detail ☐ Large ☐ Small ☐ List

Yêu cầu:

- Khi load form: chương trình load tất cả các món ăn lên listview.
- Khi người dùng chọn 1 món trên listview, thông tin chi tiết của món ăn sẽ hiển thị sang “thông tin chi tiết sản phẩm”. Người dùng có thể thay đổi thông tin sản phẩm đã chọn thông qua các chức năng: thêm, xóa, sửa sản phẩm.
- Khi load thông tin sản phẩm lên listview, chế độ view mặc định là Detail. Người dùng có thể thay đổi chế độ view theo các tùy chọn được liệt kê trong mục View.

Bài tập mở rộng

KIỂM TRA LẦN 01

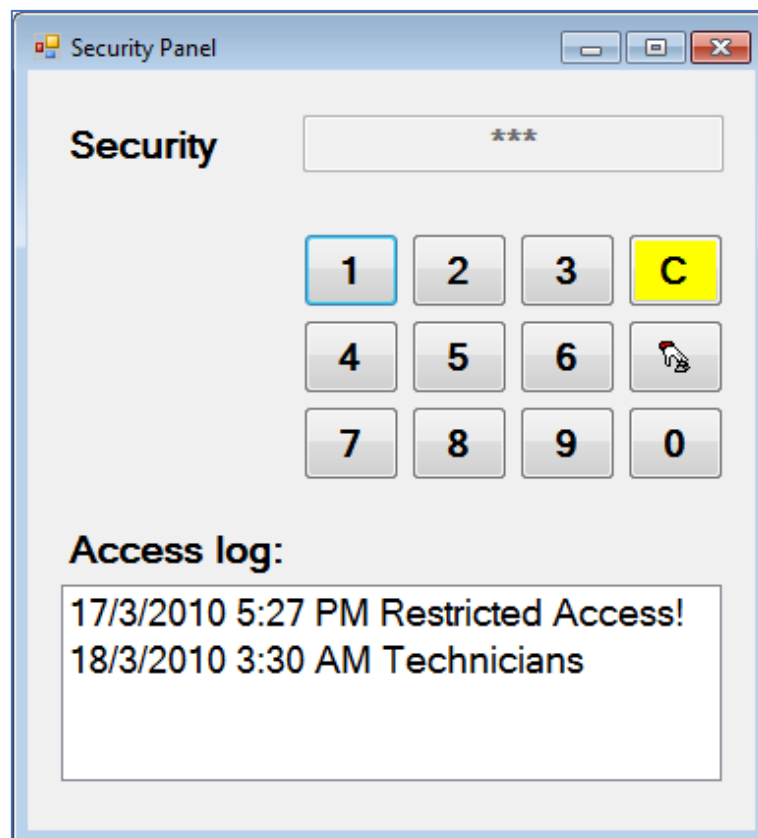


Mục tiêu

- Ôn tập các control cơ bản đã học
- Thực hiện bài kiểm tra thực hành số 1

Ôn tập

Bài tập 1: Thiết kế giao diện Security Panel như hình bên dưới và thực hiện yêu cầu:



Mỗi nhân viên muốn vào phòng Lab thì phải nhập security code được cung cấp. Nếu họ nhập đúng security code được cung cấp thì sẽ có trạng thái là **Granted**, ngược lại là **Denied**.

Tất cả thông tin truy cập đều được hiển thị ở khung Access log.

- Nếu trạng thái là Granted thì ngày-giờ, nhóm nhân viên (*Technicians, Custodians, Scientist*) sẽ được hiển thị
- Nếu trạng thái là Denied thì chỉ hiển thị ngày-giờ và thông tin “Access denied”
- Ngoài ra nếu nhân viên chỉ nhấn 1 con số security code thì hiển thị ngày-giờ và thông tin “Restricted Access!”

Bảng security code:

Security code hợp lệ	Nhóm nhân viên
6990, 6999	<i>Technicians</i>
0100	<i>Custodians</i>
9998, 1006 – 1008	<i>Scientist</i>

Chức năng các phím:

- **Số từ 0 đến 9:** nhập security code
- **Phím C:** xóa security code



- **Phím** : kiểm tra security code có hợp lệ hay không và hiển thị thông tin lên Access log.

Bài tập 2: Thiết kế giao diện như hình bên dưới và thực hiện các yêu cầu sau:

- Danh sách phim trong combobox “Chọn phim”:

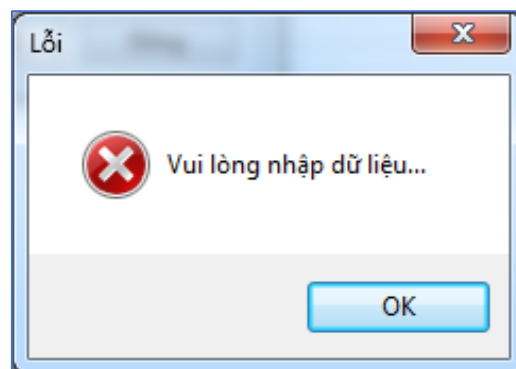
- Danh sách rạp trong combobox “Chọn rạp”:

- Một số ràng buộc về phim và rạp như sau:

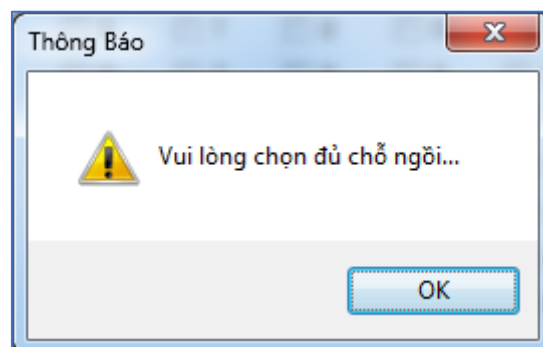
Phim	Rạp
Hai Phượng	1, 2
Alita	3, 4
Chị 13	5

Rạp	Dãy ghế
1	A → J
2	A → H
3	A → I
4	A → G
5	A → J

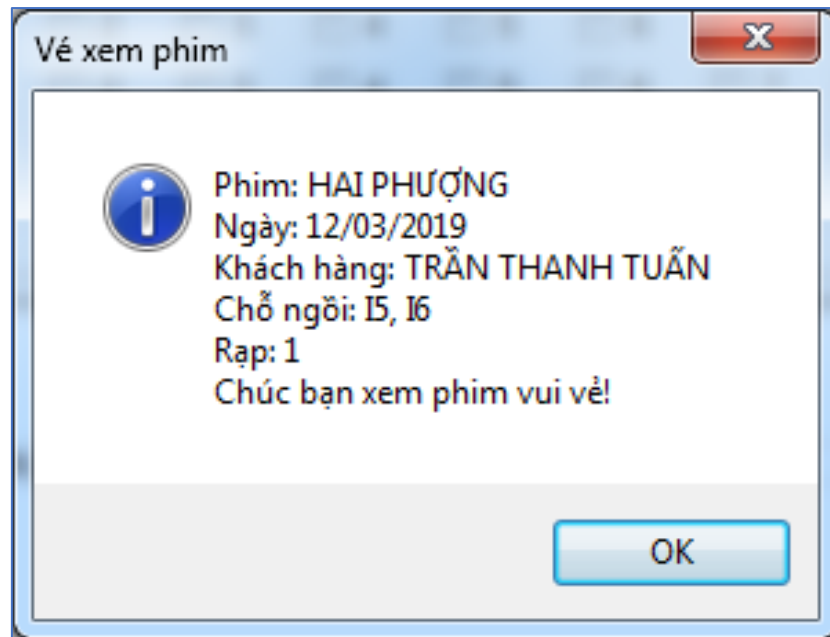
- Phim **“Hai Phượng”** và **Rạp 1** được chọn mặc định
- Nếu đã chọn đủ số lượng chỗ ngồi được chọn (*số checkbox được check bằng Số lượng vé (trong NumericUpDown)*) thì vô hiệu hoá các checkbox còn lại.
- Nhấn nút **“In vé”**
 - o Nếu chưa nhập dữ liệu *Họ tên khách hàng, Số lượng vé, Ngày đặt vé ...* thì hiển thị thông báo **“Vui lòng nhập dữ liệu...”** lên MessageBox;



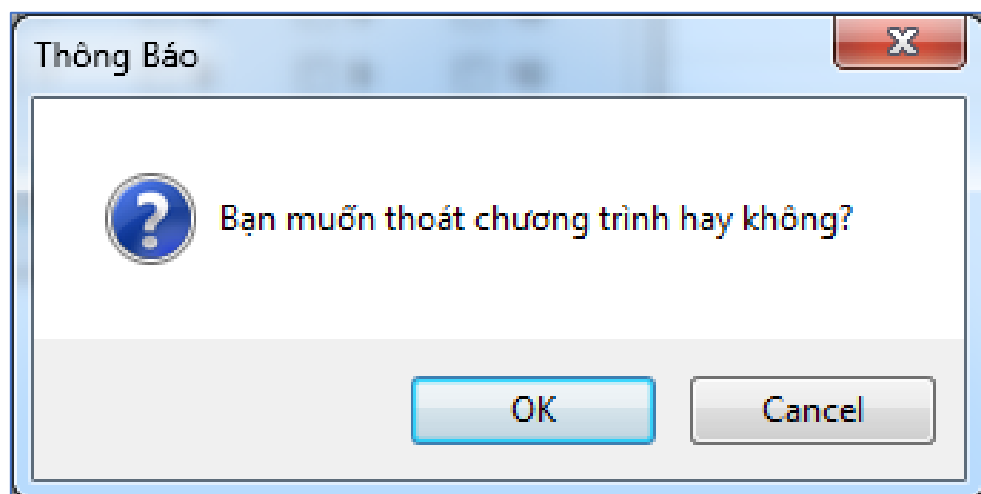
- o Nếu chưa chọn đủ số lượng vé thì hiển thị thông báo **“Vui lòng chọn đủ chỗ ngồi...”** lên MessageBox;

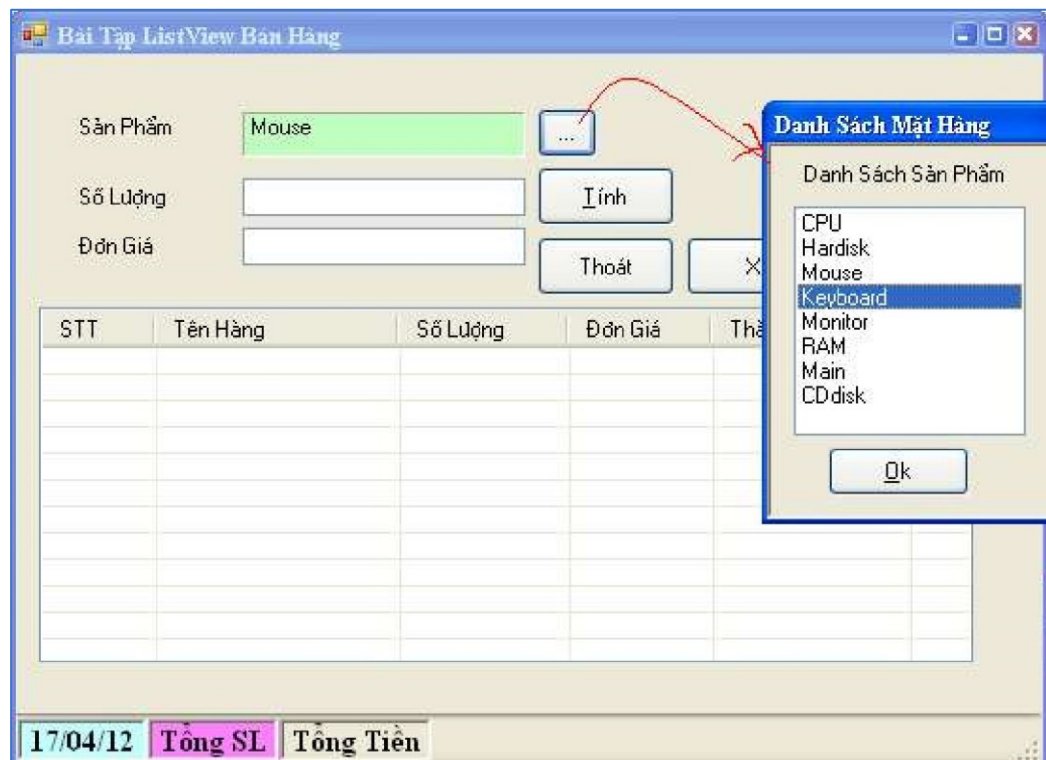


- o Nếu đã đủ thông tin thì hiển thị thông tin vé theo mẫu sau lên MessageBox.



- Nhấn nút **“Vé mới”**
 - Chọn lại Phim và Rạp mặc định
 - Xóa nội dung các TextBox
 - Bỏ check chỗ ngồi
- Nhấn nút **“Đóng”**: Thoát chương trình
- Khi thoát chương trình, hiển thị thông báo **“Bạn muốn thoát chương trình hay không?”**, nếu người dùng nhấn vào nút:
 - **Yes**: thoát
 - **No**: không thoát



Bài tập 3: Thiết kế giao diện như hình bên dưới và thực hiện các yêu cầu sau:**❖ Yêu cầu:**

- 1) Khi chọn sản phẩm qua button ... thì hiện form danh sách mặt hàng
 - Nếu double click trên list danh sách sản phẩm hoặc chọn button Ok thì lấy tên sản phẩm đang chọn đưa vào label sản phẩm.
- 2) Khi nhấn nút tính
 - Kiểm tra nhập đủ thông tin trên các textbox
 - Kiểm tra xem có nhập đúng số hay phần thập phân
 - Đưa dữ liệu vào listview qua các Item và đưa thêm thông tin vào StatusStrip lấy từ listview
- 3) Khi nhấn nút xoá xác nhận lại việc xoá Item trên listview

BÀI TẬP THỰC HÀNH TUẦN 08

ADO.NET



Mục tiêu

- Tìm hiểu về cách thức kết nối đến dữ liệu thông qua hệ quản trị cơ sở dữ liệu Microsoft SQL Server.
- Vận dụng các cách thức truy vấn dữ liệu theo mô hình kết nối.

Kiến thức

- ❑ **Đối tượng kết nối (connection):** dùng để tạo kết nối giữa ứng dụng với cơ sở dữ liệu

```
SqlConnection connect = new SqlConnection([chuỗi_kết_nối]);
```

```
Data Source=myServerAddress; Initial Catalog=myDataBase; Integrated Security=true; User ID=myUsername; Password=myPassword;
```

- ❑ **Đối tượng thực thi (Command):** dùng để thực thi câu lệnh truy vấn như SELECT, INSERT, UPDATE, DELETE hoặc một thủ tục (Store Procedure).

```
SqlCommand cmd = new SqlCommand(<câu_lệnh_SQL>, <đối_tượng_kết_nối>);
```

Phương thức thực thi đối tượng

- **ExecuteNonQuery():** thực thi các câu lệnh INSERT, UPDATE, DELETE; kết quả trả về số dòng thực thi được.
- **ExecuteScalar():** thực thi câu lệnh SELECT có sử dụng các hàm; kết quả trả về một giá trị duy nhất (giá trị của cột đầu tiên của dòng đầu tiên kết quả truy vấn) có kiểu object.
- **ExecuteReader():** thực thi câu lệnh SELECT; kết quả trả về đối tượng SqlDataReader chứa kết quả câu truy vấn SELECT.

Bài tập vận dụng

6. Cho CSDL QLSV với bảng sinh viên (SINH_VIEN) chứa các trường thông tin như sau:

Column Name	Data Type
MSSV	nvarchar(10)
HọTen	nvarchar(50)
NgàySinh	date
GioiTinh	bit
DiemTB	float

Hãy xây dựng màn hình quản lý sinh viên như màn hình giao diện trên với các yêu cầu:

- Thiết kế giao diện chương trình
 - Đặt tên các controls đúng chuẩn thiết kế;
 - Thêm icon cho form và buttons (**Icon; BackgroundImage**);
 - Chương trình hiển thị ở giữa màn hình; (**StartPosition: CenterScreen**);
 - Kích thước màn hình được cố định không cho phép co giãn; (**FormBorderStyle: FixedSingle**);
 - Hiển thị ngày sinh theo định dạng “dd/MM/yyyy”; (**CustomFormat; Format**);
 - Điểm trung bình cho phép tăng giảm 0.1 đơn vị; (**Increment; DecimalPlaces**);
 - Giới tính mặc định là Nam; (**Checked: True**);
 - Điểm trung bình mặc định là 5.0;
 - Ngày sinh mặc định là ngày cách ngày hiện tại 18 năm;
 - Thiết lập tab index từ trái sang phải, từ trên xuống cho các controls; (**TabIndex**);
 - Xây dựng tooltip chú thích cho các control buttons trong giao diện.

Thêm → Thêm sinh viên mới

Cập nhật → Cập nhật sinh viên theo MSSV

Xoá → Xoá sinh viên theo MSSV

Số lượng sinh viên → Đếm số lượng sinh viên trong lớp

b. Hiện thực hoá chức năng thêm sinh viên mới với các ràng buộc về chức năng sau:

- Các trường thông tin về MSSV, họ tên không được để trống
- Mã số sinh viên bắt buộc kiểu số, phải đủ 10 ký số và không được trùng lặp
- Sinh viên phải từ 18 tuổi trở lên;
- Điểm trung bình nằm trong đoạn $[0, 10]$

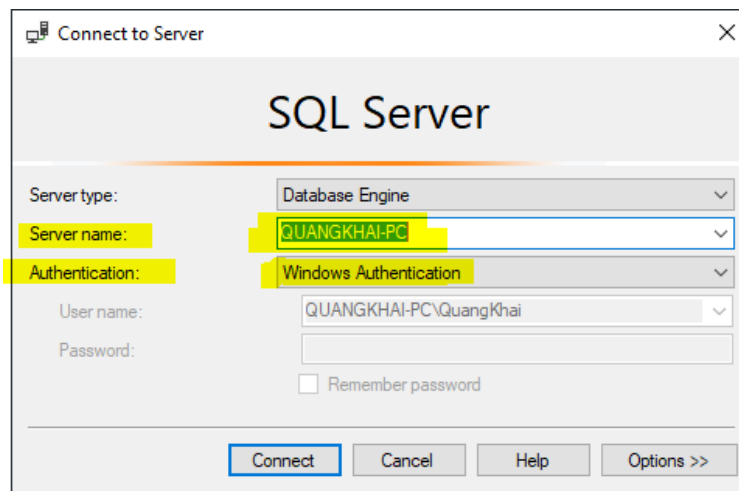
Nếu thoả các điều kiện trên, thêm thông tin sinh viên vào CSDL và hiện thông báo “Thêm sinh viên _1_ mã số _2_ thành công!”, với ____ lần lượt là Họ tên và MSSV của sinh viên đó. Sau khi thêm thành công, thực hiện việc đưa toàn bộ control nhập liệu về trạng thái ban đầu trước khi thêm.

Xử lý sự kiện cho control MSSV với yêu cầu sau: khi người dùng gõ MSSV trùng khớp (đã tồn tại) với một đối tượng trong bảng SINH_VIEN tiến hành điền tất cả thông tin của sinh viên đó vào các control tương ứng; trường hợp không trùng khớp, tiến hành đưa toàn bộ control nhập liệu về trạng thái ban đầu.

- c. Hiện thực hoá chức năng cập nhật sinh viên theo MSSV được cung cấp; yêu cầu kiểm tra các ràng buộc đã được liệt kê trong **câu b** trừ phần MSSV. Nếu thoả các ràng buộc nêu trên, tiến hành cập nhật dữ liệu và hiện thông báo “Cập nhật sinh viên _1_ mã số _2_ thành công!”, với _1_ và _2_ lần lượt là Họ tên và MSSV của sinh viên đó.
- d. Hiện thực hoá chức năng xoá sinh viên theo MSSV được cung cấp. Nếu xoá thành công hiện thông báo “Xoá sinh viên _1_ mã số _2_ thành công!”, với ____ lần lượt là Họ tên và MSSV của sinh viên đó. Tiếp theo, thực hiện việc đưa toàn bộ control nhập liệu về trạng thái ban đầu.
- e. Hiện thực hoá chức năng đếm số lượng sinh viên tồn tại trong CSDL. Hiện thị thông báo “Lớp có ____ sinh viên.”, với ____ là số lượng sinh viên.
- f. Thêm nút **“Extra 1”**. Cài đặt chức năng xuất ra số lượng sinh viên đạt (≥ 5) và không đạt (< 5) dựa vào điểm trung bình. **(mở rộng)**
- g. Thêm nút **“Extra 2”**. Cài chức năng xuất ra tỷ lệ sinh viên nam và tỷ lệ sinh viên nữ (định dạng %) của toàn bộ sinh viên. **(mở rộng)**

GỢI Ý:

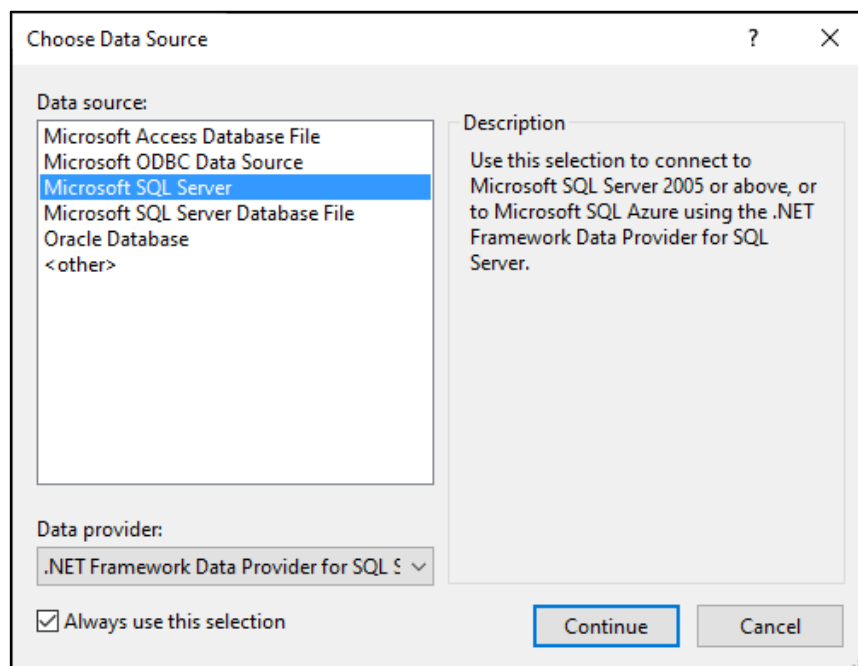
- Lấy chuỗi kết nối: xác định servername và cơ chế xác thực của MS SQL Server



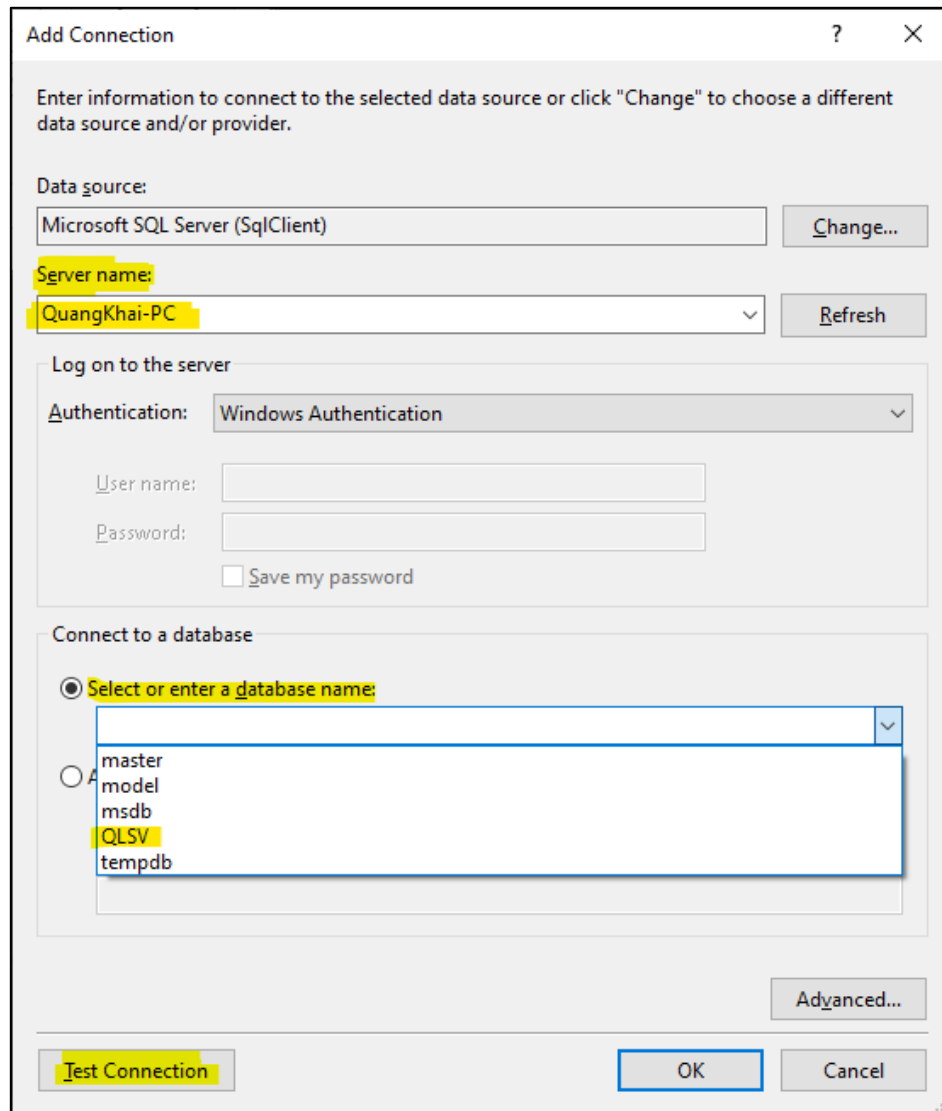
Trong Visual Studio → View → Server Explorer

Chọn Connect to Database  click phải vào Data Connections → Add Connection

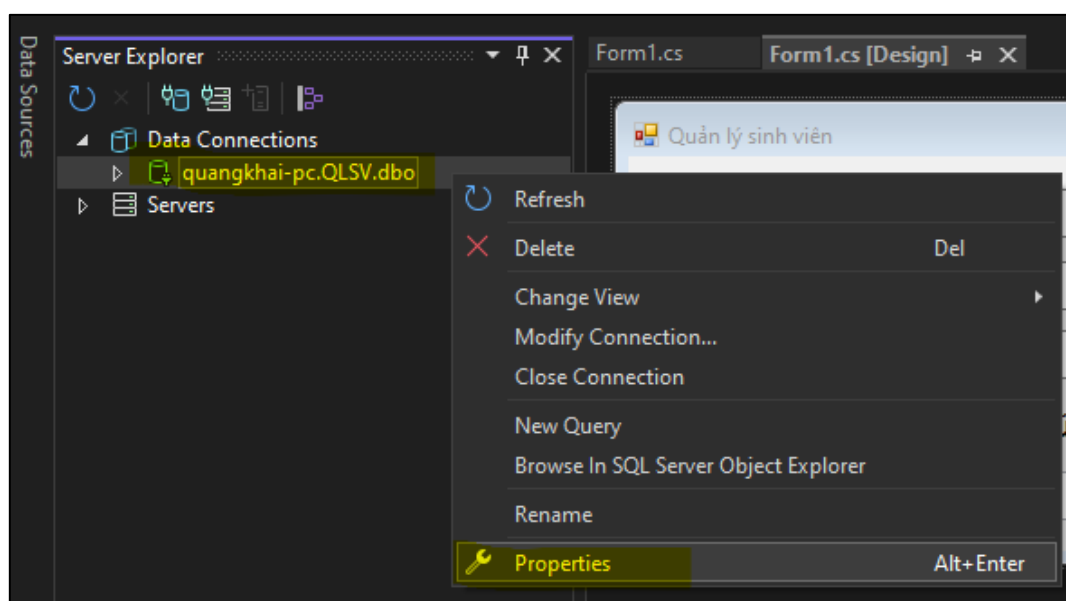
Chọn Microsoft SQL Server để tiến hành kết nối đến hệ quản trị MS SQL Server



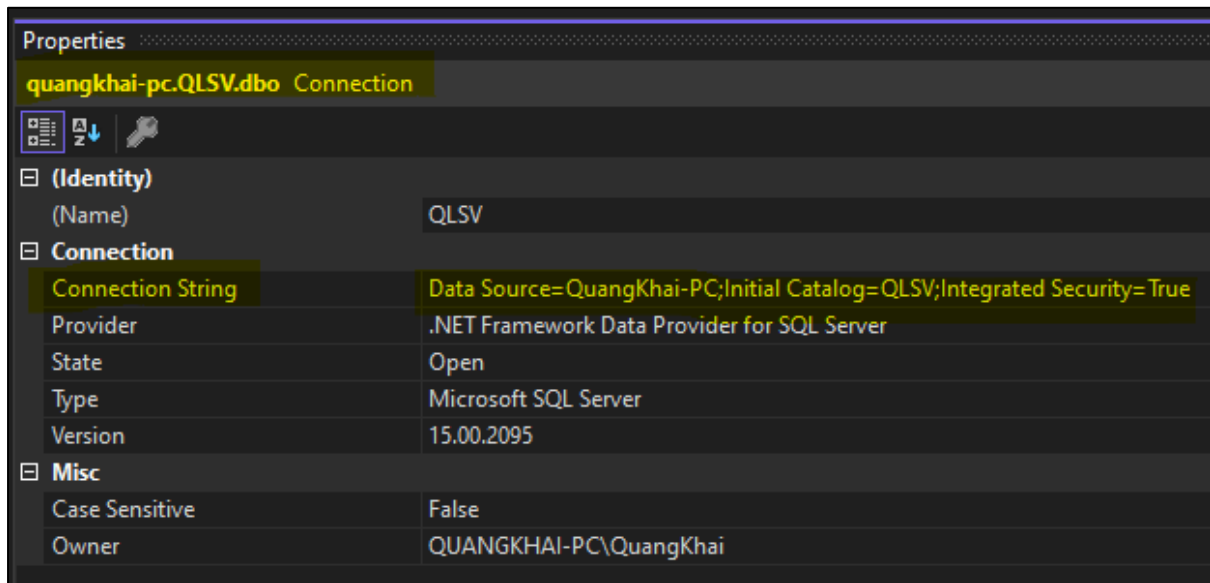
Nhập thông tin servername và chọn database cần kết nối.



Chọn OK, trong cửa sổ Server Explorer xuất hiện đối tượng kết nối CSDL vừa khởi tạo



Lúc này, chọn Properties để tiến hành lấy chuỗi kết nối như hình sau



- Các bước thao tác với đối tượng SqlCommand

```
// Bước 1: Tạo đối tượng kết nối
SqlConnection conn = new SqlConnection(strChuoiKetNoi);

// Bước 2: Mở kết nối
conn.Open();

// Bước 3: Thực hiện các thao tác dữ liệu (truy vấn, thêm, xóa, sửa)

// Bước 4: Đóng kết nối
conn.Close();
```

- Thao tác với parameters với SqlCommand. Xây dựng chức năng xoá sinh viên theo MSSV với parameter.

Xây dựng đối tượng thực thi kết hợp parameter trong câu truy vấn với cú pháp @tên_biến

```
SqlCommand cmd = new SqlCommand("DELETE FROM SINH_VIEN WHERE MSSV = @mssv", conn);
```

Định nghĩa parameter được sử dụng trong đối tượng thực thi

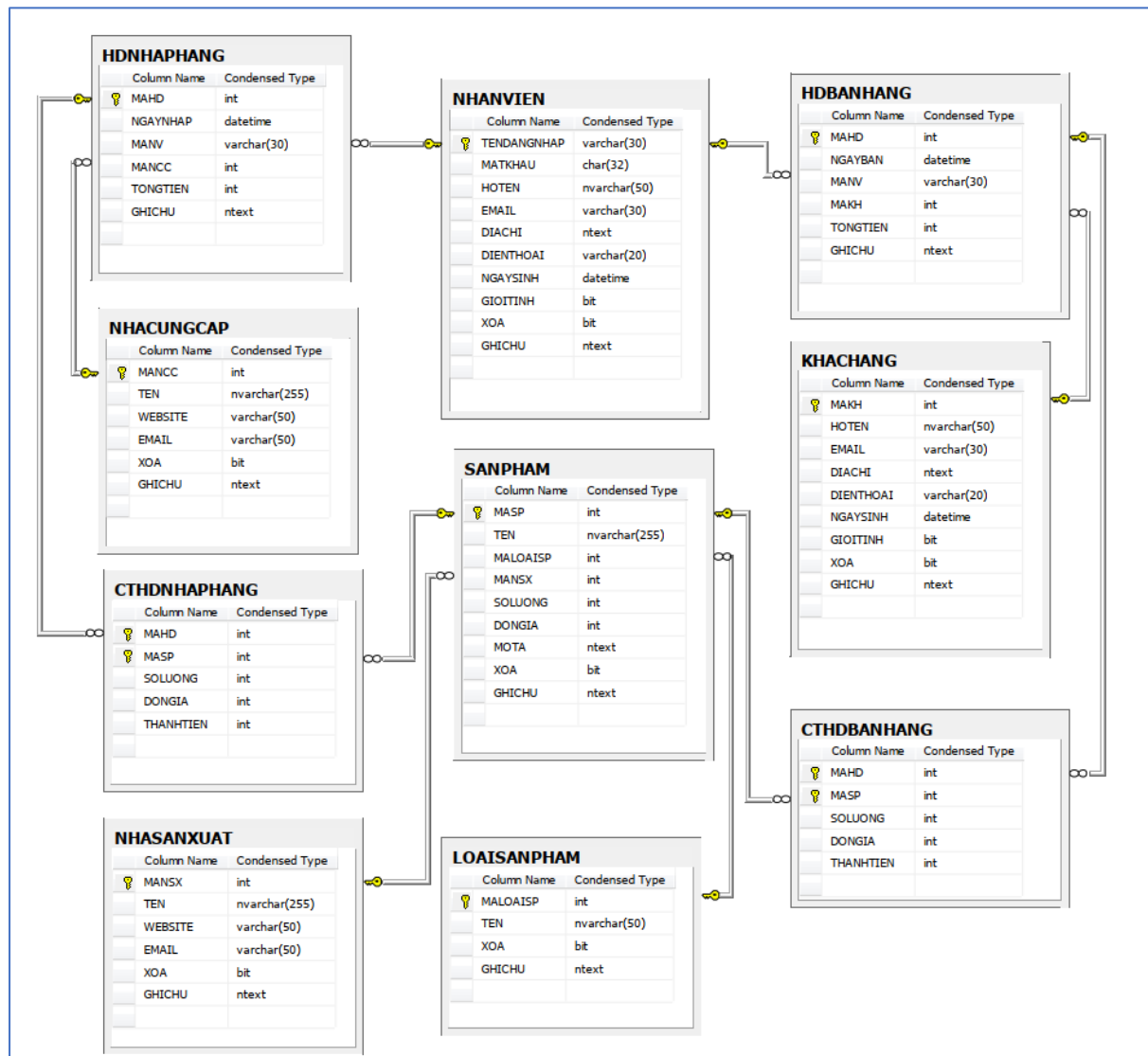
```
SqlParameter param = new SqlParameter();
param.ParameterName = "@mssv";
param.Value = <giá_trị>;
```

Thêm parameter vào đối tượng thực thi

```
cmd.Parameters.Add(param);
```

Bài tập mở rộng (dự án)

Cho lược đồ cơ sở dữ liệu như hình sau:



Sinh viên thiết kế màn hình Đăng nhập với các yêu cầu về giao diện như sau:

Đăng nhập frmDangNhap

Tên đăng nhập: tqkhai

Mật khẩu: *****

txtTenDangNhap

txtMatKhai

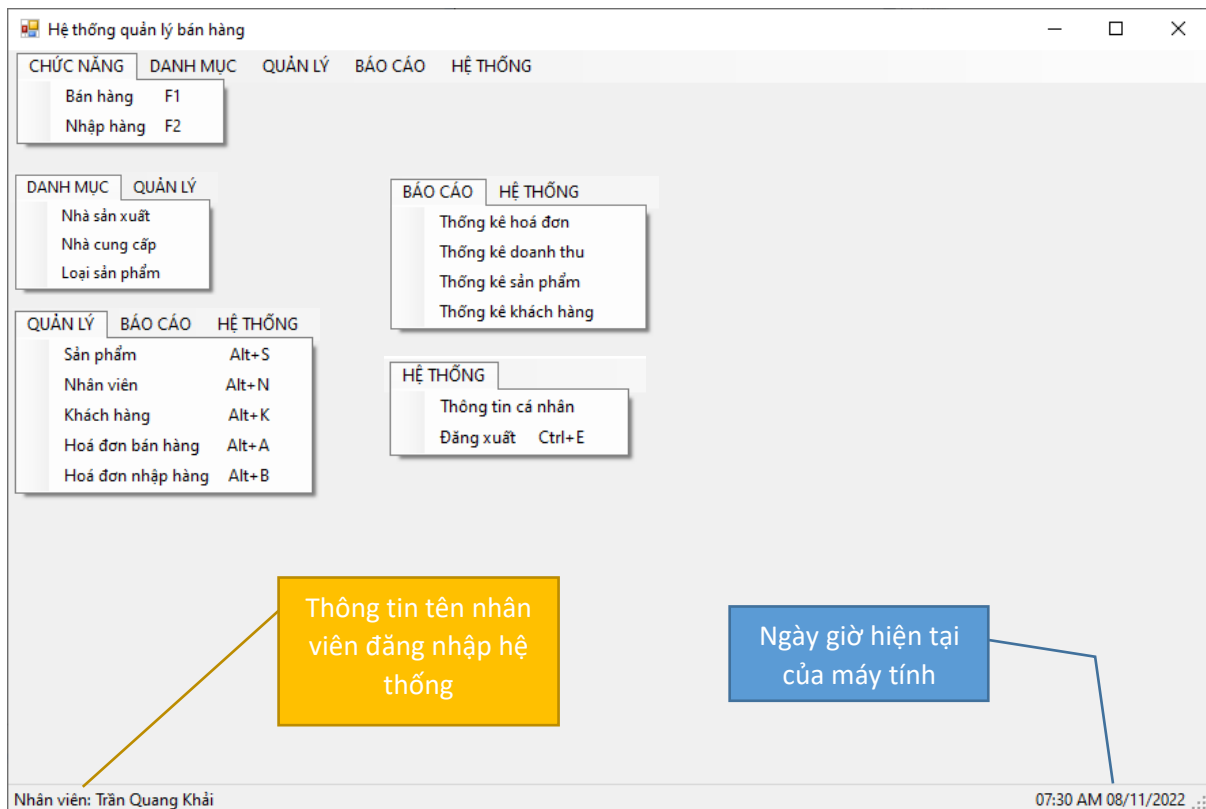
btnThoát Thoát Đăng nhập btnDangNhap

- Đặt tên các controls như hình trên.
- Màn hình xuất hiện ở vị trí giữa màn hình; không cho phép người dùng thay đổi kích thước màn hình.
- Textbox mật khẩu phải ẩn dữ liệu bằng dấu *

Thực hiện các yêu cầu về chức năng như sau:

- a. Khi người dùng nhấn nút “Thoát” → tắt màn hình và dừng chương trình
- b. Khi người dùng nhấn nút “Đăng nhập”
 - Chương trình kiểm tra các thông tin về tên đăng nhập và mật khẩu không được để trống, nếu vi phạm, hiển thị thông báo lỗi “Vui lòng điền đầy đủ thông tin”.
 - Thỏa điều kiện trên, tiến hành kiểm tra dữ liệu người dùng cung cấp và đối chiếu dữ liệu trong bảng NHANVIEN; nếu trùng khớp, thông báo “Đăng nhập thành công” → ẩn màn hình đăng nhập và hiển thị màn hình giao diện chính (hình bên dưới); nếu không trùng khớp, thông báo “Đăng nhập thất bại. Tên đăng nhập hoặc mật khẩu không chính xác!”

Màn hình giao diện chính



Sinh viên cài đặt các yêu cầu về giao diện và chức năng cho màn hình chính như sau:

- a. Thiết kế MenuStrip với cấu trúc như trên cùng các phím tắt cho các chức năng chính
- b. StatusStrip nằm dưới màn hình giao diện chứa các thông tin liên quan đến họ tên nhân viên đăng nhập và ngày giờ hiện tại của hệ thống.
- c. Thực hiện chức năng “**Đăng xuất**”; thoát khỏi màn hình giao diện chính, hiển thị màn hình “**Đăng nhập**”

BÀI TẬP THỰC HÀNH TUẦN 09

ADO.NET (Command)



Mục tiêu

- Tìm hiểu biểu diễn cấu trúc dữ liệu, dữ liệu bảng với DataTable, DataSet và đối tượng SqlDataAdapter.
- Vận dụng các cách thức truy vấn dữ liệu theo mô hình phi kết nối.
- Vận dụng sử dụng các đối tượng Insert/Update/Delete Command.

Kiến thức

❑ **DataSet**: cấu trúc phức tạp chứa nhiều DataTable cùng mối quan hệ giữa chúng (relationship) và các ràng buộc (constraint) để bảo vệ tính toàn vẹn dữ liệu.

```
DataSet <tên_biến> = new DataSet();
```

❑ **DataTable**: là đối tượng chứa dữ liệu từ dữ liệu nguồn nằm trong DataSet.

```
DataTable <tên_biến> = new DataTable();  
DataTable <tên_biến> = new DataTable("<tên_bảng>");
```

❑ **SqlDataAdapter**: là lớp tạo ra cầu nối giữa DataSet (chứa các bảng), để chứa dữ liệu từ các CSDL về DataSet, dữ liệu được các cấu trúc lệnh Insert/Update/Delete trong DataSet để cập nhật trở lại CSDL. Một đối tượng DataAdapter có các thuộc tính quan trọng để tạo ra các thao tác tương tác với Database như:

- SelectCommand thuộc tính chứa đối tượng SqlCommand, lấy dữ liệu bằng cách gọi phương thức Fill()
- InsertCommand thuộc tính chứa đối tượng SqlCommand, thực hiện thêm dữ liệu
- UpdateCommand thuộc tính chứa đối tượng SqlCommand, thực hiện cập nhật
- DeleteCommand thuộc tính chứa đối tượng SqlCommand, thực hiện xóa dòng dữ liệu

❑ **DataView**: là chế độ xem dữ liệu trên một cơ sở dữ liệu, hỗ trợ các chức năng lọc, sắp xếp. Ngoài ra còn có thể tùy chỉnh để hiển thị tập con dữ liệu cho DataTable.

❑ **SqlCommandBuilder**: tạo ra các câu lệnh Insert/Delete/Update dựa trên câu lệnh Select của SqlDataAdapter.

Bài tập vận dụng

1. Cho CSDL QLSV với bảng sinh viên (SinhVien, Lop) chứa các trường thông tin như sau:

SinhVien			
Column Name	Data Type	Allow Nulls	
ID	int	☐	
MSSV	nvarchar(10)	☐	
Họ	nvarchar(50)	☒	
Tên	nvarchar(50)	☒	
MaLop	varchar(10)	☐	
NgàySinh	date	☒	
ĐịaChỉ	nvarchar(MAX)	☒	
TrạngThái	smallint	☒	

Lop			
Column Name	Data Type	Allow Nulls	
MaLop	varchar(10)	☐	
TênLop	nvarchar(50)	☒	

The screenshot shows a Windows application window titled "Quản lý sinh viên" (Student Management). The window has a title bar with standard Windows controls. The main area contains a search section with the text "Tìm Kiếm" (Search). There are two radio buttons: "Tất Cả" (All) and "MSSV". A dropdown menu for "Lớp" (Class) is set to "CĐ TH 17A". Below the search section is a DataGrid with columns: ID, MSSV, Họ (Last Name), Tên (First Name), Ngày sinh (Date of Birth), Lớp học (Class), and Địa chỉ (Address). The DataGrid contains five rows of student data. At the bottom of the window are two buttons: "Lưu" (Save) and "Luu".

ID	MSSV	Họ	Tên	Ngày sinh	Lớp học	Địa chỉ
1	0306181122	Nguyễn Văn	A	12/05/99	CĐ TH 17A	Tân An - Long An
2	0306191253	Lê Thị	B	23/11/98	CĐ TH 17B	Bến Lức - Long An
3	0306171266	Phạm Quang	C	05/06/00	CĐN QTM 18C	Tân Trụ - Long An
4	0306171256	Trần Đình	D	10/10/91	TCTH 19	Đức Hoà - Long An
6	0306191152	Vũ Trung	E	04/08/98	CĐN QTM 18C	Kiến Tường - Long...

Hãy xây dựng màn hình quản lý sinh viên như màn hình giao diện trên với các yêu cầu:

- Thiết kế giao diện chương trình
 - Đặt tên các controls đúng chuẩn thiết kế;
 - Các cột trong DataGridView được dàn toàn bộ control
 - Thêm icon cho form và buttons (**Icon; BackgroundImage**)
 - Chương trình hiển thị ở giữa màn hình; (**StartPosition: CenterScreen**)
 - Kích thước màn hình được cố định không cho phép co giãn; (**FormBorderStyle: FixedSingle**)
 - Hiển thị ngày sinh theo định dạng "dd/MM/yyyy"; (**CustomFormat; Format**)
 - Thiết lập tab index từ trái sang phải, từ trên xuống cho các controls; (**TabIndex**)

- **DataGridView** chứa các cột dữ liệu trong bảng SinhVien gồm ID, MSSV, Ho, Ten, MaLop, NgaySinh, DiaChi trong đó cột Lớp học được thể hiện theo dạng control DataGridViewComboBoxColumn, các cột còn lại DataGridViewTextBoxColumn.
- Xây dựng tooltip chú thích cho các control buttons trong giao diện.
 Button tìm kiếm → Tìm kiếm sinh viên
 Button lưu → Lưu thông tin sinh viên
- b. Nạp dữ liệu danh sách lớp học vào các control Combobox (gồm trên khung tìm kiếm và cột Lớp học trong bảng DataGridView)
- c. Cài đặt các xử lý cho InsertCommand, UpdateCommand, DeleteCommand
- d. Hiển thị dữ liệu danh sách sinh viên lên trên DataGridView sao cho các trường dữ liệu vào đúng các cột tương ứng.
- e. Người dùng thao tác xóa, nhập, chỉnh sửa dữ liệu trực tiếp trên DataGridView, sau đó chọn nút “Lưu” để lưu trữ dữ liệu đã thay đổi trên DataGridView (dgvDSSV).
- f. Phát triển tính năng lọc dữ liệu cho phép tìm kiếm theo nhiều tiêu chí kết hợp (MSSV, Họ, Tên, Ngày sinh, Lớp học, Địa chỉ) theo dạng các CheckBox. Người dùng chọn vào CheckBox tương ứng nào thì sẽ lọc theo các tiêu chí đó. **(nâng cao)**

Gợi ý:

Khởi tạo các đối tượng sau:

```
SqlDataAdapter daLop; // đối tượng thực hiện thao tác dữ liệu cho lớp
SqlDataAdapter daSinhVien; // đối tượng thực hiện thao tác dữ liệu cho sinh viên
DataSet dsQLSV = new DataSet(); // chứa các bảng dữ liệu
DataGridView dvSinhVien; // chứa đối tượng hiển thị dữ liệu
```

Ở sự kiện `frmQLSV_Load()` thực hiện các công việc khởi tạo đối tượng

```
// Lấy dữ liệu lớp dựa vào thực hiện câu truy vấn thông qua SqlDataAdapter
daLop = new SqlDataAdapter("SELECT * FROM LOP", strKetNoi);
// Đổ dữ liệu sau khi thực hiện truy vấn
// vào bảng (được đặt tên) trong đối tượng DataSet đã khởi tạo
daLop.Fill(dsQLSV, "tblLop");
```

Thực hiện viện đổ dữ liệu vào các control hiển thị dữ liệu cho đối tượng lớp

```
//Đổ dữ liệu vào Combobox lớp học trong khung tìm kiếm
cbbLop.DataSource = dsQLSV.Tables["tblLop"].Copy(); // tạo bản sao bảng dữ liệu
cbbLop.DisplayMember = "TenLop"; // chỉ định thuộc tính hiển thị trong combobox
cbbLop.ValueMember = "MaLop"; // chỉ định thuộc tính giá trị cho đối tượng
```

Thực hiện tương tự với việc đổ dữ liệu vào DataGridViewComboBoxColumn trong DataGridView.

```
// Lấy dữ liệu sinh viên dựa vào câu truy vấn thông qua đối tượng SqlDataAdapter
daSinhVien = new SqlDataAdapter("Select * From SinhVien WHERE TrangThai=1", strKetNoi);
```

Khai báo đối tượng thêm dữ liệu

```
// Khai báo câu truy vấn thêm dữ liệu (insert) cho đối tượng SqlCommand
// có sử dụng parameters
SqlCommand insCom = new SqlCommand("Insert into SinhVien (MSSV, Ho, Ten, MaLop, DiaChi,
TrangThai, NgaySinh) Values (@MSSV, @Ho, @Ten, @Lop, @DiaChi, 1, @NgaySinh)");
```



```
// Ánh xạ các parameter với trường dữ liệu trong đối tượng bảng SinhVien
// theo cấu trúc <tên biến>, <kiểu dữ liệu>, <kích thước dữ liệu>, <tên trường>
insCom.Parameters.Add("@MSSV", SqlDbType.NVarChar, 10, "MSSV");
insCom.Parameters.Add("@Ho", SqlDbType.NVarChar, 50, "Ho");
insCom.Parameters.Add("@Ten", SqlDbType.NVarChar, 50, "Ten");
insCom.Parameters.Add("@Lop", SqlDbType.Int, 10, "MaLop");
insCom.Parameters.Add("@DiaChi", SqlDbType.NVarChar, 1000, "DiaChi");
insCom.Parameters.Add("@NgaySinh", SqlDbType.Date, 20, "NgaySinh");
```

```
// Thêm đối tượng thêm dữ liệu vào InsertCommand
daSinhVien.InsertCommand = insCom;
```

Tương tự, thực hiện cho các đối tượng UpdateCommand, DeletCommand trong đó, xoá sinh viên dựa vào việc cập nhật giá trị thuộc tính **TrangThai**

Thiết lập SqlCommandBuilder cho phép thực hiện thao tác dữ liệu dựa vào các đối tượng Command

```
// Cho phép thực hiện cập nhật dữ liệu
SqlCommandBuilder scb = new SqlCommandBuilder(daSinhVien);
```

Đổ thông tin danh sách sinh viên vào DataGridView

```
// Lấy bảng sinh viên đổ vào DataTable trong DataSet
daSinhVien.Fill(dsQLSV, "tblSinhVien");
dvSinhVien = dsQLSV.Tables["tblSinhVien"].DefaultView;
dgvSinhVien.DataSource = dvSinhVien;
```

Cài đặt lệnh cho phép thực hiện thao tác dữ liệu khi nhấp nút “Lưu”

```
// Thực hiện việc cập nhật các thay đổi dữ liệu trong DataSet
daSinhVien.Update(dsQLSV, "tblSinhVien");
// Xóa dữ liệu cũ trong bảng sinh viên
dsQLSV.Tables["tblSinhVien"].Rows.Clear();
// Lấy dữ liệu mới của những thay đổi cho DataSet → cập nhật DataGridView
daSinhVien.Fill(dsQLSV, "tblSinhVien");
dvSinhVien = dsQLSV.Tables["tblSinhVien"].DefaultView;
dgvSinhVien.DataSource = dvSinhVien;
```

Thực hiện tìm kiếm hoặc lọc dữ liệu bằng thuộc tính **RowFilter** trong **DataView**. Cài đặt cho tìm kiếm theo mã số sinh viên (MSSV)

```
if(radMSSV.Checked) dvSinhVien.RowFilter = "MSSV='" + txtMSSV.Text + "'";
```

Tương tự cho việc tìm kiếm theo:

```
Lớp: dvSinhVien.RowFilter = "Lop=" + <mã_lớp>;
```

```
Tất cả: dvSinhVien.RowFilter = "";
```

```
Họ tên: dvSinhVien.RowFilter = "Ho+' '+Ten like '%" + hoten + "%'";
```

Bài tập mở rộng (dự án)

Sinh viên tái sử dụng project trong bài tập mở rộng ở bài tập thực hành buổi trước và thực hiện các yêu cầu:

Xây dựng giao diện và chức năng quản lý cho các đối tượng thuộc “DANH MỤC”

BÀI TẬP THỰC HÀNH TUẦN 10

ADO.NET (Store Procedure)



Mục tiêu

- Tìm hiểu về cách thức kết nối đến dữ liệu thông qua hệ quản trị cơ sở dữ liệu Microsoft SQL Server.
- Sử dụng thủ tục (store procedure) trong việc thực hiện truy vấn dữ liệu.

Kiến thức

- ❑ Khai báo đối tượng thủ tục thông qua SqlCommand

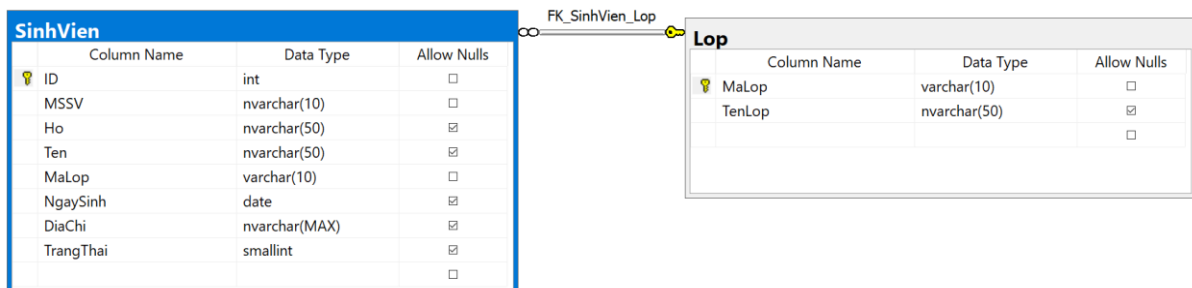
```
SqlCommand cmd = new SqlCommand();
cmd.Connection = <đối_tượng_kết_nối>;
cmd.CommandText = "<tên_thủ_tục>";
// chỉ định loại đối tượng thực thi là một thủ tục
cmd.CommandType = CommandType.StoredProcedure;
```

- ❑ Khai báo biến đầu vào cho thủ tục (nếu có)

```
// Khai báo các thuộc tính của tham số
SqlParameter param = new SqlParameter
{
    ParameterName = "@<tên_biến>",
    SqlDbType = <kiểu_dữ_liệu>,
    Value = <giá_trị>,
    Direction = ParameterDirection.Input
};
// Thêm tham số vào đối tượng SqlCommand
cmd.Parameters.Add(param);
```

Bài tập vận dụng

- Cho CSDL QLSV với bảng sinh viên và lớp chứa các trường thông tin như sau:



frmQLSV

QUẢN LÝ SINH VIÊN

MSSV	Ho	Tên	Ngày sinh	Lớp học	Địa chỉ
0306151046	Đoàn Ngọc	Long	30/10/1997	CĐ TH16MMT	TP. HCM
0306151116	Trần Như Gia	Bảo	24/02/1997	CĐ TH16MMT	TP. HCM
0306151158	Nguyễn Tấn	Liêm	06/06/1996	CĐ TH16MMT	TP. HCM
0306151167	Nguyễn Minh	Nhân	05/05/1997	CĐ TH16MMT	TP. HCM
0306151202	Nguyễn Quốc	Toán	07/04/1997	CĐ TH16MMT	TP. HCM
0306151332	Mai Thị Thúy	Án	04/10/1997	CĐ TH16MMT	TP. HCM
0306151347	Lê Vinh	Đình	09/09/1997	CĐ TH16MMT	TP. HCM
0306151367	Bùi Quốc	Khánh	09/04/1997	CĐ TH16MMT	TP. HCM
0306151416	Nguyễn Quốc	Thái	29/05/1997	CĐ TH16MMT	TP. HCM
0306151437	Ng		15/06/1997	CĐ TH16MMT	TP. HCM
0306151439	Ph		17/11/1997	CĐ TH16MMT	TP. HCM
0306161027	Nguyễn Bình	Điện	15/08/1998	CĐ TH16MMT	TP. HCM
0306161030	Lê Thần Trần	Đú	21/08/1998	CĐ TH16MMT	TP. HCM
0306161077	Lữ Anh	Phong	30/05/1998	CĐ TH16MMT	TP. HCM
0306161081	Nguyễn Xuân	Phúc	29/10/1998	CĐ TH16MMT	TP. HCM

Chi tiết sinh viên

MSSV: 0306151046

Họ: Đoàn Ngọc

Tên: Long

Ngày sinh: 30/10/1997

Lớp: CĐ TH16MMT

Địa chỉ: TP. HCM

+ Pencil Trash Refresh

btnThem, btnSua, btnXoa, btnMoi

2. Hãy xây dựng màn hình quản lý sinh viên như màn hình giao diện trên với các yêu cầu:

a. Thiết kế giao diện chương trình

- Đặt tên các controls đúng chuẩn thiết kế;
- Các cột trong DataGridView được dàn toàn bộ control
- Thêm icon cho form và buttons (**Icon; BackgroundImage**)
- Chương trình hiển thị ở giữa màn hình; (**StartPosition: CenterScreen**)
- Kích thước màn hình được cố định không cho phép co giãn;
- (**FormBorderStyle: FixedSingle**)
- Hiển thị ngày sinh theo định dạng “dd/MM/yyyy”; (**CustomFormat; Format**)
- Thiết lập tab index từ trái sang phải, từ trên xuống cho các controls; (**TabIndex**)
- **DataGridView** chứa các cột dữ liệu trong bảng SinhVien gồm MSSV, Ho, Ten, MaLop, NgàySinh, DiaChi trong đó cột Lớp học được thể hiện theo dạng control DataGridViewComboBoxColumn, các cột còn lại DataGridViewTextBoxColumn.
- Xây dựng tooltip chú thích cho các control buttons trong giao diện.

Thêm → Thêm sinh viên mới

Cập nhật → Cập nhật sinh viên theo MSSV được chọn

Xoá → Xoá sinh viên theo MSSV được chọn

Làm mới → Làm mới dữ liệu nhập

b. Hiện thực hoá chức năng làm mới dữ liệu nhập

- Tiến hành mở khoá tất cả các control nhập liệu nếu bị bất hoạt hoặc readonly.

c. Khi người dùng chọn một dòng dữ liệu trong DataGridView danh sách sinh viên:

- Hiển thị tất cả thông tin sinh viên được chọn vào các control tương ứng trong phần chi tiết sinh viên.
 - Thiết lập không cho người dùng thay đổi thông tin mã số sinh viên.
- d. Hiện thực hoá chức năng thêm sinh viên mới với các ràng buộc về chức năng sau:
- Các trường thông tin về MSSV, họ tên không được để trống
 - Mã số sinh viên bắt buộc kiểu số, phải đủ 10 ký số và không được trùng lặp
 - Sinh viên phải từ 18 tuổi trở lên;
 - Lớp bắt buộc phải chọn ít nhất một giá trị trong Combobox

Nếu thoả các điều kiện trên, thêm thông tin sinh viên vào CSDL và hiện thông báo “Thêm sinh viên _1_ mã số _2_ thành công!”, với ____ lần lượt là Họ tên và MSSV của sinh viên đó. Sau khi thêm thành công, thực hiện việc đưa toàn bộ control nhập liệu về trạng thái ban đầu trước khi thêm.

Xử lý sự kiện cho control MSSV với yêu cầu sau: khi người dùng gõ MSSV trùng khớp (đã tồn tại) với một đối tượng trong bảng SinhVien tiến hành điền tất cả thông tin của sinh viên đó vào các control tương ứng; trường hợp không trùng khớp, tiến hành đưa toàn bộ control nhập liệu về trạng thái ban đầu.

- e. Hiện thực hoá chức năng cập nhật sinh viên theo MSSV được cung cấp; yêu cầu kiểm tra các ràng buộc đã được liệt kê trong **câu d** trừ phần MSSV. Nếu thoả các ràng buộc nêu trên, tiến hành cập nhật dữ liệu và hiện thông báo “Cập nhật sinh viên _1_ mã số _2_ thành công!”, với _1_ và _2_ lần lượt là Họ tên và MSSV của sinh viên đó.
- f. Hiện thực hoá chức năng xoá sinh viên theo MSSV được cung cấp. Nếu xoá thành công hiện thông báo “Xoá sinh viên _1_ mã số _2_ thành công!”, với ____ lần lượt là Họ tên và MSSV của sinh viên đó. Tiếp theo, thực hiện việc đưa toàn bộ control nhập liệu về trạng thái ban đầu.
- g. Phát triển thêm chức năng tìm kiếm thông tin sinh viên theo các tiêu chí. **(mở rộng)**

Yêu cầu chung: các câu truy vấn thêm, xoá, cập nhật đòi hỏi phải được thực thi bằng thủ tục (store procedure)

Bài tập mở rộng (dự án)

Sinh viên tái sử dụng project trong bài tập mở rộng ở bài tập thực hành buổi trước và thực hiện các yêu cầu:

Xây dựng giao diện và chức năng quản lý **Sản phẩm – Nhân viên – Khách hàng** trong đó có sử dụng thủ tục trong việc **thêm và cập nhật** đối tượng (khuyến khích sử dụng Store)

BÀI TẬP THỰC HÀNH TUẦN 11

ADO.NET (Entity Framework)



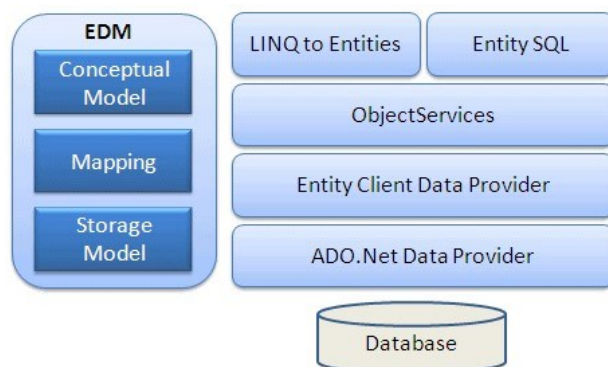
Mục tiêu

- Tìm hiểu về cách thức thao tác với dữ liệu thông qua Entity Framework.
- Vận dụng các thao tác xử lý với dữ liệu thêm – xoá – cập nhật dữ liệu.

Kiến thức

Entity Framework là một nền tảng để ánh xạ dữ liệu mô tả bằng lớp (đối tượng) vào cơ sở dữ liệu quan hệ, cho phép ánh xạ vào các bảng cơ sở dữ liệu, tạo cơ sở dữ liệu, truy vấn làm việc với LINQ, tạo và cập nhật vào database. Hay nói đúng hơn, Entity Framework là một khung ORM (Object Relational Mapper) mã nguồn mở cho các ứng dụng .NET được Microsoft hỗ trợ. là một phần của .NET Framework. là một Open source ORM Framework.

□ Cấu trúc của Entity Framework



- Conceptual Model: chứa các model class và các quan hệ, độc lập thiết kế bảng CSDL.
- Mapping: gồm có thông tin về cách Conceptual model kết nối với Storage model.
- Storage Model: là database design model bao gồm các bảng, views, stored procedures, và những quan hệ và các khóa.

□ Kiểm tra EF đã cài đặt sẵn trong Visual Studio:

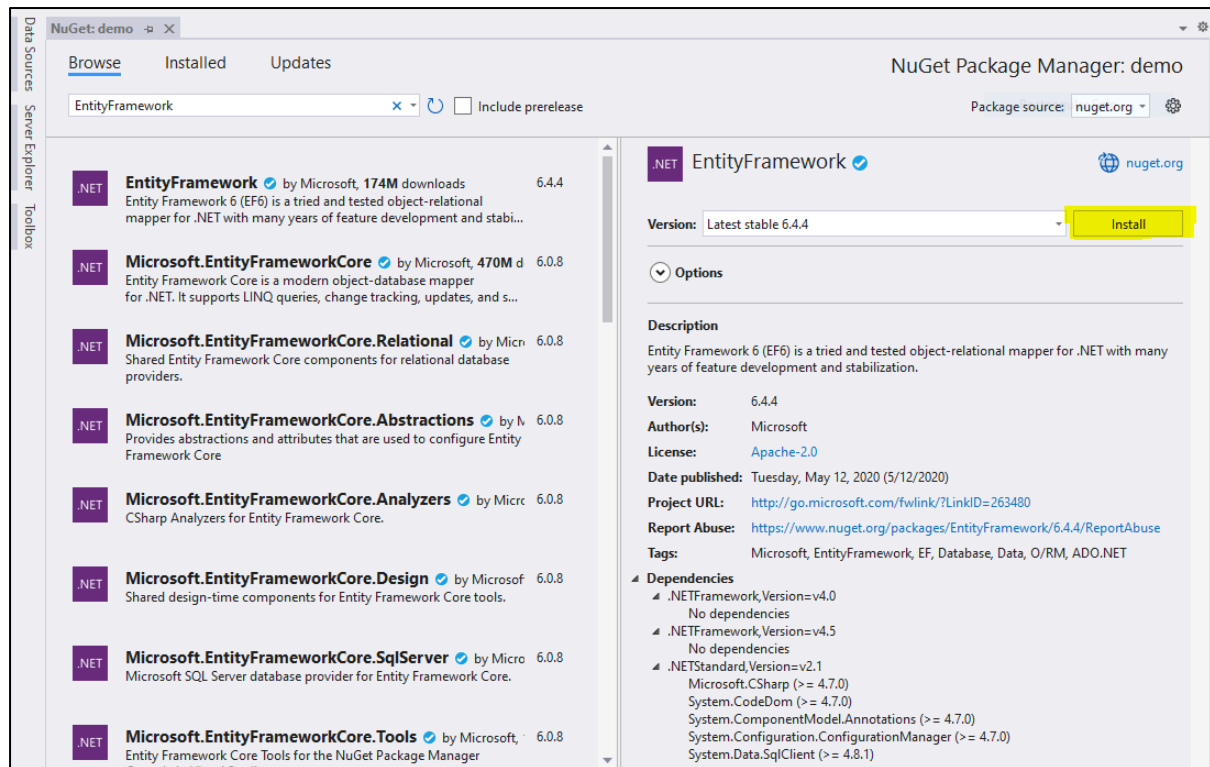
Click phải vào project → Add → New Item

Kiểm tra mục C# Items → Data → ADO.NET Entity Data Model (đã hỗ trợ)

□ Cài đặt EF nếu chưa hỗ trợ:

Click phải vào project → Manage NuGet Packages...

Chọn Browse → Gõ "EntityFramework" → Chọn phiên bản Microsoft và tiến hành cài đặt

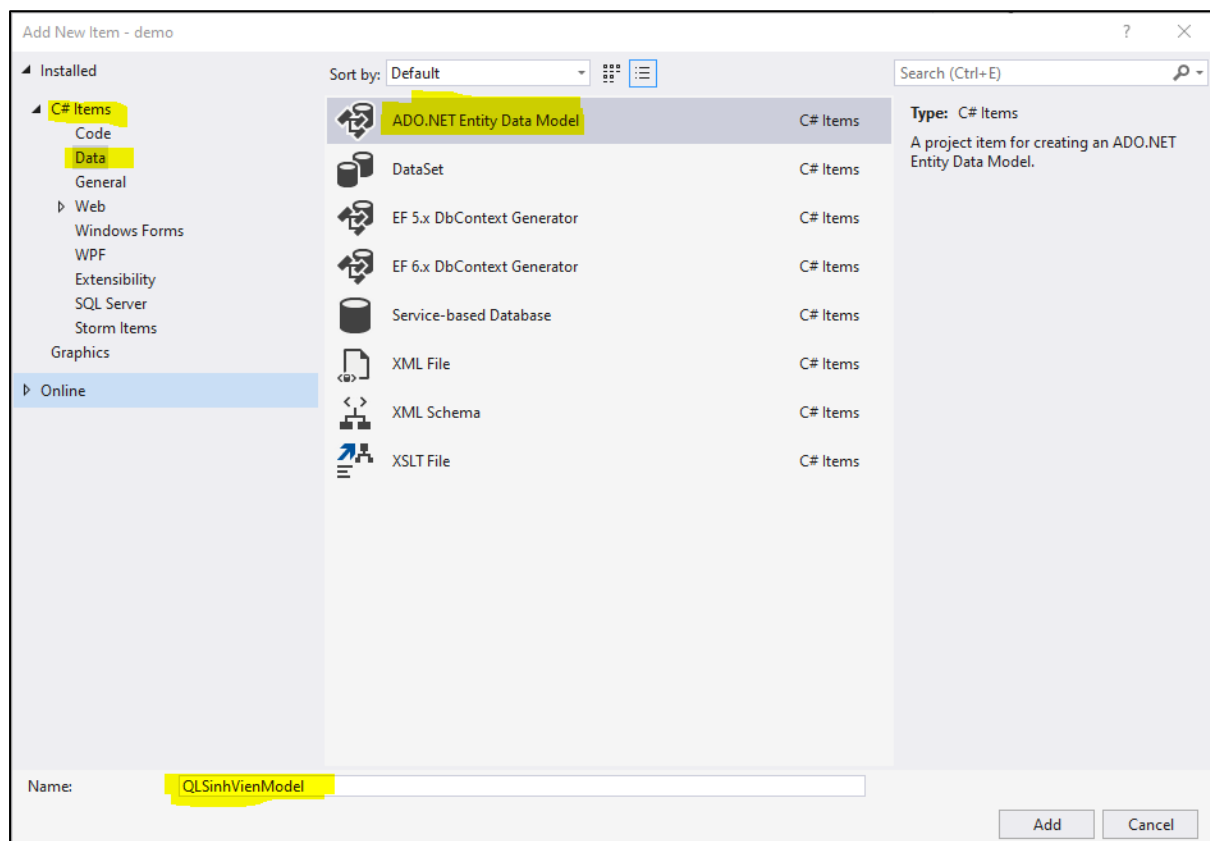


❑ Khởi tạo đối tượng EF theo Database First:

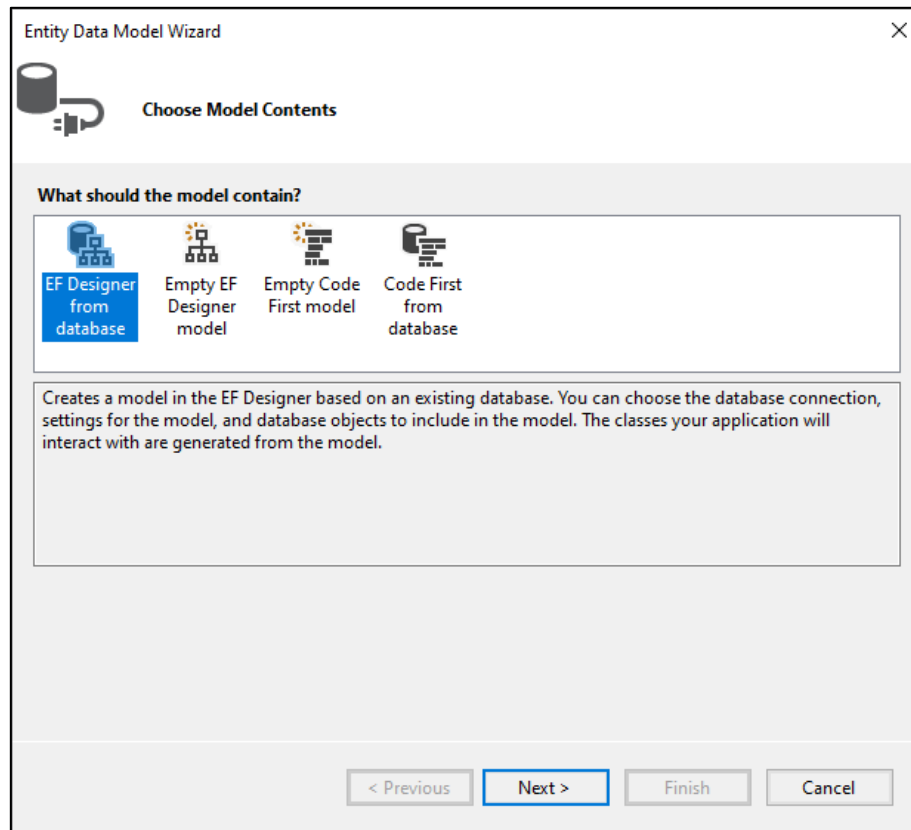
Click phải vào project → Add → New Item

Kiểm tra mục C# Items → Data → ADO.NET Entity Data Model

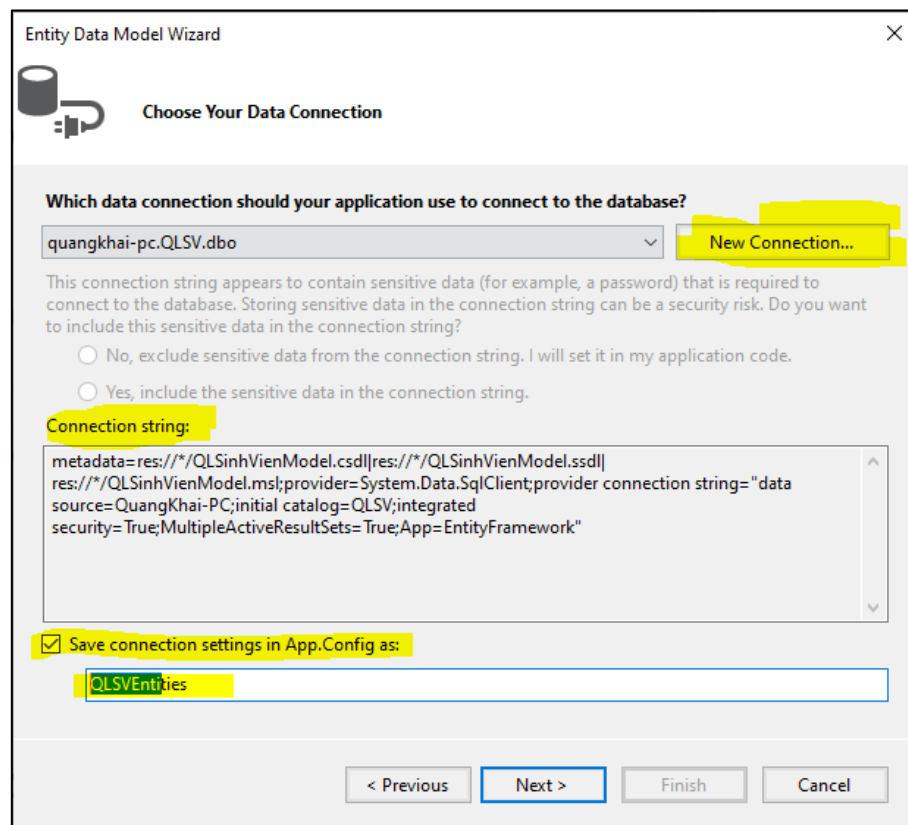
Đặt tên cho đối tượng EF được khởi tạo



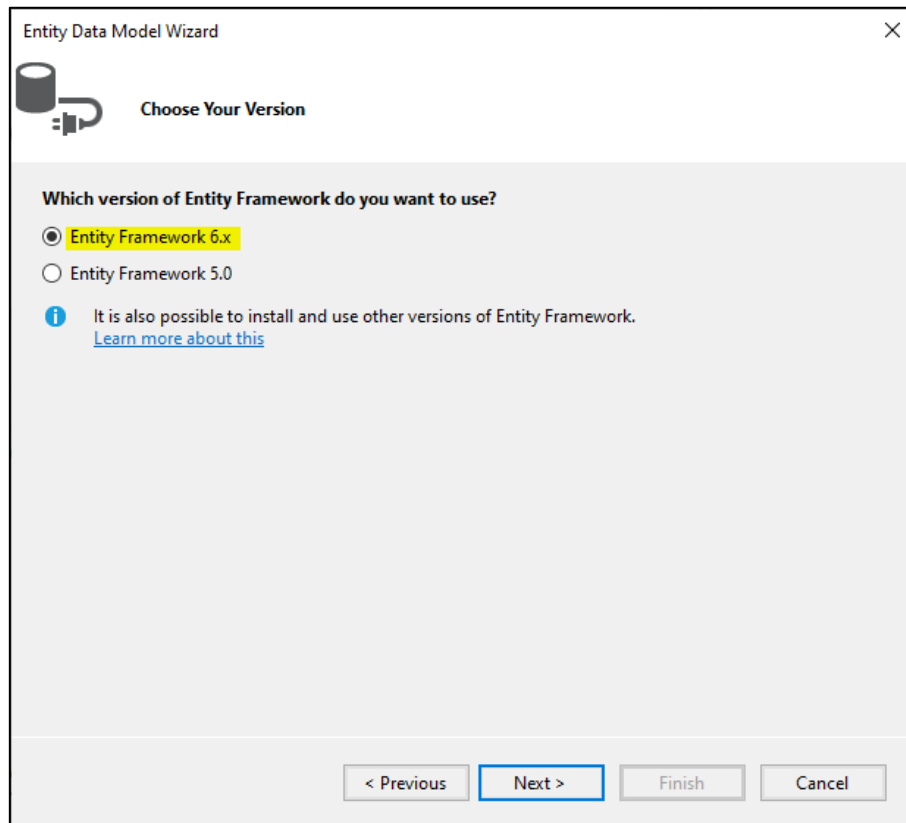
Chọn EF Designer from database (xây dựng EF từ cơ sở dữ liệu đã có sẵn)



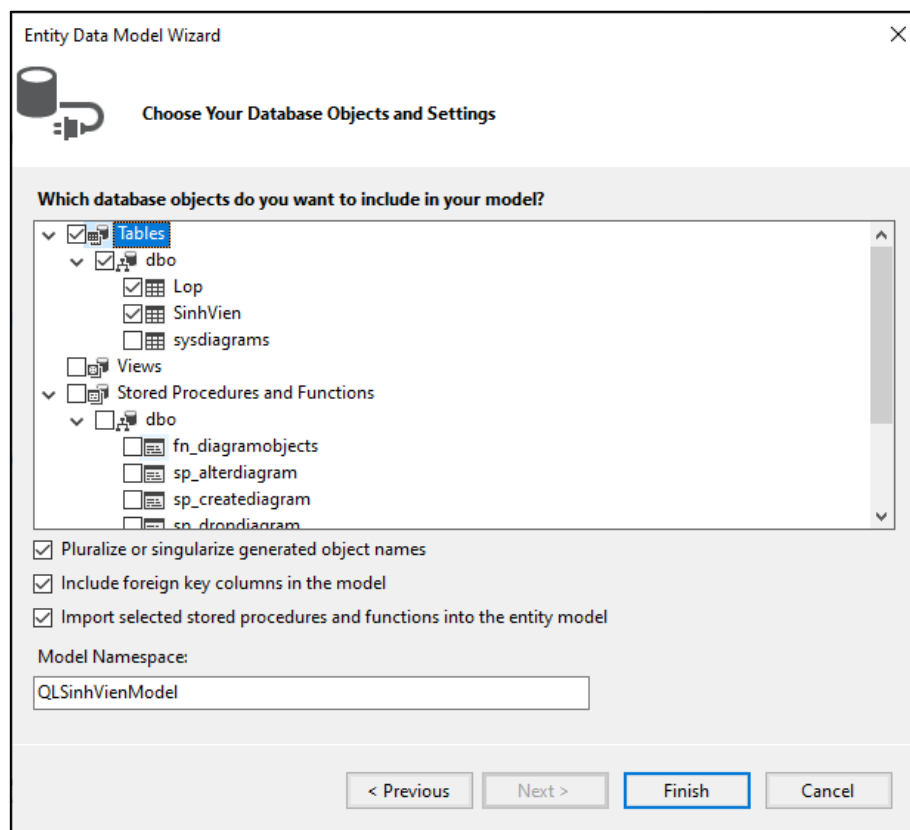
Lưu trữ các cấu hình liên quan đến chuỗi kết nối sử dụng Entity Framework vào trong App.Config thông qua tên đối tượng QLSVEntities.



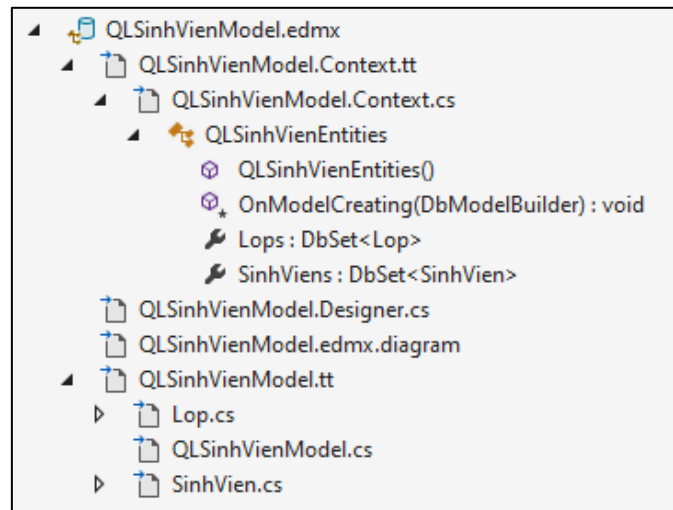
Lựa chọn phiên bản Entity Framework muốn sử dụng:



Lựa chọn các bảng dữ liệu, view, thủ tục (store procedure) muốn thao tác truy vấn.



Đối tượng được phát sinh như sau:



Bài tập vận dụng

1. Thực hiện bài tập tuần 10 bằng Entity Framework.

Gợi ý:

☐ Sử dụng Entity Framework trong việc truy xuất dữ liệu

```
// Khởi tạo đối tượng EF để truy vấn dữ liệu
QLSinhVienEntities qlsvEnt = new QLSinhVienEntities();
// Lấy danh sách sinh viên theo trạng thái
List<SinhVien> sinhViens = qlsvEnt.SinhViens.Where(u => u.TrangThai == 0).ToList();
```

☐ Sử dụng Entity Framework trong việc cập nhật dữ liệu

```
// Lấy sinh viên theo MSSV
SinhVien sv = qlsvEnt.SinhViens.SingleOrDefault(u => u.MSSV == txtMSSV.Text);
// Cập nhật thông tin sinh viên
sv.Ho = txtHo.Text;
sv.Ten = txtTen.Text;
// tương tự cho các trường thuộc tính còn lại
// Lưu những thay đổi cập nhật của đối tượng
qlsvEnt.SaveChanges();
```

☐ Sử dụng Entity Framework trong việc thêm dữ liệu

```
// Khởi tạo đối tượng chứa thông tin sinh viên
SinhVien sv = new SinhVien
{
    MSSV = txtMSSV.Text,
    Ho = txtHo.Text,
    // ... lấy thông tin sinh viên cho các thuộc tính còn lại
};
// Thêm đối tượng sinh viên mới
qlsvEnt.SinhViens.Add(sv);
// Lưu những thay đổi cập nhật của đối tượng
qlsvEnt.SaveChanges();
```

☐ Sử dụng Entity Framework trong việc xóa dữ liệu

```
// Lấy sinh viên theo MSSV
SinhVien sv = qlsvEnt.SinhViens.SingleOrDefault(u => u.MSSV == txtMSSV.Text);
// Xóa thông tin sinh viên
qlsvEnt.SinhViens.Remove(sv);
// Lưu những thay đổi cập nhật của đối tượng
qlsvEnt.SaveChanges();
```

Bài tập mở rộng (dự án)

Sinh viên tái sử dụng project trong bài tập mở rộng ở bài tập thực hành buổi trước và thực hiện các yêu cầu:

1. Chuyển đổi việc thao tác dữ liệu cho các đối tượng và chức năng **đang hiện hữu** bằng Entity Framework.
2. Khảo sát chức năng bán hàng trong thực tế (siêu thị, smart-store, nhà hàng, quán ăn, thương mại điện tử,...). Qua đó thiết kế giao diện và cài đặt xử lý cho chức năng bán hàng và nhập hàng.

BÀI TẬP THỰC HÀNH TUẦN 12

MÔ HÌNH 3 LỚP (3 – LAYERS)



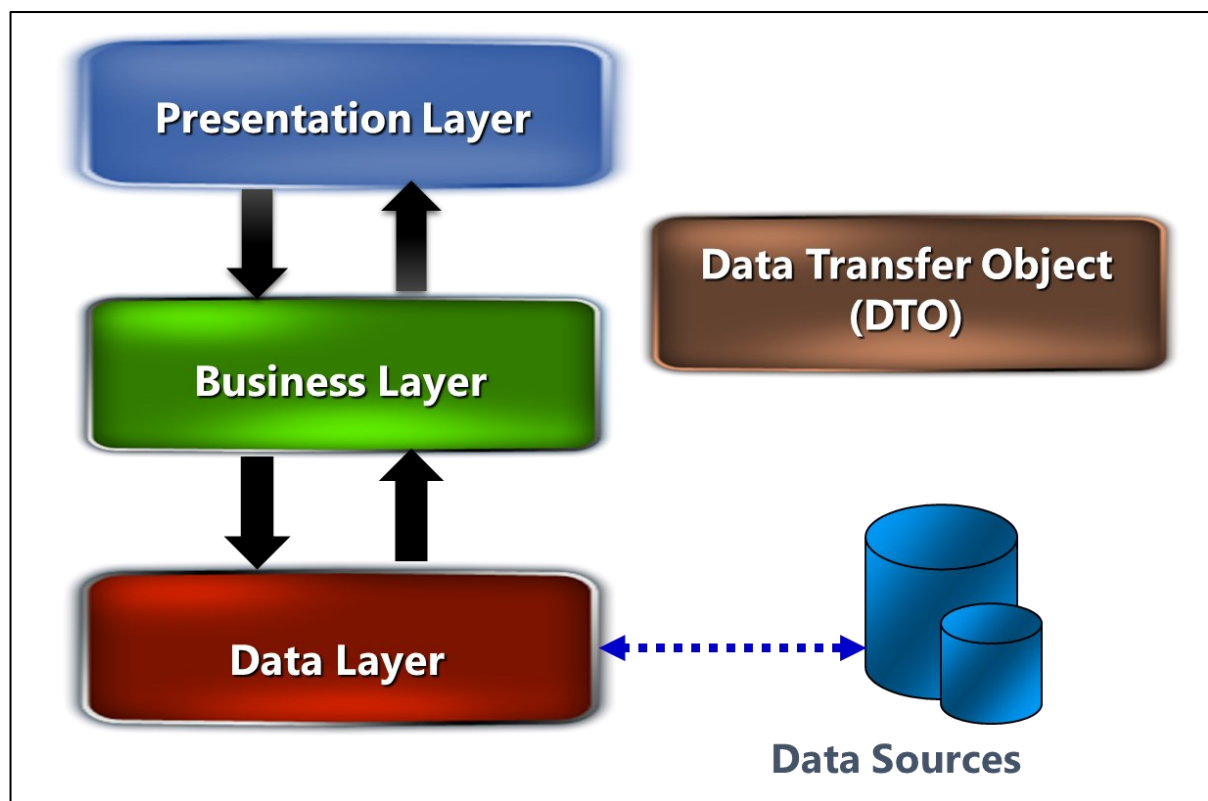
Mục tiêu

- Nắm kiến thức về việc xây dựng mô hình 3 lớp trong việc phát triển chương trình.
- Thiết kế được chương trình theo mô hình 3 lớp.

Kiến thức

❑ Mô hình 03 lớp:

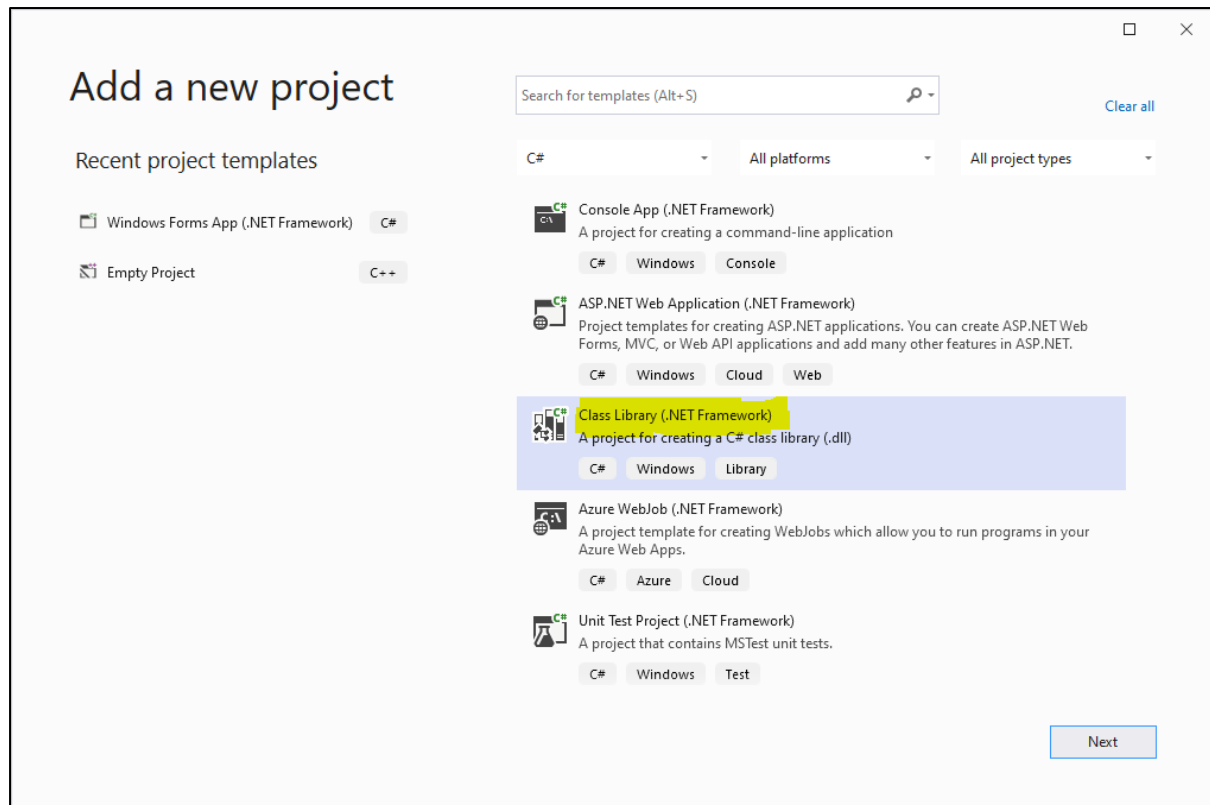
- Chia thiết kế ứng dụng thành các nhóm, thành phần gọi là lớp/tầng (Layer)
- Mỗi tầng đóng vai trò, chức năng khác nhau trong ứng dụng
- Mỗi tầng có thể được phân thành các nhóm nhỏ
- Xác định sự tương tác, quan hệ, vai trò giữa các tầng với nhau



❑ Khởi tạo cấu trúc chương trình:

Khởi tạo project cho lớp Business (BUS)

Click phải vào Solution → Add → New Project → Class Library (cho .NET Framework)



Đặt tên Project → Chọn nơi lưu trữ → Chọn phiên bản Framework phù hợp

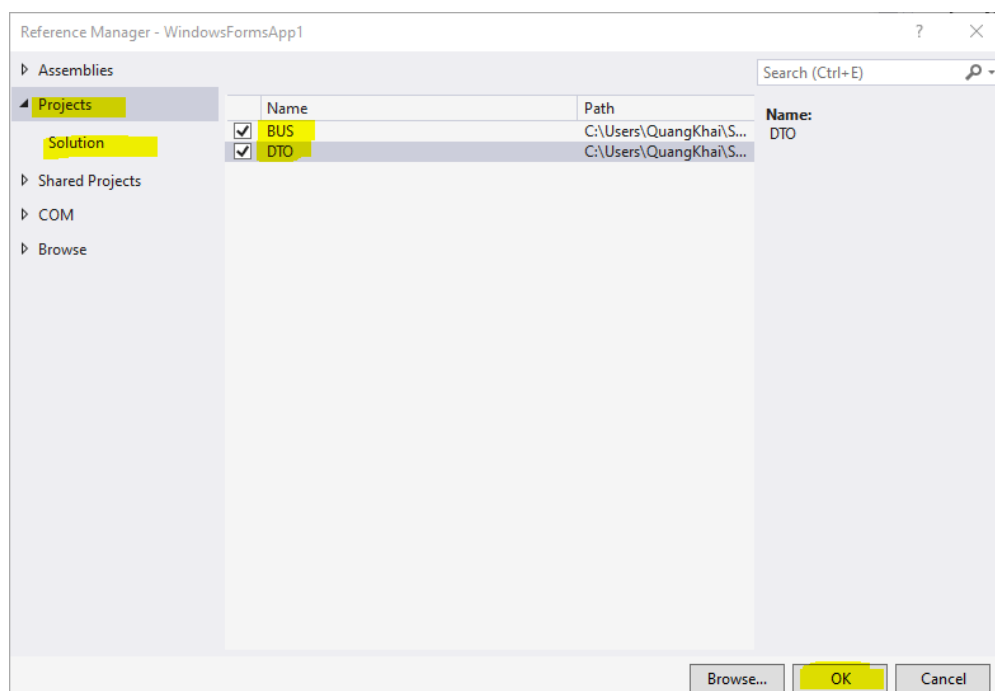
Tương tự cho các lớp giao diện Graphic User Interface (GUI), Data Access Layer (DAO) và đối tượng dữ liệu Data Transfer Object (DTO).

❑ Thêm tính liên kết giữa các lớp trong cấu trúc chương trình

GUI truy xuất các đối tượng của lớp BUS và DTO ta tiến hành thêm các *.dll tương ứng

Click phải vào mục References trong lớp GUI → Add Reference...

Chọn mục Projects → Solution → Đánh dấu vào các library BUS và DTO → Chọn OK



Tương tự cho các lớp còn lại

BUS truy xuất các đối tượng của lớp DAO và DTO

DAO truy xuất các đối tượng của lớp DTO

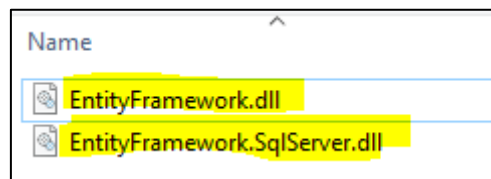
❑ Lưu ý khi sử dụng mô hình 03 lớp với Entity Framework

- Đối tượng EF được khởi tạo ở lớp tương tác với CSDL là DAO
- Cần thêm thư viện *.dll của EF cho lớp giao diện GUI

Click phải vào References (GUI) → Add Reference... → Chọn Browse

Trở vào thư mục packages của chương trình → Chọn thư mục EF phiên bản đã tạo

Chọn lib → net45 (hoặc mới nhất) → Chọn 2 thư viện



- Cần thêm đối tượng EF trong App.Config của lớp DAO cho App.Config của GUI

Vào App.Config (DAO) sao chép đoạn khai báo <connectionStrings></connectionStrings> vào App.Config (GUI)

Bài tập vận dụng

1. Thực hiện bài tập tuần 10 theo mô hình 03 lớp.

Bài tập mở rộng (dự án)

Sinh viên tái sử dụng project trong bài tập mở rộng ở bài tập thực hành buổi trước và thực hiện các yêu cầu:

Chuyển đổi project theo mô hình 03 lớp; sau đó thực hiện xây dựng giao diện và chức năng cho màn hình quản lý “Thông tin cá nhân”, “Quản lý hoá đơn bán hàng”, “Quản lý hoá đơn nhập hàng”.

BÀI TẬP THỰC HÀNH TUẦN 13

REPORT (1)



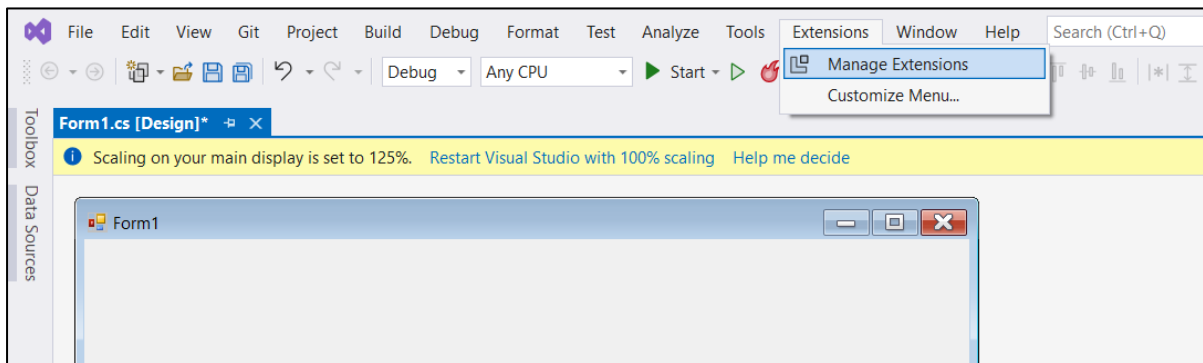
Mục tiêu

- Nắm kiến thức về yêu cầu cần thiết khi xây dựng báo cáo (report) đơn giản và sử dụng tham số (parameter).
- Sử dụng được báo cáo trong việc thống kê các số liệu cho phần mềm quản lý.

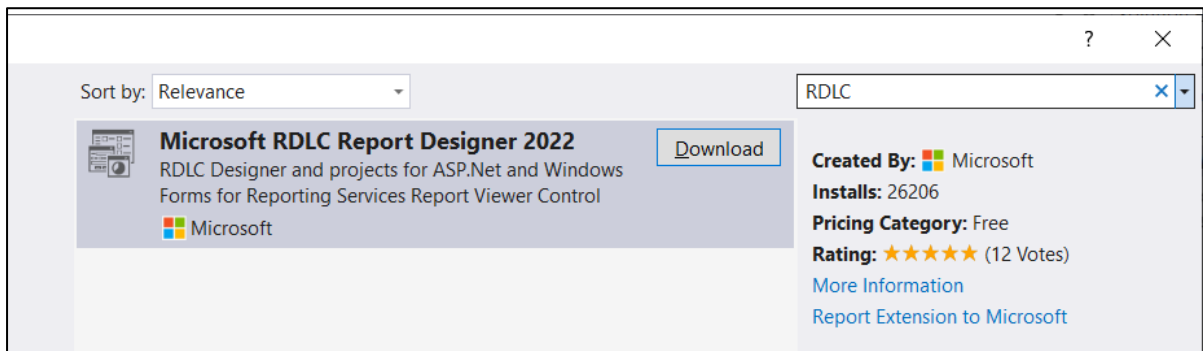
Kiến thức

❑ Cài đặt thư viện cho Local Report

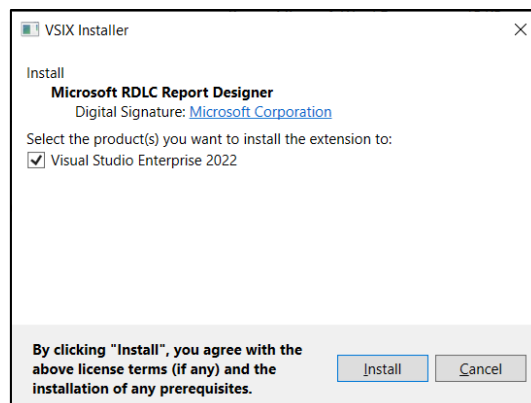
Vào Extensions → Manage Extensions



Nhập vào ô tìm kiếm từ khoá “RDLC” → Chọn Download



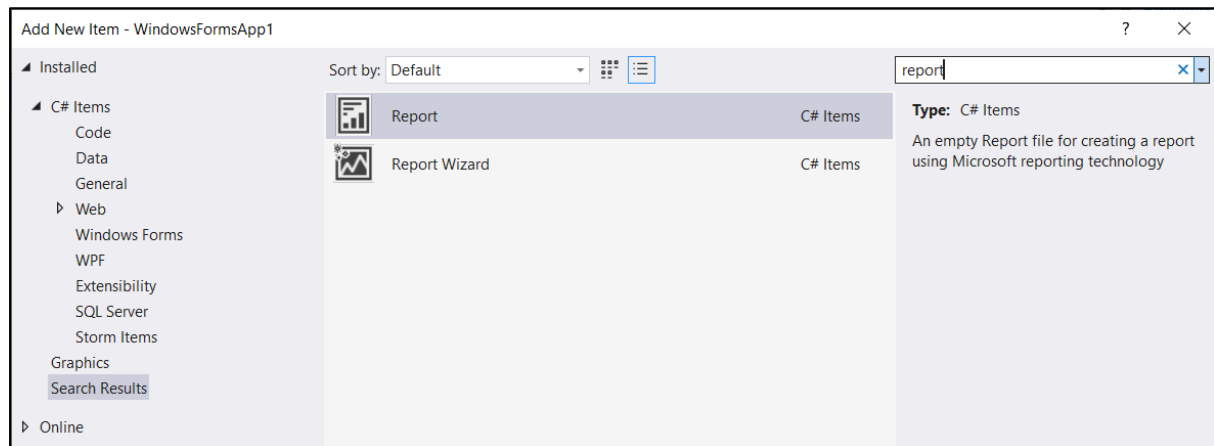
Sau khi trình Download hoàn tất → Tiến hành cài đặt (yêu cầu tắt Visual Studio)



❑ Khởi tạo report

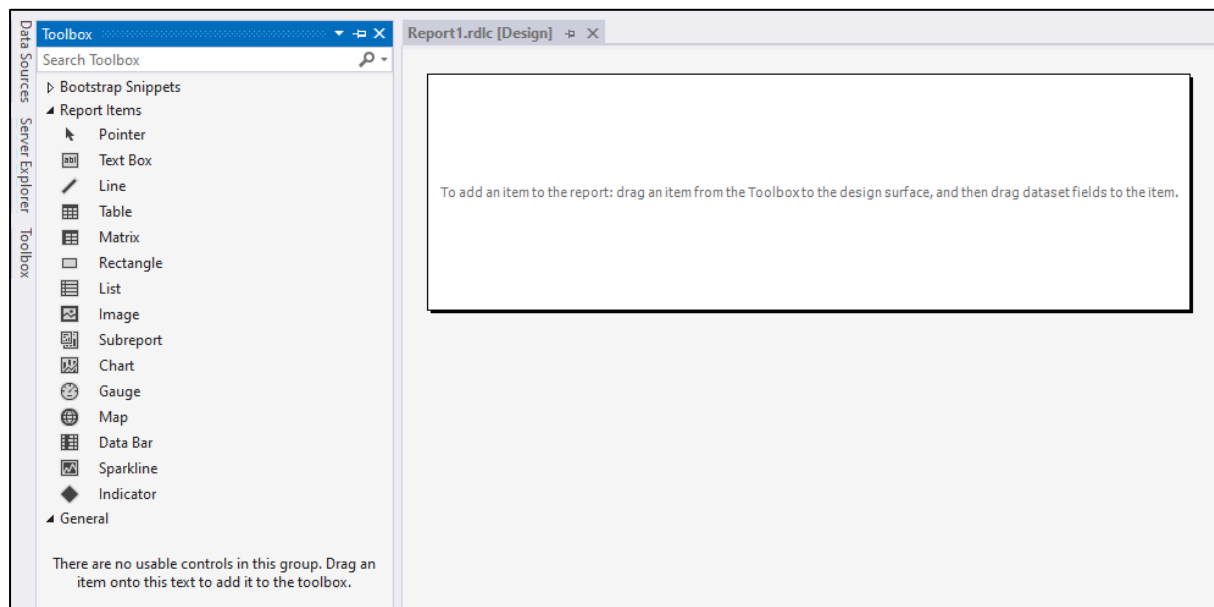
Click phải vào project → Add → New Item

Chọn mục Reporting hoặc tìm kiếm với từ khoá Report



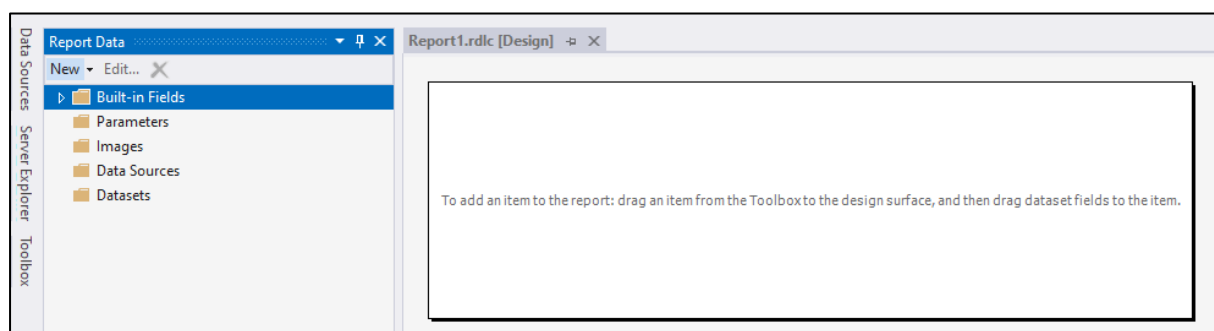
Chọn Report → Tiến hành đặt tên cho báo cáo

❑ Thiết kế layout cho report

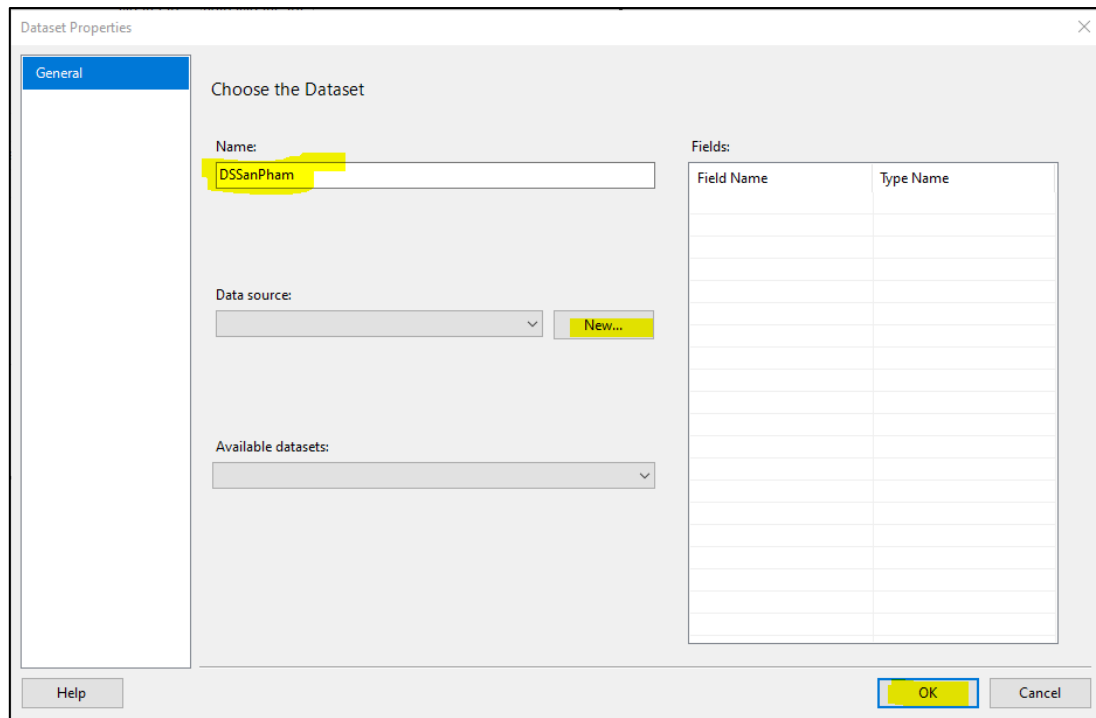


Kéo các control trong cửa sổ Toolbox vào report để thiết kế báo cáo.

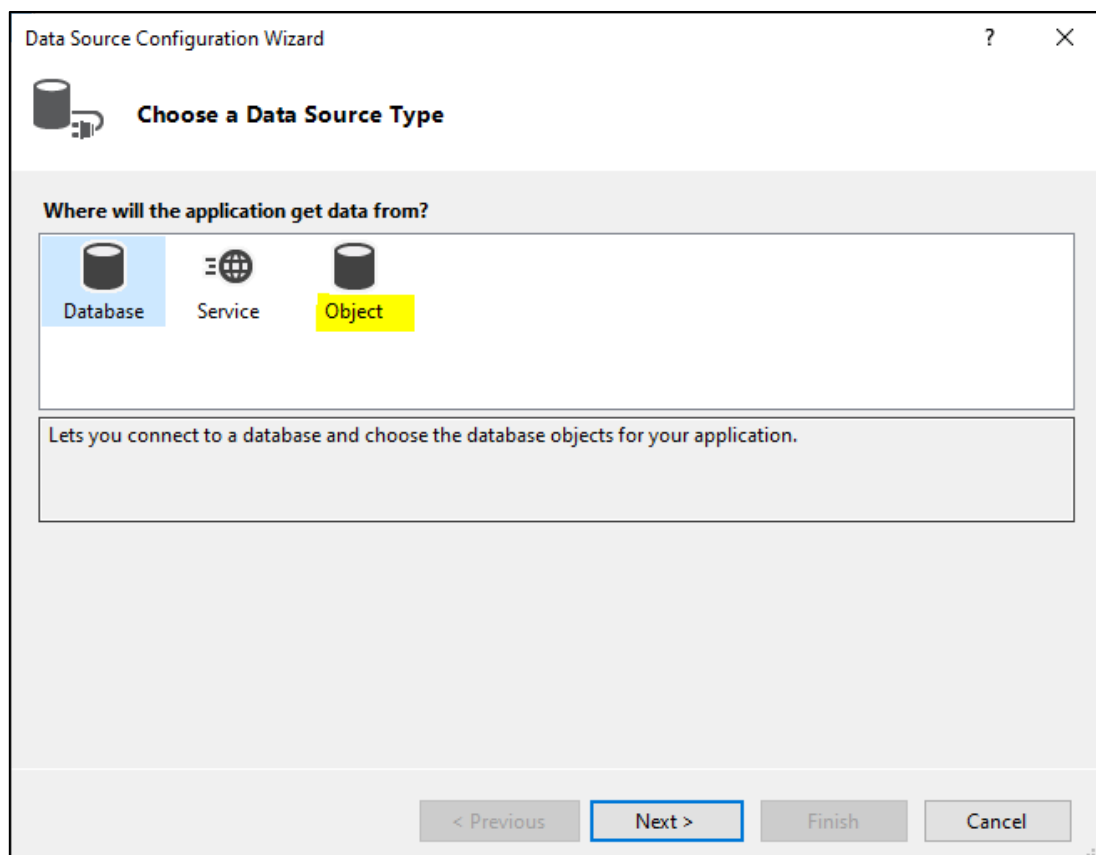
❑ Thiết kế dữ liệu cho report (Report Data)



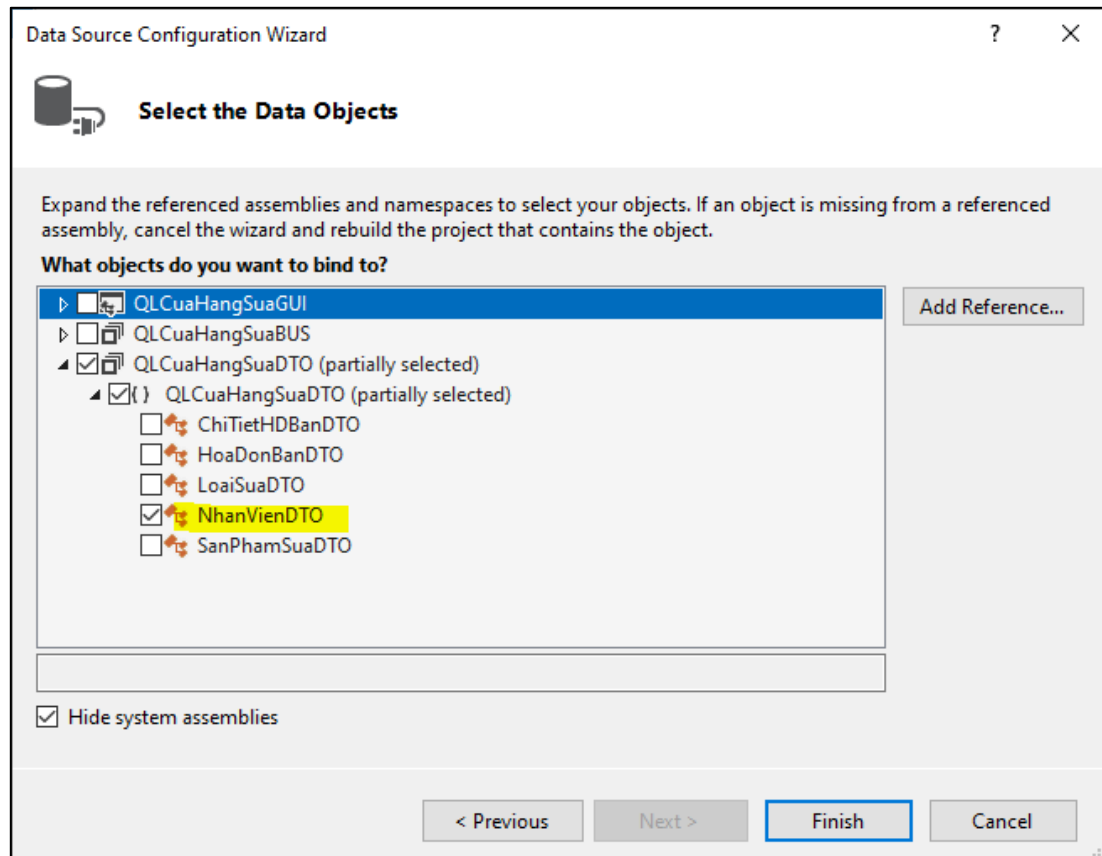
Click phải vào DataSet → Add DataSet → Đặt tên cho DataSet → Chọn Data source nhằm chỉ định đối tượng dữ liệu nguồn cho DataSet



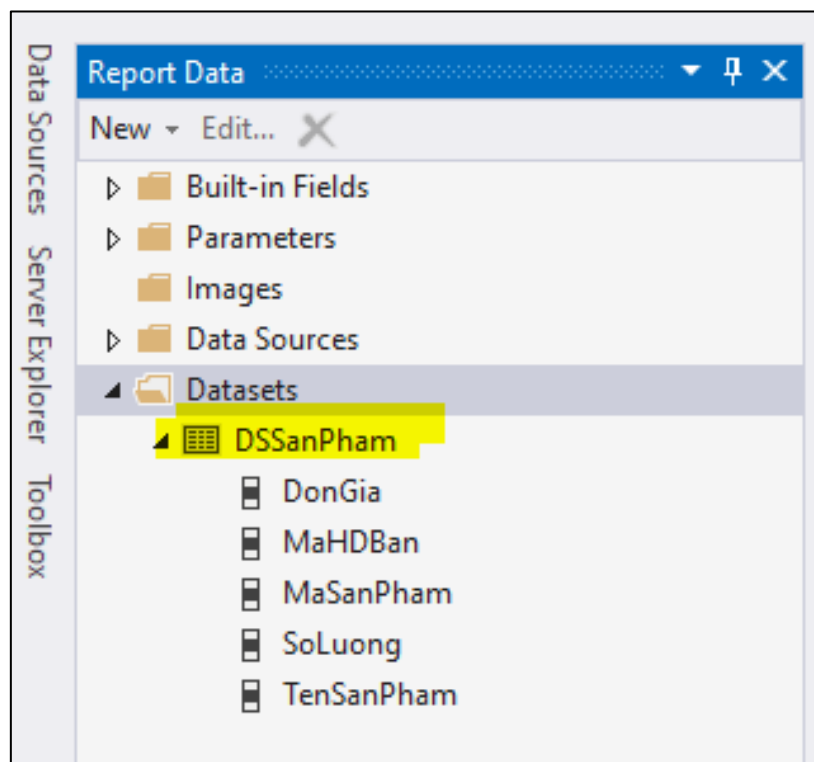
Chọn dữ liệu nguồn từ CSDL, dịch vụ hoặc đối tượng đã có trong Project



Ở đây ta chọn dữ liệu nguồn là Object nhằm chọn DTO tương ứng với dữ liệu hiển thị.



Sau khi thiết lập, DataSet sẽ hiển thị cấu trúc đối tượng dữ liệu cho Report, ta có thể kéo thả các trường dữ liệu vào Report nếu muốn hiển thị thông tin dữ liệu đó.



Ví dụ như:

HÓA ĐƠN BÁN HÀNG

Người lập: [@paNguoiLap]

Ngày lập: [@paNgayLap]

Mã HD: [@paMaHD]

STT	Sản Phẩm	Đơn Giá	Số Lượng	Thành tiền
«Expr»	[TenSanPham]	«Expr»	[SoLuong]	«Expr»
			Tổng tiền	«Expr»

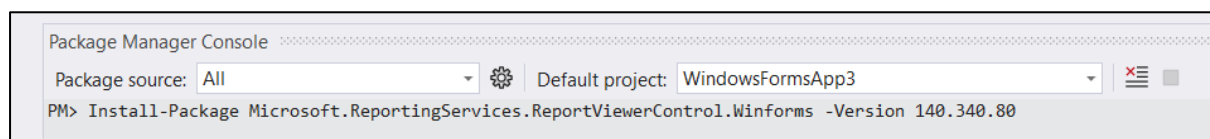
«Expr»

☐ Cài đặt thư viện cho ReportViewer để hiển thị các report

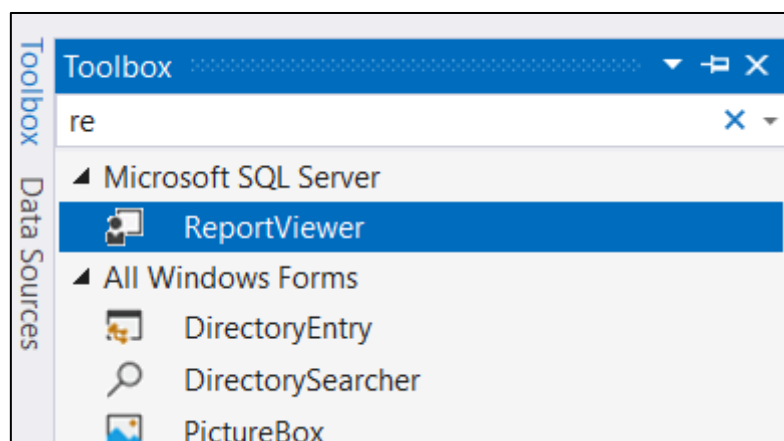
Chọn Tool → NuGet Package Manager → Package Manager Console

```
Install-Package Microsoft.ReportingServices.ReportViewerControl.WinForms -Version 140.340.80
```

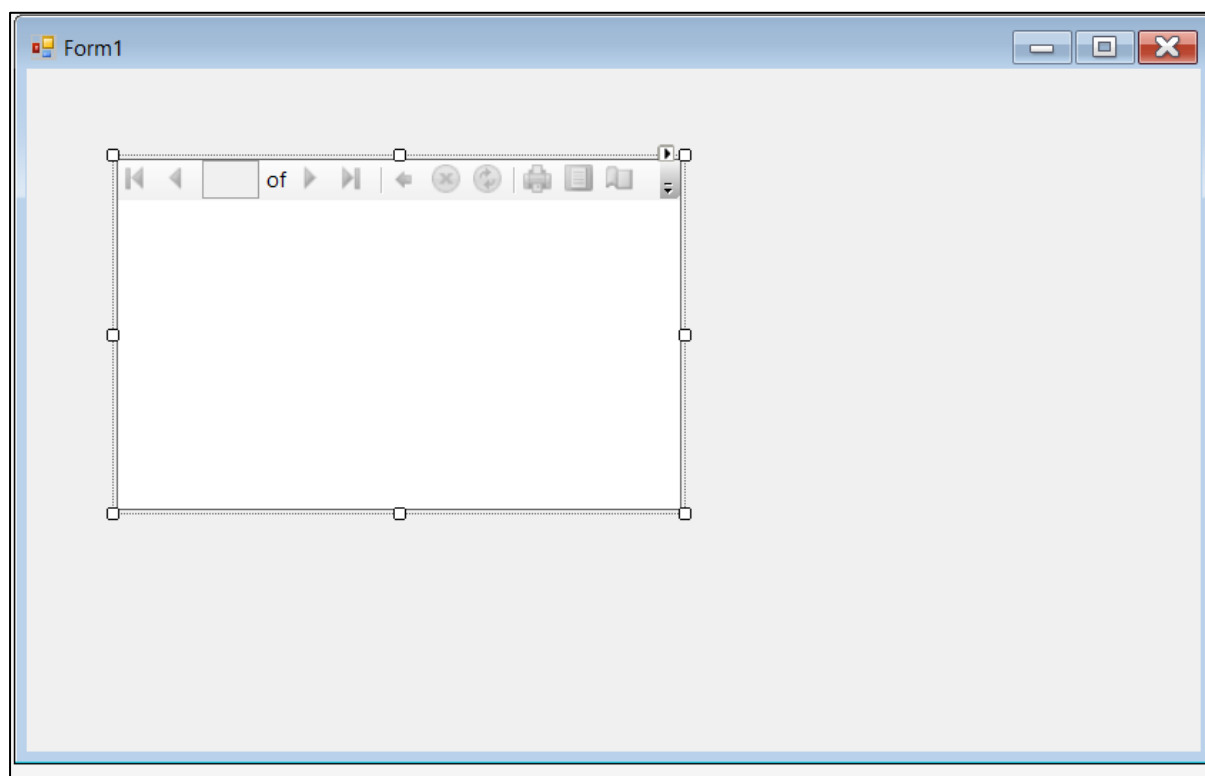
Sao chép lệnh trên vào Package Console để tiến hành cài đặt thư viện



Sử dụng công cụ ReportViewer



Kéo vào trong màn hình thiết kế hiển thị báo cáo (report)



Bài tập vận dụng

1. Sử dụng chương trình ở bài tập tuần 12 và thực hiện các yêu cầu sau:

- Xây dựng báo cáo xuất ra toàn bộ danh sách lớp
- Xây dựng báo cáo xuất ra toàn bộ danh sách lớp khoá 19
- Xây dựng báo cáo xuất ra toàn bộ danh sách sinh viên
- Xây dựng báo cáo xuất ra toàn bộ danh sách sinh viên theo Combobox lớp được chọn
- Xuất ra những sinh viên với trạng thái đã bị buộc thôi học (đã xoá)
- Xuất ra những sinh viên học lớp

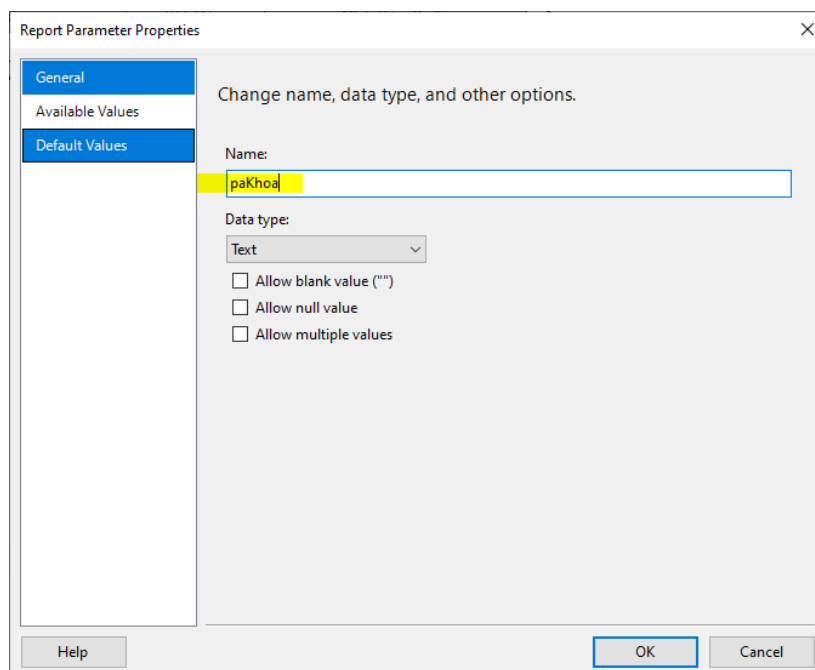
Gợi ý:

Khởi tạo các hàm và phương thức cần thiết để lấy toàn bộ thông tin lớp có trong CSDL thông qua hàm DanhSachLop() trong lớp BUS.

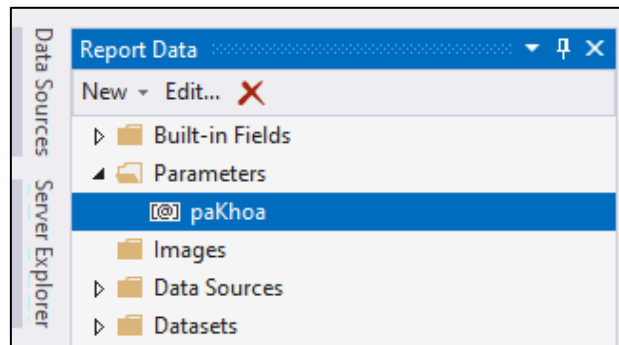
```
// Lấy danh sách lớp
List<LopDTO> lops = LopBUS.DanhSachLop();
// Chọn report hiển thị theo cấu trúc
// <tên_namespace_chứa_report>.<tên_file_report>.rdlc
rpvBaoCao.LocalReport.ReportEmbeddedResource = "QLSVGUI.rptDSLop.rdlc";
// Chỉ định DataSet và đối tượng chứa dữ liệu theo cấu trúc
// new ReportDataSource("<tên_dataset>", "<đối_tượng_chứa_dữ_liệu>")
rpvBaoCao.LocalReport.DataSources.Add(new ReportDataSource("DSLop", lops));
// Hiển thị report
this.rpvBaoCao.RefreshReport();
```

Để xuất danh sách lớp theo khoá ta cần khởi tạo biến tham số cho Report

Trang thiết kế report, click phải vào Paramaters (trong Report Data) → Add Parameter



Tham số xuất hiện dưới mục Parameters tham số đã khởi tạo paLop



Tiến hành thiết kế Report như hình sau:

DANH SÁCH LỚP THEO KHOÁ

Khoá: [@paKhoa]

Mã số	Tên Lớp
[MSSV]	[TenLop]

Thực hiện cài đặt xử lý sau:

```

List<LopDTO> lops = LopBUS.DanhSachLop();
lops = lops.FindAll(o => o.TenLop.Contain(khoa));
// Chọn report hiển thị
rpvBaoCao.LocalReport.ReportEmbeddedResource = "QLSVGUI.rptDSLop.rdlc";
rpvBaoCao.LocalReport.DataSources.Add(new ReportDataSource("DSLop", lops));
// Gán giá trị cho tham số của Report theo cấu trúc
// ReportParameter("<tên_tham_số>", <giá_trị>)
List<ReportParameter> ls = new List<ReportParameter>();
ls.Add(new ReportParameter("paKhoa", khoa));
rpvBaoCao.LocalReport.SetParameters(ls);
// Hiển thị report
this.rpvBaoCao.RefreshReport();
  
```

Bài tập mở rộng (dự án)

Sinh viên tái sử dụng project trong bài tập mở rộng ở bài tập thực hành buổi trước và thực hiện các yêu cầu:

1. Thống kê doanh danh sách hoá đơn theo ngày, tháng, năm, khoảng thời gian.
2. Thống kê doanh thu theo ngày, tuần, tháng, quý, năm

BÀI TẬP THỰC HÀNH TUẦN 14

REPORT (2)



Mục tiêu

- Nắm kiến thức về yêu cầu cần thiết khi xây dựng báo cáo (report) với subreport và expression.
- Sử dụng được báo cáo trong việc thống kê các số liệu cho phần mềm quản lý.

Kiến thức

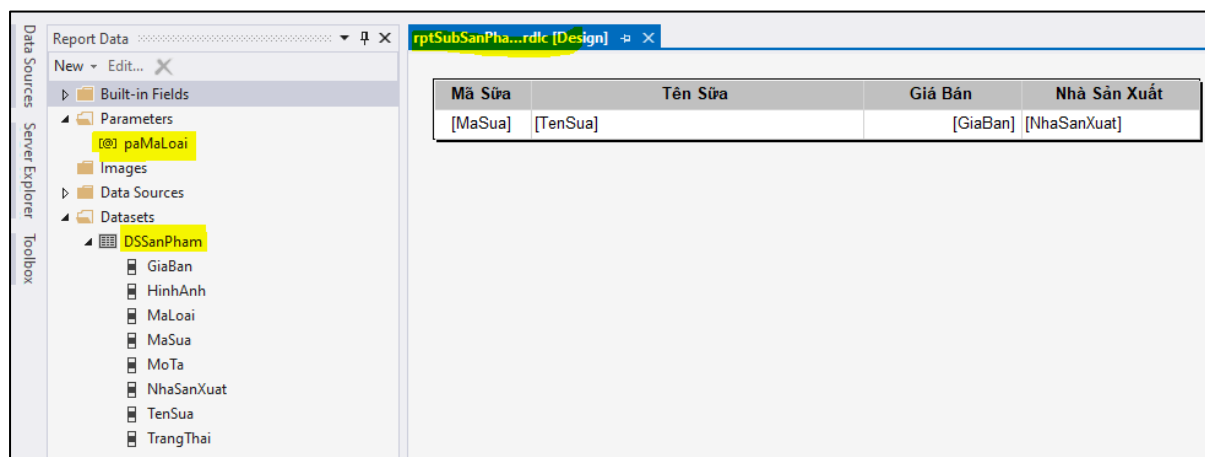
❑ **SubReport** là các report nằm bên trong một report khác cho phép hiển thị report tạo ra danh sách các dữ liệu được phân nhóm theo yêu cầu

Ta cần tạo ra các tham số (parameter) giữa hai report nhằm tạo tính liên kết cho việc truy vấn và hiển thị đúng dữ liệu.

❑ **Expression** cho phép tạo các công thức tính toán được sử dụng trong report nhằm hiển thị các dữ liệu đặc trưng cho report đó.

❑ **Ví dụ 01 (subreport):** tạo report hiển thị các danh sách sản phẩm sữa theo loại

Khởi tạo biến loại sữa trong thiết kế subreport hiển thị danh sách sản phẩm sữa



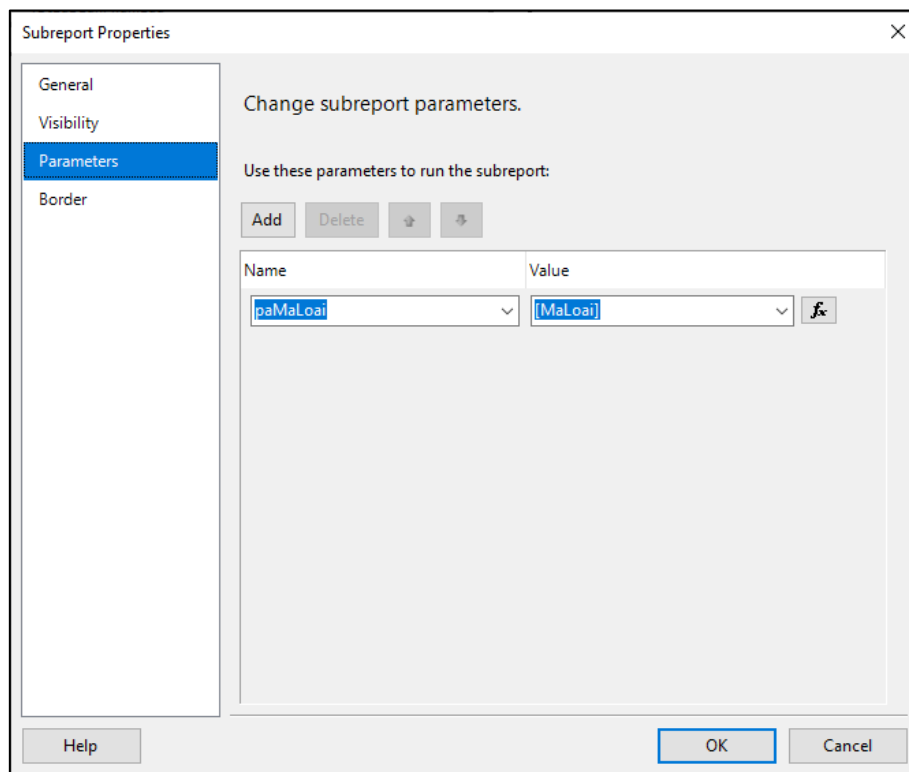
Trong report cha chứa subreport khởi tạo subreport tương ứng với report cần hiển thị



Trong đó, thiết lập subreport chứa tham số cần truyền.

Click phải vào subreport → Subreport Properties → Parameters

Chỉ định tên tham số trong subreport tương ứng với giá trị của thuộc tính trong report



Xây dựng hàm

```
lsSanPhamSua = SanPhamSuaBUS.DanhSachSanPhamSua();
List<LoaiSuaDTO> lsLoai = LoaiSuaBUS.DanhSachLoaiSua();

// Chọn report hiển thị
rpvBaoCao.LocalReport.ReportEmbeddedResource = "QLCuaHangSuaGUI.rptGroupSanPhamSua.rdlc";
rpvBaoCao.LocalReport.SubreportProcessing+=new SubreportProcessingEventHandler(Local_Sub_SPSua);
rpvBaoCao.LocalReport.DataSources.Add(new ReportDataSource("DSLoaiSua", lsLoai));
```

```
// Hiển thị report
this.rpvBaoCao.RefreshReport();
```

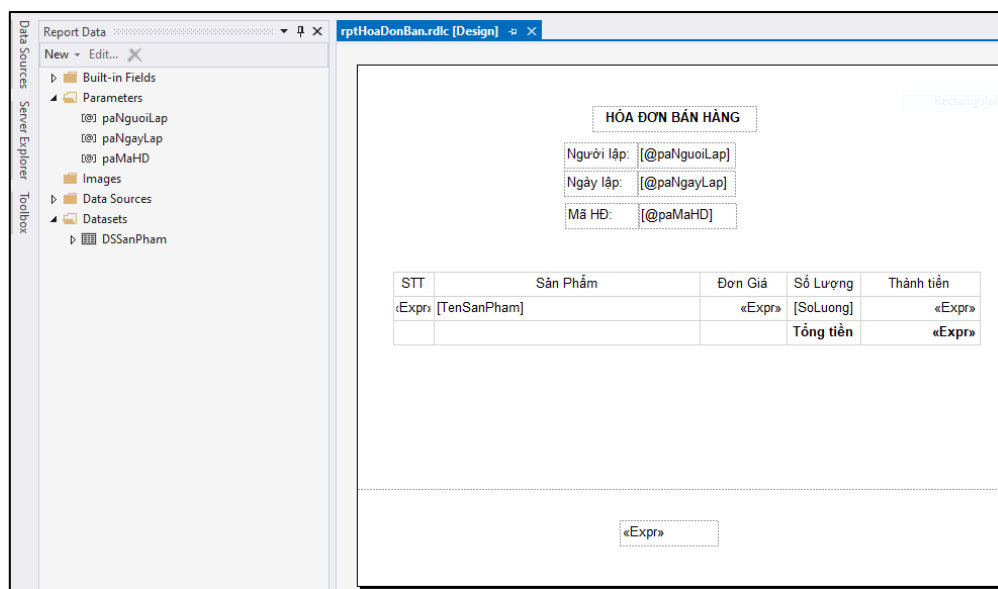
Viết hàm xử lý cho subreport

```
// Xử lý dữ liệu cho subreport
void Local_Sub_SPSua(object sender, SubreportProcessingEventArgs e)
{
    // Lấy mã loại sản phẩm sửa trên dòng report cha
    int maloai = int.Parse(e.Parameters["paMaLoai"].Values[0]);
    List<SanPhamSuaDTO> lsSuaTheoLoai = lsSanPhamSua.FindAll(o=>o.MaLoai == maloai);

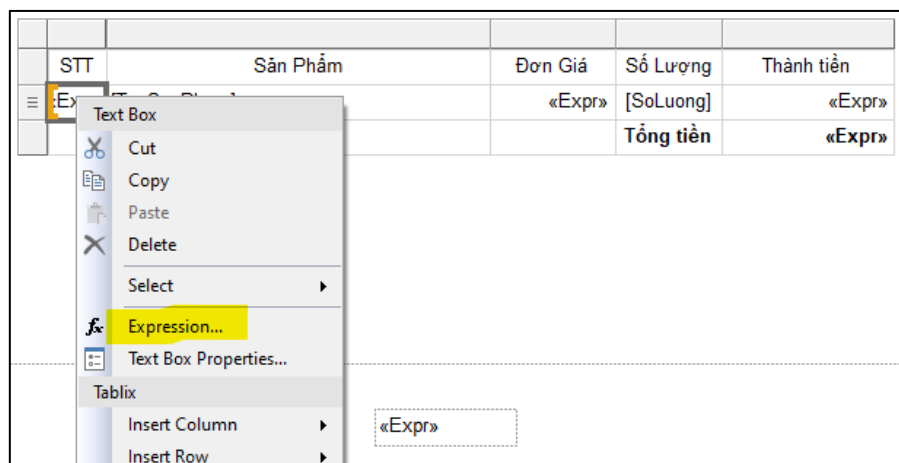
    // Đổ dữ liệu subreport
    e.DataSources.Add(new ReportDataSource("DSSanPham", lsSuaTheoLoai));
}
```

❑ **Ví dụ 02 (expression):** tạo report hoá đơn tính tổng tiền của hoá đơn đó

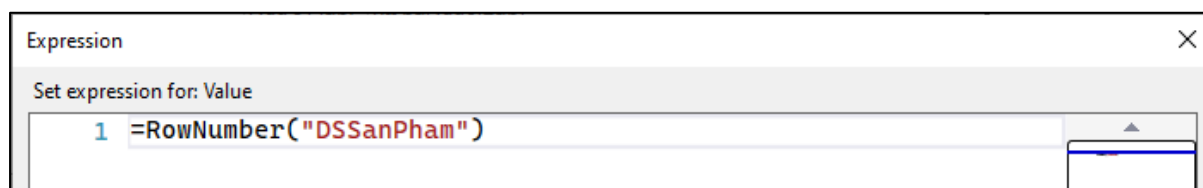
Khởi tạo report hoá đơn chứa thông tin về hoá đơn đó, trong đó, khởi tạo các tham số về người lập, ngày lập, mã hoá đơn, bên cạnh đó khởi tạo các biểu thức cho cột số thứ tự, cột đơn giá, cột thành tiền cho từng sản phẩm (số lượng x đơn giá), tổng tiền (tổng các thành tiền của từng sản phẩm) và thiết kế cấu trúc hoá đơn như hình sau:



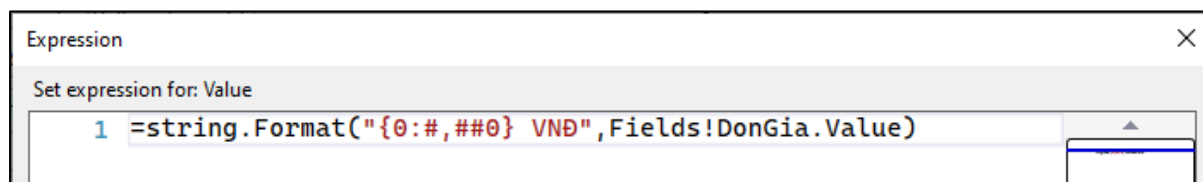
Click phải vào ô trong bảng dữ liệu của report → Expression để thiết lập biểu thức



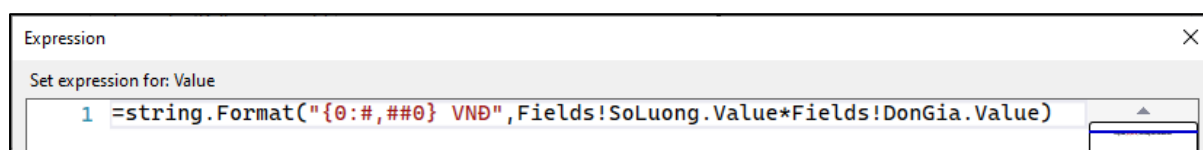
Biểu thức cho cột số thứ tự



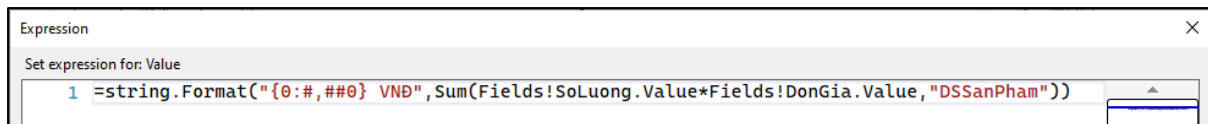
Biểu thức cho cột đơn giá



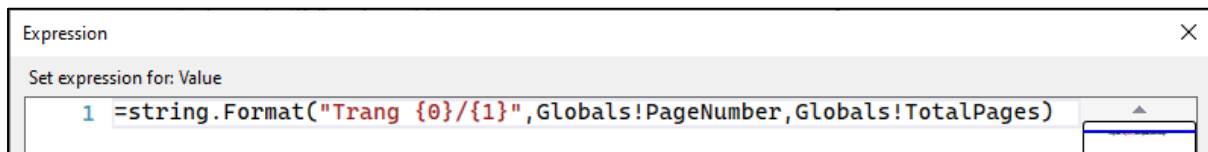
Biểu thức cho cột thành tiền



Biểu thức cho giá trị tổng tiền của hoá đơn



Biểu thức cho số trang báo cáo



Thực hiện xử lý cho việc đổ dữ liệu vào report

```
internal void InHoaDon(HoaDonBanDTO hdhientai, List<ChiTietHDBanDTO> lsCTHDBan)
{
    //chọn report hiển thị
    rpvBaoCao.LocalReport.ReportEmbeddedResource = "QLCuaHangSuaGUI.rptHoaDonBan.rdlc";
    rpvBaoCao.LocalReport.DataSources.Add(new ReportDataSource("DSSanPham", lsCTHDBan));
    rpvBaoCao.LocalReport.SetParameters(new ReportParameter("paNguoiLap", hdhientai.MaNV));

    rpvBaoCao.LocalReport.SetParameters(new
    ReportParameter("paNgayLap", hdhientai.NgayLap.ToString("dd/MM/yyyy")));

    rpvBaoCao.LocalReport.SetParameters(new ReportParameter("paMaHD",
    hdhientai.MaHDBan.ToString()));

    // Hiển thị report
    this.rpvBaoCao.RefreshReport();
}
```

Bài tập vận dụng

1. Sử dụng chương trình ở bài tập tuần 13 và thực hiện các yêu cầu sau:

- Xây dựng báo cáo xuất danh sách kèm danh sách sinh viên ở mỗi lớp, bên cạnh đó hiển thị số lượng sinh viên ở mỗi lớp, hiển thị cột số thứ tự, số trang của report.
- Xây dựng báo cáo danh sách nhóm sinh viên đang theo học hoặc buộc thôi học dựa vào thuộc tính trạng thái. Cho biết thêm các thông tin về số thứ tự, số trang, số sinh viên ở từng nhóm.

2. Xây dựng báo cáo xuất ra danh sách sinh viên, ứng với mỗi sinh viên xuất ra thông tin về lớp mà sinh viên đó thuộc về (mã lớp, tên lớp)

3. Xây dựng báo cáo xuất ra các năm học, ứng với mỗi năm xuất ra danh sách sinh viên cùng thông tin số lượng sinh viên. Biết rằng, năm học căn cứ vào cấu trúc MSSV.

Bài tập mở rộng (dự án)

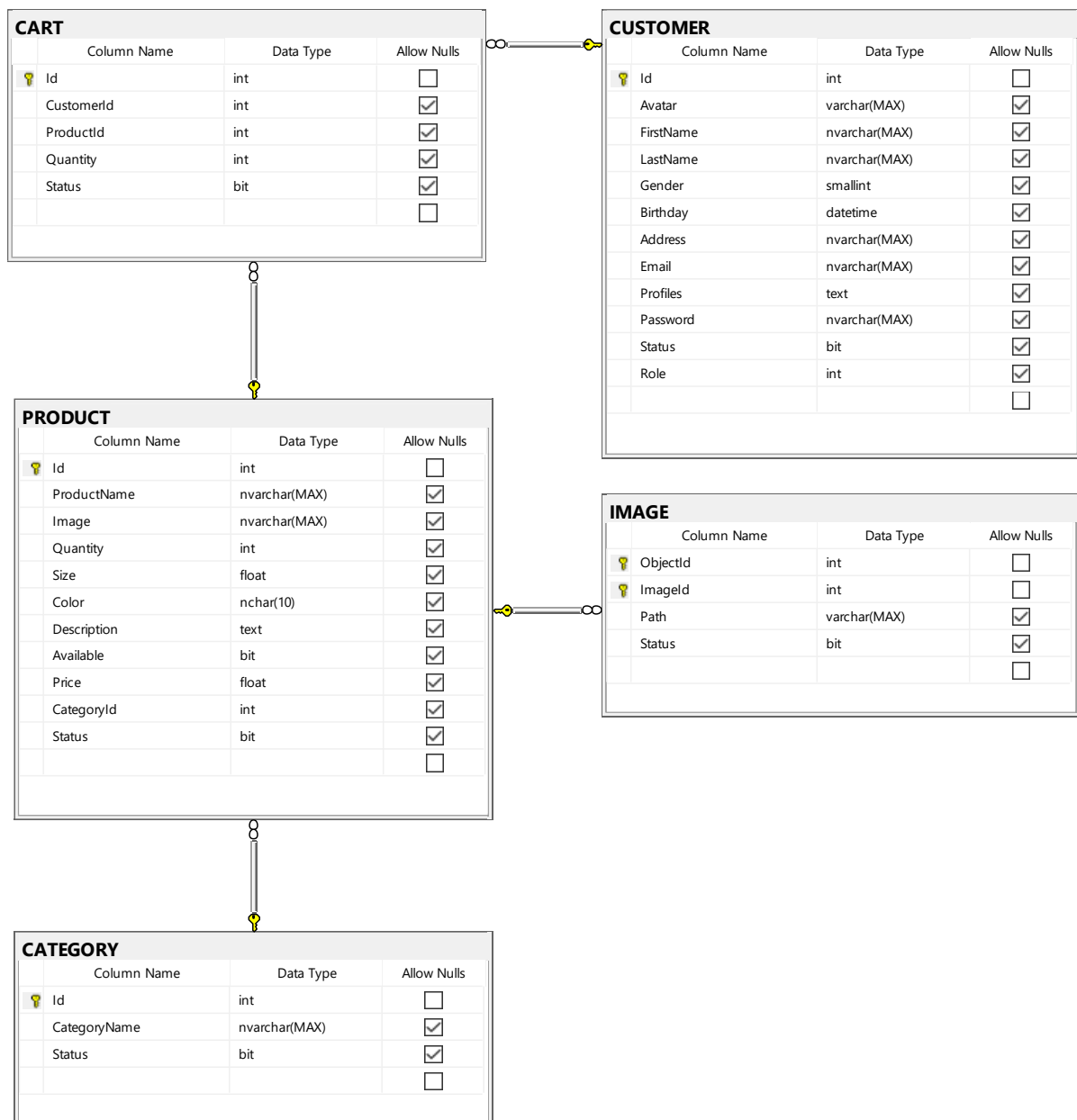
Sinh viên tái sử dụng project trong bài tập mở rộng ở bài tập thực hành buổi trước và thực hiện các yêu cầu:

1. Thống kê doanh danh sách sản phẩm theo loại sản phẩm.
2. Thống kê danh sách hoá đơn theo khách hàng theo ngày, tháng, năm.

KIỂM TRA LẦN 02



Cho lược đồ CSDL như hình vẽ sau:



Thực hiện việc nhập dữ liệu ít nhất 5 dòng cho từng bảng dữ liệu trên. Sinh viên sử dụng CSDL cho việc thực hiện các bài kiểm tra mẫu dưới đây.

ĐỀ MẪU SỐ 01 (45 phút)

Cho cơ sở dữ liệu QL_BanHang được lưu trữ trong tập tin QL_BanHang (role).sql

Xây dựng project với tên là MSSV đặt tại ổ đĩa D:\ máy tính. Thiết kế kiến trúc theo mô hình 03 lớp, sử dụng tùy ý kỹ thuật kết nối và truy vấn CSDL. Sinh viên thực hiện các yêu cầu sau:

1. Xây dựng chức năng quản lý loại sản phẩm (CATEGORY), cho phép thực hiện các thao tác thêm – xoá – cập nhật thông tin. Khi xoá cần kiểm tra nếu có loại sản phẩm đang tồn tại trong sản phẩm nào đó, hiển thị thông báo “Loại sản phẩm đang được sử dụng, không thể xoá!” cho người dùng và huỷ thao tác xoá.

Chú ý: Sinh viên được phép sử dụng DataProvider (đã tự xây dựng trước), cần thiết kế giao diện chỉnh chu, hợp lý, tiện ích cho người sử dụng.

ĐỀ MẪU SỐ 02 (45 phút)

Cho cơ sở dữ liệu QL_BanHang được lưu trữ trong tập tin QL_BanHang (role).sql

Xây dựng project với tên là MSSV đặt tại ổ đĩa D:\ máy tính. Thiết kế kiến trúc theo mô hình 03 lớp, sử dụng tùy ý kỹ thuật kết nối và truy vấn CSDL. Sinh viên thực hiện các yêu cầu sau:

1. Xây dựng chức năng quản lý khách hàng, cho phép thực hiện các thao tác thêm – xoá – cập nhật tất cả thông tin khách hàng với yêu cầu kiểm tra dữ liệu trong tầng BUS: đúng định dạng email, mật khẩu phải từ 8 ký tự trở lên và có ít nhất một ký tự hoa, ngày sinh không được lớn hơn ngày hiện tại. Khi xoá khách hàng cần kiểm tra nếu có khách hàng tồn tại trong giỏ hàng (CART), hiển thị thông báo “Khách hàng đang mua hàng, không thể xoá!” cho người dùng và huỷ thao tác xoá.

Chú ý: Sinh viên được phép sử dụng DataProvider (đã tự xây dựng trước), cần thiết kế giao diện chỉnh chu, hợp lý, tiện ích cho người sử dụng.

ĐỀ MẪU SỐ 03 (45 phút)

Cho cơ sở dữ liệu QL_BanHang được lưu trữ trong tập tin QL_BanHang (role).sql

Xây dựng project với tên là MSSV đặt tại ổ đĩa D:\ máy tính. Thiết kế kiến trúc theo mô hình 03 lớp, sử dụng tùy ý kỹ thuật kết nối và truy vấn CSDL. Sinh viên thực hiện chức năng quản lý loại sản phẩm với các yêu cầu sau:

1. Hiển thị danh sách loại sản phẩm (2 điểm)
2. Thêm thành công loại sản phẩm mới, không được trùng tên (2.5 điểm)
3. Cập nhật thông tin loại sản phẩm, không được trùng tên (2.5 điểm)
4. Xóa loại sản phẩm không có sản phẩm nào thuộc về (2.5 điểm)
5. Giao diện chỉnh chu (0.5 điểm)

Chú ý: Sinh viên được phép sử dụng DataProvider (đã tự xây dựng trước), cần thiết kế giao diện chỉnh chu, hợp lý, tiện ích cho người sử dụng.

ĐỀ MẪU SỐ 04 (60 phút)

Cho cơ sở dữ liệu QL_BanHang được lưu trữ trong tập tin QL_BanHang (role).sql

Xây dựng project với tên là MSSV đặt tại ổ đĩa D:\ máy tính. Thiết kế kiến trúc theo mô hình 03 lớp, sử dụng tùy ý kỹ thuật kết nối và truy vấn CSDL. Sinh viên thực hiện các yêu cầu sau:

1. Xây dựng chức năng đăng nhập dựa vào trường Email và Password trong bảng CUSTOMER. Trong đó, giao diện cần kiểm tra các control nhập liệu bắt buộc không được để trống, email cần đúng định dạng. Đăng nhập thành công hiển thị thông báo, sau đó vào màn hình chính của ứng dụng.
2. Màn hình chính ứng dụng là màn hình quản lý sản phẩm với các chức năng thêm – xóa – cập nhật tất cả thông tin sản phẩm. Trong đó, màn hình hiển thị danh sách tất cả sản phẩm được đánh trạng thái tồn tại trong CSDL; khi xóa sản phẩm, cần kiểm tra nếu sản phẩm đã tồn tại trong giỏ hàng (CART), hiển thị thông báo “Sản phẩm đang được sử dụng, không thể xóa!” cho người dùng và hủy thao tác xóa.

Chú ý: Sinh viên được phép sử dụng DataProvider (đã tự xây dựng trước), cần thiết kế giao diện chỉnh chu, hợp lý, tiện ích cho người sử dụng.

LỜI KẾT

Thay mặt nhóm biên soạn giáo trình Thực tập Lập trình trên môi trường Windows

Xin cảm ơn các bạn đã lựa chọn và sử dụng giáo trình này, mong rằng giáo trình là một quyển sách bổ ích và là hành trang cho các bạn trong môn Lập trình trên môi trường Windows. Bên cạnh đó, nhóm biên soạn cũng rất mong nhận được đóng góp chia sẻ từ chính các bạn sinh viên cùng quý thầy cô đã và đang sử dụng trong các vấn đề về tính rõ ràng dễ hiểu trong quá trình thực hành thực tế ở các buổi lên lớp cũng như mong muốn của các bạn đóng góp thêm về các kỹ thuật, kiến thức mới để nhóm có thể bổ sung, chỉnh sửa và hoàn thiện hơn trong tương lai, đây chính là một hành động, nghĩa cử cao đẹp nhằm giúp các bạn sinh viên khoá sau nhận được sự hướng dẫn rõ ràng hơn, chính xác hơn và cập nhật kịp thời với diễn tiến khôn lường của thời đại.

Mọi sự đóng góp xin gửi thư về địa chỉ: cntt@caothang.edu.vn

Xin chân thành cảm ơn,
Nhóm biên soạn giáo trình